

基于形式化模型的晶圆制造系统调度问题的研究

作者姓名 高霖

学校导师姓名、职称 刘鼎 副教授

企业导师姓名、职称 韦伟 高工

申请学位类别 电子信息硕士

学校代码 10701
分 类 号 TP271

学 号 22041212639
密 级 公开

西安电子科技大学

硕士学位论文

基于形式化模型的晶圆制造系统调度问题的研究

作者姓名：高霖

领 域：控制工程

学位类别：电子信息硕士

学校导师姓名、职称：刘鼎 副教授

企业导师姓名、职称：韦伟 高工

学 院：机电工程学院

提交日期：2025 年 05 月

Research on the Scheduling Problem of Wafer Manufacturing System Based on Formal Models

A thesis submitted to
XIDIAN UNIVERSITY
in partial fulfillment of the requirements
for the degree of Master
in Electronic Information

By

Lin GAO

Supervisor: Ding Liu Title: Associate Professor

Supervisor: WEI wei Title: Senior Engineer

May 2025

西安电子科技大学 学位论文独创性（或创新性）声明

秉承学校严谨的学风和优良的科学道德，本人声明所呈交的论文是我个人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢中所罗列的内容以外，论文中不包含其他人已经发表或撰写过的研究成果；也不包含为获得西安电子科技大学或其它教育机构的学位或证书而使用过的材料。与我一同工作的同事对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

学位论文若有不实之处，本人承担一切法律责任。

本人签名：_____ 日 期：_____

西安电子科技大学 关于论文使用授权的说明

本人完全了解西安电子科技大学有关保留和使用学位论文的规定，即：研究生在校攻读学位期间论文工作的知识产权属于西安电子科技大学。学校有权保留并向国家有关部门或机构送交学位论文的复印件和电子版，以学术交流为目的赠送和交换学位论文，允许学位论文被查阅、借阅和复印，将学位论文的全部或部分内容编入有关数据库进行检索和提供相应阅览服务；允许采用影印、缩印或其它复制手段保存学位论文。同时本人保证，结合学位论文研究成果完成的论文、发明专利等成果，署名单位为西安电子科技大学。

本人保证遵守上述规定。

（保密的论文在解密后遵守此规定）

本人签名：_____ 导师签名：_____

日 期：_____ 日 期：_____

摘 要

半导体制造作为现代工业体系的核心驱动力，其晶圆加工系统中的组合设备调度问题因涉及多工序集成、资源竞争和严格时序约束而异常复杂。该问题直接影响生产周期、设备利用率和产品质量，成为提升生产效率、降低制造成本和保障产品稳定性的关键瓶颈。因此，研究组合设备调度问题不仅对推动半导体产业技术进步具有重要意义，还为实现智能制造提供了重要的理论支撑和实践价值。

形式化模型通过数学和逻辑手段精确定义系统的结构与行为，能够在制造系统建模中准确刻画动态过程、约束条件和资源交互，从而为系统验证与优化提供严谨的理论依据。本文围绕离散事件系统中的两种形式化模型——时间 Petri 网与时间自动机展开研究，分别使用强化学习与模型检测对其调度问题进行研究，旨在实现组合设备的高效调度与形式化验证。主要研究内容如下：

1、本文基于已有的组合设备库所变迁时延 Petri 网 (PTTPN) 模型，并针对实际系统深入探讨了其建模思路及动态行为刻画能力。PTTPN 通过结合库所与变迁的时间约束，能够精确描述组合设备的加工流程与资源调度关系。基于组合设备的 PTTPN 模型，本文设计了以 Q-learning 和深度 Q 网络 (DQN) 为核心的强化学习算法，通过定义状态空间、动作集及奖励函数，训练智能体生成最小化工期的调度策略；为应对状态空间爆炸问题，DQN 引入深度神经网络和经验回放机制，显著提升了算法在高维复杂场景下的适应性与收敛效率。针对基础奖励函数可能导致局部最优或调度结果不理想的问题，本文进一步设计了加权组合奖励函数，通过平衡各奖励项对调度策略的贡献并引导中间状态，有效避免了局部最优和训练失败的风险。

2、提出了一种基于时间自动机的组合设备建模方法，针对实际制造系统中组合设备多工序协同与资源共享的特点，构建了加工流程、机械手、资源场所以及系统状态等子自动机模型。各子自动机分别描述各自模块的工作逻辑和时间约束，并通过设计同步通道和共享全局变量，实现了各自动机间的协调与并行工作。多个子自动机构成的时间自动机网络能够完整模拟实际组合设备在动态环境下的运行状态和逻辑交互，为后续形式化验证及其调度方案的研究奠定了坚实的基础。

3、在完成组合设备的时间自动机模型构建后，本文利用 UPPAAL 模型检测工具搭建了相应的自动机网络。通过时间计算树逻辑 (TCTL) 对系统的可调度性、安全性及最优性等关键性质进行形式化描述，并借助 UPPAAL 的诊断路径生成功能，提取时间最优的调度方案。这一过程不仅确保了调度策略的形式化验证，还通过穷尽式模型检测遍历整个状态空间，保证了调度方案的全局最优性。

本文以组合设备的 Petri 网和时间自动机两种形式化模型为基础，分别采用强化

学习算法与模型检测方法研究调度问题。其中，强化学习算法能够高效应对复杂调度情境，而模型检测方法则为生成方案的正确性与最优性提供有力保障。实验结果表明，无论是 Q-learning、DQN 还是模型检测方法，在组合设备调度问题上均取得良好效果，其调度结果均优于传统算法。此外，通过对基础单步惩罚奖励函数进行改进，引入多因素加权组合，不仅显著提高了算法的收敛性能，也进一步优化了调度效果。

关键词：组合设备，时间 Petri 网，时间自动机，强化学习，模型检测

ABSTRACT

Semiconductor manufacturing, a cornerstone of modern industry, faces significant challenges in scheduling cluster tool systems due to its complex multi-process integration, resource contention, and stringent temporal constraints. Efficient scheduling is critical for optimizing production cycles, equipment utilization, and product quality, directly impacting cost reduction and manufacturing reliability. Addressing cluster tool scheduling holds significant importance for advancing semiconductor technology and provides theoretical and practical foundations for intelligent manufacturing.

Formal modeling employs mathematical and logical frameworks to precisely define system structures and behaviors, enabling accurate characterization of dynamic processes, constraints, and resource interactions in manufacturing systems. This study investigates two formal models in discrete event systems—Timed Petri Nets and Timed Automata—to develop scheduling methods integrating reinforcement learning and model checking. The main contributions are outlined as follows:

1. Based on existing Place-Transition Timed Petri Net (PTTPN) models of cluster tools, this work analyzes its modeling principles and dynamic behavior characterization for cluster tools. By integrating temporal constraints in places and transitions, the model accurately describes processing workflows and resource scheduling. Q-learning and Deep Q-Network (DQN)-based reinforcement learning algorithms are designed with defined state spaces, action sets, and reward functions to train agents in generating makespan-minimized scheduling policies. To address state space explosion, DQN incorporates deep neural networks and experience replay mechanisms, significantly enhancing adaptability and convergence efficiency in high-dimensional scenarios. A weighted composite reward function is further proposed to mitigate local optima and training failures by balancing reward components and guiding intermediate states.
2. A Timed Automata-based hierarchical modelling framework is proposed to represent multi-process coordination and resource sharing. Sub-automata models for processing modules, robots, and resource chambers are synchronised via shared variables and communication channels, thereby enabling holistic simulation of cluster tool operations.

3. This study leverages the UPPAAL model checker to implement the constructed Timed Automata network. Key system properties—schedulability, safety, and optimality—are formalized via Timed Computation Tree Logic (TCTL). By exploiting UPPAAL's diagnostic path generation, time-optimal scheduling solutions are systematically extracted. This process not only ensures the formal verification of scheduling strategies but also guarantees the global optimality of scheduling solutions through exhaustive model checking that traverses the entire state space.

This thesis is based on the formal models of Petri nets and timed automata for cluster tools, and it investigates the scheduling problem using reinforcement learning algorithms and model checking methods. The reinforcement learning algorithms can efficiently handle complex scheduling scenarios, while the model checking method provides assurance for the correctness and optimality of the generated scheduling schemes. Experimental results indicate that regardless of whether Q-learning, DQN, or model checking methods are employed, all yield good performance on the scheduling problem of cluster tools, with outcomes superior to those of traditional algorithms. In addition, by improving the basic single-step penalty reward function through the introduction of a weighted combination of multiple factors, the convergence performance of the algorithms is significantly enhanced and the scheduling outcomes are further optimized.

Keywords: Cluster Tools, Timed Petri Nets, Timed Automata, Reinforcement Learning, Model Checking

插图索引

图 2.1	一个 Petri 网 (N, M_0)	12
图 2.2	Petri 网发射变迁序列后的标识变化	13
图 2.3	图 2.1 Petri 网的可达图	16
图 2.4	变迁时间 Petri 网示例	18
图 2.5	库所时间 Petri 网示例	19
图 2.6	库所变迁时间 Petri 网示例	20
图 2.7	一个确定性有限状态自动机示例	22
图 2.8	时间自动机示例	23
图 2.9	强化学习中智能体与环境的交互	24
图 2.10	模型检测流程框图	26
图 3.1	组合设备示意图	27
图 3.2	一个 AMS 的 POPN 模型示例	29
图 3.3	机器 1 和机器 2 的 ROPN 子网	30
图 3.4	一个 AMS 的 ROPN 模型示例	31
图 3.5	图 3.1 组合设备的简单顺序流程 Petri 网	32
图 3.6	机械手模块示意图	32
图 3.7	加工模块 Petri 网模型	33
图 3.8	加工系统的 Petri 网模型	33
图 3.9	深度神经网络结构示意图	43
图 4.1	晶圆加工流程自动机	50
图 4.2	机械手自动机模型	51
图 4.3	需要准备时间的加工场所工作逻辑	52
图 4.4	需要准备时间的加工场所自动机	53
图 4.5	系统状态自动机	54
图 4.6	系统 UPPAAL 时间自动机网络	55
图 4.7	多组合设备示意图	55
图 4.8	$E \diamond \phi$	56
图 4.9	安全性逻辑示意图	57
图 4.10	活性逻辑示意图	57
图 4.11	Fischer 协议自动机	58

图 4.12	组合设备配方三的时间自动机模型	62
图 5.1	不同待加工晶圆数量下的 Q-learning 训练曲线	66
图 5.2	不同待加工晶圆数量下的 DQN 训练曲线	68
图 5.3	算法收敛速度对比	70
图 5.4	不同奖励函数下 Q-learning 训练曲线的对比	70
图 5.5	不同奖励函数下 DQN 训练曲线的对比	71
图 5.6	不同算法调度结果对比	72
图 5.7	安全性性质验证结果	74
图 5.8	是否存在死锁性质验证详细信息	74
图 5.9	是否进入错误状态性质验证详细信息	74
图 5.10	可调度性验证详细信息	75
图 5.11	调度方案最优性验证详细信息	75
图 5.12	案例二待验证性质验证结果详细信息	76
图 5.13	算法收敛速度对比	77
图 5.14	组合设备案例二的 Petri 网模型	79

表格索引

表 2.1	图 2.7DFA 的转换函数	22
表 3.1	图 3.4 所示的 ROPN 模型中元素含义.....	31
表 3.2	图 3.8 的库所及变迁含义	34
表 4.1	$Task(i)$ 自动机变量及含义.....	51
表 4.2	需要准备时间的加工场所自动机变量及含义	52
表 4.3	Fischer 协议系统性质 TCTL 及验证结果.....	59
表 4.4	多组合设备配方三时间自动机模型变量及含义	63
表 5.1	实验环境配置	65
表 5.2	组合设备案例一不同待加工晶圆下 Q-learning 算法调度结果.....	67
表 5.3	DQN 训练参数.....	67
表 5.4	组合设备案例一不同待加工晶圆下 DQN 算法调度结果.....	68
表 5.5	组合设备案例二不同待加工晶圆下 Q-learning 算法调度结果.....	69
表 5.6	组合设备案例二不同待加工晶圆下 DQN 算法调度结果.....	69
表 5.7	案例一调度结果对比	71
表 5.8	案例二调度结果对比	72
表 5.9	案例二下训练时间和响应时间对比	72
表 5.10	组合设备待验证性质及其 TCTL	73
表 5.11	不同待加工晶圆数下的可调度性验证结果	74
表 5.12	不同待加工晶圆数下的诊断路径最优性验证结果	75
表 5.13	案例二不同待加工晶圆数下的可调度性验证结果	76
表 5.14	案例二待验证性质及结果	76
表 5.15	案例一调度结果对比	77
表 5.16	案例二调度结果对比	77
表 5.17	图 5.14 组合设备 Petri 网库所变迁含义.....	80

符号对照表

符号	符号名称
$\mathbb{N}^+$	正整数集
$\mathbb{Q}_0^+$	有理数集
F	Petri 网的有向弧集合
G	确定性有限自动机 DFA
M	Petri 网的标识
N	Petri 网
P	Petri 网的库所集合
T	Petri 网的变迁集合
W	Petri 网的有向弧集合的权重映射集合
M_0	Petri 网的初始标识
$[N]$	Petri 网 N 的关联矩阵
(N, M_0)	Petri 网系统
$R(N, M_0)$	Petri 网的可达图
Y	DFA 的状态集
Y_m	标记状态集
p	Petri 网的库所
t	Petri 网的变迁
y_0	DFA 的初始状态
α	学习率
γ	折扣因子
ϵ	探索因子
η	DFA 的部分转换函数
Σ	DFA 的事件集
$\vec{\sigma}$	变迁序列
ω_1	加权奖励函数进度权值

ω_2	加权奖励函数设备利用率权值
$\bullet x$	节点 x 的前置集
$x\bullet$	节点 x 的后置集

缩略语对照表

缩略语	英文全称	中文对照
A3C	Asynchronous Advantage Actor-Critic	异步优势演员-评论家
AMS	Automated Manufacturing System	自动制造系统
B&B	Branch and Bound	分支定界法
BIP	Binary Integer Programming	二进制整数规划
DES	Discrete Event System	离散事件系统
DFA	Deterministic Finite Automaton	确定性有限自动机
DQN	Deep Q Network	深度 Q 网络
FSM	Finite State Machine	有限状态机
MDP	Markov Decision Process	马尔科夫决策过程
MIP	Mixed Integer Programming	混合整数规划
NFA	Nondeterministic Finite Automaton	非确定性有限自动机
NP	Non-deterministic Polynomial-time	非确定性多项式时间
POP	Process-Oriented Petri Nets	面向过程的 Petri 网
PTPN	Place Time Petri Net	库所时间 Petri 网
PTTPN	Place Transition Time Petri Net	库所变迁时延 Petri 网
ROP	Resource-Oriented Petri Nets	面向资源的 Petri 网
TCTL	Timed Computation Tree Logic	时间计算树逻辑
TTPN	Transition Time Petri Net	变迁时间 Petri 网

目 录

摘要.....	I
ABSTRACT.....	III
插图索引.....	V
表格索引.....	VII
符号对照表	IX
缩略语对照表	XI
第一章 绪论	1
1.1 研究背景和意义	1
1.2 国内外研究现状	3
1.2.1 组合设备建模方法研究现状	3
1.2.2 调度算法研究现状	5
1.3 本文主要研究内容	7
1.4 本文结构.....	8
第二章 相关基础理论和方法	11
2.1 Petri 网及时间 Petri 网基础概念	11
2.1.1 Petri 网基础概念	11
2.1.2 时间 Petri 网基础概念	17
2.2 自动机及时间自动机基础概念	21
2.2.1 自动机基础概念	21
2.2.2 时间自动机基础概念	22
2.3 强化学习	23
2.4 模型检测.....	25
2.5 本章小结.....	26
第三章 基于时间 Petri 网和强化学习算法的组合设备调度	27
3.1 组合设备的 Petri 网建模.....	27
3.1.1 组合设备.....	27
3.1.2 Petri 网建模方法	28
3.1.3 组合设备的 Petri 网建模	31
3.2 基于 Petri 网和 Q-learning 算法的组合设备调度算法	34
3.2.1 Q-learning 算法	34
3.2.2 基于 Q-learning 算法的组合设备调度算法设计	36

3.3	基于 Petri 网和 DQN 算法的组合设备调度算法	37
3.3.1	DQN 算法	38
3.3.2	基于 DQN 算法的组合设备调度算法设计	39
3.3.3	加权组合奖励函数	43
3.4	本章小结	45
第四章	基于时间自动机和模型检测方法的组合设备调度	47
4.1	UPPAAL 模型检测工具	47
4.2	组合设备的时间自动机建模	49
4.2.1	加工任务流程自动机	50
4.2.2	机械手自动机	50
4.2.3	资源场所自动机	52
4.2.4	系统状态自动机	54
4.3	基于模型检测的组合设备调度方法	56
4.3.1	UPPAAL 查询语言	56
4.3.2	组合设备的性质验证及调度方案	59
4.4	本章小结	61
第五章	实验结果与分析	65
5.1	组合设备案例介绍	65
5.2	基于时间 Petri 网和强化学习算法的组合设备调度实验结果	65
5.2.1	基础奖励函数	66
5.2.2	加权组合奖励函数	70
5.2.3	比较和分析	71
5.3	基于时间自动机和模型检测方法的组合设备调度实验结果	73
5.3.1	组合设备案例一	73
5.3.2	组合设备案例二	76
5.3.3	比较与分析	76
5.4	本章小结	78
第六章	总结与展望	83
6.1	总结	83
6.2	展望	84
	参考文献	87
	致谢	93
	作者简介	95

第一章 绪论

本章首先阐述了半导体制造业晶圆加工系统中组合设备调度问题的研究背景和意义；随后，从组合设备建模方式和调度算法两方面对相关研究现状进行了综述；最后，介绍了本文的研究内容、创新工作以及文章的整体结构安排。

1.1 研究背景和意义

作为现代工业体系的核心驱动力，半导体技术已深度渗透至通信、计算、医疗及国防等关键领域，其衍生的智能自动化、先进机器人系统与深度人工智能等突破性技术，正重塑全球产业格局^[1]。在我国现代化建设进程中，半导体产业不仅构成数字经济发展的战略支柱，更成为保障数字主权、提升国际科技话语权的核心要素。近年国际技术贸易壁垒的强化已形成显著的产业制约，这使得突破关键核心技术瓶颈、构建自主可控的产业生态体系上升为国家战略层面的紧迫课题^[2]。

在半导体制造工艺链中，晶圆作为基础载体材料，其制造过程集中体现了现代微电子工业的技术复杂度。由超高纯度单晶硅棒经精密切割而成的晶圆基板，需经历包括纳米级图形转移、原子层沉积、量子级掺杂等在内的 200 余道精密制造工序^[3]。特别是在三维集成技术快速发展的背景下，晶圆表面形成的多层互连结构已突破百层量级，单个 300mm 晶圆可集成超过万亿个晶体管元件。这种以原子尺度制造精度为特征的加工体系，直接决定了集成电路的运算效能与能耗表现，其中 FinFET 结构中的栅极宽度控制精度已达到亚纳米级别，充分彰显了现代半导体制造技术的极致化发展趋势^[4]。

在半导体制造工艺不断升级、精度要求日益提高的背景下，传统单一功能设备已难以满足各工序高效衔接与精准制造的需求，组合设备因此应运而生^[5]。这类设备将纳米级图形转移、原子层沉积、量子级掺杂等多道高精度工序有机整合在一台设备中，从而大幅提高了制造效率和集成度。然而，正是这种多工序集成的特点，也使得组合设备在调度上呈现出前所未有的复杂性和挑战性。

组合设备在制造过程中既要保证各单元工序的精准对接，又要协调各工序之间的时序安排，确保整体系统的高效运行。设备调度问题不仅涉及到工序之间的顺序优化，还需要兼顾设备利用率、加工精度以及工艺间相互影响等多重因素^[6]。因此，如何制定合理的调度策略、平衡各项制造指标，成为实现工艺优化和提高产品良率的关键所在。

面对组合设备调度问题中多工序并行、事件驱动和严格时序要求等复杂特性，传

统的优化方法（如整数规划、启发式算法等）在描述系统动态和处理时间约束方面往往存在局限。而离散事件系统（Discrete Event Systems, DES）作为专门用于建模和分析离散动态行为的数学框架，能够精准刻画系统在离散时刻发生的状态变化和事件触发过程^[7]。DES 适用于所有状态有限、事件驱动的系统，能够处理并发、同步、竞争和资源共享等复杂调度问题。它通过定义系统的状态、事件及其转换关系来刻画设备的运行逻辑，并采用事件驱动机制确保调度方案的动态适应性，从而为优化调度策略提供了坚实的理论支撑。

对于自动制造系统调度问题，通常分为系统建模与调度算法两个部分。在系统建模方面，离散事件系统为研究提供了坚实的理论支撑，其常用方法包括 Petri 网和自动机模型。Petri 网因其直观的图形化表示和并发建模能力，在自动制造系统中被广泛应用^[8]。例如，在晶圆制造中，组合设备需要协调多个工序、合理分配共享资源，并满足严格的时序约束，这些复杂性难以通过传统方法全面描述。Petri 网作为一种 DES 建模工具，利用库所、变迁和托肯等元素，直观展现工序的并发执行、资源竞争与同步机制，从而为调度策略的制定提供清晰的支持。同时，它能精确刻画各工序的状态演化、事件触发和时序依赖，有助于深入理解流程运行并预判瓶颈与死锁问题。借助可达性分析、死锁检测与性能评估等工具^[9]，调度方案在实施前可进行严谨验证，从而有效降低风险、提高设备利用率和生产效率。

自动机以状态和转换为核心，清晰刻画系统在不同操作模式间的切换。在晶圆制造组合设备调度中，自动机可准确描述各工序阶段，通过状态转换反映工序顺序、资源占用和调度约束。其层次化表达方式使得复杂逻辑能够被分解为模块化约束，并与调度算法紧密融合，适用于多种自动机模型。对于确定性调度，它能够支持有限状态机^[10]，以确保调度过程的可预测性和稳定性。针对需要探索多种方案的场景，该方法可与非确定性自动机结合^[11]，以提高调度的灵活性。与此同时，通过引入带权自动机^[12]，还能进一步优化系统性能，实现更精细化的调度决策。此外，自动机模型可结合形式化验证方法，通过可达性分析、死锁检测和最优路径搜索，确保调度方案满足关键约束。对于时间敏感的调度问题，还可扩展为时间自动机，引入时钟变量严格控制时间约束，从而进一步提升设备利用率与制造效率。

在对组合设备进行建模后，我们需要基于其模型选择合适的调度算法以实现高效生产。近年来，强化学习由于不依赖于精确的系统模型，能够通过与环境的交互不断优化调度策略，已成为解决复杂调度问题的热门方法^[13]。为此，本文采用两种强化学习算法对组合设备的 Petri 网模型进行研究，获得相应的调度方案。同时，为了确保系统调度方案的最优性，本文还利用模型检测方法对组合设备的时间自动机模型进行研究，对调度方案的最优性等性质进行验证和分析，从而提升系统的整体生产效率。

1.2 国内外研究现状

1.2.1 组合设备建模方法研究现状

在半导体制造领域，组合设备调度问题由于涉及多道高精度工序、共享资源以及严格的时序约束，呈现出极高的复杂性和动态性。因此，建立精确、有效的模型不仅是理解系统内在运行规律的关键前提，更是制定合理调度策略的重要基础。通过系统建模，可以将复杂的物理过程、工序依赖以及不确定性因素以数学或图形化形式进行抽象和表达，为后续调度算法的设计与验证提供理论支撑和实践依据。这也是国内外众多学者不断探索和改进组合设备调度模型的主要原因，其研究成果在推动生产效率提升和资源优化配置方面均发挥了至关重要的作用。目前，常用的组合设备的建模方法如下：

(1) 数学规划方法 (Mathematical programming)

数学规划方法通过构建目标函数与约束条件的数学表达式，将组合设备调度问题转化为优化问题求解^[14]，其核心在于通过线性或非线性关系描述晶圆加工时间、机械手动作序列及设备状态等要素间的关联性。Kim 等人在 2015 年针对在线多集群工具调度问题，提出分层优化方法^[15]：使用混合整数规划 (Mixed Integer Programming, MIP) 实现加工模块负载均衡，优先级规则优化机器人任务序列，并通过多模式并行加工提升系统利用率。该框架将 NP 难问题分解为可解子问题，虽受限于 MIP 的小规模求解能力，但其“精确建模 + 启发式扩展”范式为复杂制造系统调度研究提供了方法论基础。Bao 等人^[16]提出了一种基于 MIP 的多集群工具循环调度方法，首次将 MIP 扩展至多集群调度，支持双机械臂、晶圆驻留时间约束、并行/重入工艺等复杂场景，避免死锁且无需预设任务序列。通过模型重构显著降低计算复杂度，适应大规模腔室场景，实验验证其优于传统分解方法与 Petri 网模型。Qiao 等人^[17]构建了一个高效的二进制整数规划 (Binary Integer Programming, BIP) 模型，提出了一种基于虚拟晶圆的调度方法，通过在单晶圆循环调度中引入虚拟晶圆，使得在处理真实晶圆的同时，能在合适的时机进行腔室清洁，从而满足严格的工艺要求，为集群工具调度提供了一种具有较高实际应用价值的解决方案。

(2) 时序图建模

时序图建模是一种通过图形化手段描述系统内各操作或事件在时间轴上依次发生及相互关联的方法。该方法直观展现了对象或组件间的信息传递、操作执行和同步机制，从而揭示出系统的动态行为和潜在性能瓶颈，为后续的分析与优化提供了重要依据。Perkinson 等^[18]构建了组合设备各模块间的时序图模型，定量分析了单晶圆集群工具中工艺时间、传送时间与设备数量之间的关系，揭示出影响工具最大吞吐量的关键因素。研究将操作划分为不同运行区域，推导出描述稳态周期及整体加工时

间的数学公式,并据此提出了预调度、多速传送器和双负载锁等三种改进技术,在最小化现有设计修改的前提下显著提高了设备性能。随后,Perkinson 等^[19]在前期工作的基础上,进一步探讨了冗余腔室和工序重访调度对单晶圆集群工具吞吐量的影响,通过定量描述传送时间与各工序处理时间在不同调度策略下的相互作用,揭示了慢工序对整体性能的瓶颈效应。

(3) Petri 网建模

Petri 网作为 DES 的常用建模工具之一,常用于描述和分析自动制造系统的动态行为。在调度问题中,Petri 网可以用来建模任务执行的顺序、并发性、同步和资源竞争等因素,通过库所和变迁的组合,直观地展示调度流程和系统中的瓶颈^[20]。针对组合设备的 Petri 网建模,主要分为两种面向资源的 Petri 网 (Resource-oriented Petri Nets, ROPN) 和面向过程的 Petri 网 (Process-Oriented Petri Nets, POPN),前者注重资源利用,而后者则关注生产流程。

对于 POPN 建模方法,Wu^[21]针对半导体晶圆加工中多组合设备的死锁问题,提出了一种面向资源的 Petri 网建模方法。Zhu 等^[22]建立了多组合设备的 ROPN 模型,用以描述其结构及动态行为。该模型通过引入着色托肯和受控变迁,紧凑地刻画了机械手、加工模块和真空锁等各部件在初始、稳态和趋停暂态阶段的协同工作过程,并对死锁问题进行了深入分析,提出了避免死锁的控制策略。Pan 等^[23]针对半导体制造中可重入双臂组合设备的故障响应问题,提出了一种基于 Petri 网的动态调度策略。他们通过构建具有逗留时间约束的赋时 ROPN 模型,首次建立了可重入系统并行模块故障的调度分析框架,并提出融合虚拟晶圆的暂态过渡控制方法,成功解决了故障前后稳态调度模式转换的难题。

对于 POPN 建模方法,Zhou 等^[24]针对制造系统中共享资源的 Petri 网建模问题,提出了并行互斥与顺序互斥的结构化建模方法,为系统避免死锁、保持有界性、活性和可逆性提供了理论支持,其建模方式即为 POPN。而 Chen 等^[25]则针对柔性制造系统的死锁预防问题,提出了一种基于 POPN 的最大许可活性监督器设计方法。该方法通过引入向量覆盖技术,将合法标记集和首遇坏标识集分别压缩为极小覆盖集,并结合整数线性规划迭代构造位置不变量,从而确保只禁止首遇坏标识而保留所有合法状态。

(4) 自动机建模

自动机为离散事件系统的常用建模工具之一,在自动制造系统的建模中能形式化描述系统状态与转移,直观展现设备并发、资源竞争及协同工作。Shin 等^[26]针对半导体制造中双机械臂集群工具的动态调度问题,提出了一种基于有限状态机的实时调度器建模方法。通过将正常作业的周期性调度策略与异常事件的动态响应逻辑编码到有限状态机的状态转移中,实现了复杂晶圆流模式下的调度决策与实时控制一

体化。Simon 等^[27]基于 Ramadge-Wonham 监督控制框架，提出了一种面向大规模离散事件系统的时间最优合成方法，通过将时间加权模型转换为 τ 自动机，结合顺序语言投影和剪枝技术，在保持最优性的前提下有效缓解状态爆炸问题，在线性集群工具中验证了最小制造周期调度的高效性，为复杂制造系统的高效调度提供了理论工具。

1.2.2 调度算法研究现状

调度问题是一类资源受限环境下的多任务时序调配规划问题，其核心目标在于通过合理分配时间与资源，在既定约束（如截止时间、优先级规则、资源容量等）框架下，实现系统效能指标（如完工时间、资源利用率、成本等）的最优或可行解^[28]。在半导体制造领域，组合设备因兼具并行处理与串行传输的复杂特性，其调度优化已成为提升晶圆生产效率的关键瓶颈问题，近年来受到学术界与工业界的持续关注。针对此类系统的调度优化方法，主流研究可归纳为以下四类：数学优化方法、启发式算法、元启发式算法和人工智能类算法等。

(1) 数学优化方法

数学优化方法即利用数学模型和优化算法，精确求解调度问题，适用于规模较小、约束条件明确的情况^[29]。在组合设备的调度问题中，主要使用的方法为分支定界法和数学规划法。

分支定界法（Branch and Bound, B&B）是一种基于树形搜索的精确求解框架，专用于处理整数规划及组合优化中的 NP 难（Non-deterministic Polynomial-time hard, NP hard）问题。该算法通过递归分割解空间生成子问题分支，并基于线性松弛技术预估各分支的上下界，进而采用剪枝策略动态排除非优区域，显著提升算法收敛速度，确保在有限时间内获得全局最优解^[30]。Jung 等提出了一种用于解决基于时间 Petri 网的循环调度问题的分支定界算法，旨在最小化周期时间。针对非循环调度需求，Kim 等^[31]开发基于时间 Petri 网的分支定界算法，处理异构晶圆流与频繁换型问题，但其计算效率在复杂场景中仍需优化。

数学规划方法在组合设备建模部分已做介绍^[14-17]，它既是一种建模方法，也是一类优化算法。首先，数学规划方法通过构建目标函数来设定组合设备的优化目标，然后根据系统中的约束条件，确立相关限制，最后明确决策变量及其取值范围，完成建模过程。在模型建立的基础上，通过求解算法，如线性规划、非线性规划等，寻找最优解。

(2) 启发式算法

启发式算法（Heuristic Algorithm）是一种依赖经验或直觉规则来求解问题的方法，主要目的是快速找到可行解或近似最优解，特别适用于 NP 难问题或规模较大的复杂问题^[32]。与精确算法（如动态规划、分支定界法）不同，启发式算法不保证能找

到全局最优解，但能够在较短时间内提供高质量的近似解。Ying^[33]针对半导体制造中晶圆分选阶段的调度问题，提出了一种基于迭代贪心启发式算法的优化方法。该问题需要在满足截止日期和机器容量约束的前提下，以最小化总设置时间和使用的测试机数量为核心，调度多台并行测试机处理序列依赖设置时间的晶圆批次。Zhang等^[34]针对晶圆批次在并行集群工具中的分配问题提出最大重叠作为评估指标，量化晶圆批次在工具中的步骤冲突，间接反映完成时间。将问题建模为聚类优化问题，设计基于贪心搜索的聚类算法，将混合类型的晶圆批次分配到多台工具，以最小化最大重叠。

(3) 元启发式算法

元启发式算法 (Meta-heuristic Algorithm) 是一类用于求解复杂优化问题的高效算法，它通过模拟自然界或社会现象的过程，探索问题的解空间以寻找全局最优解或近似最优解^[35]。这类算法通常不依赖于问题的具体结构，而是通过策略性搜索来应对复杂和大规模的优化问题。Qu等^[36]研究了300毫米晶圆制造中多订单批次的调度问题，旨在优化前开口统一盒的容量利用率与系统性能。针对单机环境下的两种典型加工模式，作者提出两种遗传算法的元启发式方法。实验结果表明，这些方法能在可接受的计算时间内为大规模调度问题生成近似最优解，尤其在订单分组频繁且需兼顾开发晶圆与客户订单的场景下表现出色。Lin等^[37]针对半导体制造中的晶圆分选调度问题，提出了一种多目标优化方法，以最小化总设置时间和减少机器使用数量。作者开发了三种元启发式算法——蚁群系统、遗传算法和禁忌搜索，以应对复杂的并行机调度场景。

(4) 人工智能算法

人工智能方法在解决调度问题中展现出强大的应用潜力。传统的调度方法通常依赖于人工经验和规则，难以应对复杂多变的实际需求^[38]。而人工智能方法，特别是强化学习算法，为解决调度问题提供了新的思路。Hong等^[39]提出了一种基于条件的清洗方法，以解决半导体制造中集群工具在晶圆加工过程中清洗腔室所面临的质量与生产效率的权衡问题。文章设计了多智能体强化学习框架，将机器人手臂和腔室建模为智能体，采用改进的异步优势演员-评论家 (Asynchronous Advantage Actor-Critic, A3C) 算法，通过两阶段网络结构分别优化机器人动作和腔室清洗决策。Suerich等^[40]针对半导体组合设备的复杂调度挑战，提出基于强化学习的实时动态调度方法。通过构建可实验数字孪生模拟工具环境，训练智能体在非确定性场景中优化晶圆流路径，以最大化吞吐量并减少等待时间浪费。Kim等^[41]针对单臂组合设备在半导体制造中同时处理两种晶圆类型的调度问题，提出了一种基于深度强化学习与自适应搜索的非周期性调度方法。作者通过强化学习框架，使机器人能够动态决策晶圆类型的释放顺序和任务序列，并结合自适应搜索机制探索未来状态以优化动作选

择，从而减少瓶颈处理模块的空闲时间。

1.3 本文主要研究内容

本文围绕半导体制造业中晶圆加工流程的组合设备调度问题展开研究。针对自动制造系统中常用的两种离散事件系统形式化模型——Petri 网与自动机，本文分别构建了基于时间 Petri 网与时间自动机的模型，并采用强化学习算法与模型检测方法探索组合设备在不同加工任务下的调度策略。主要工作总结如下：

1. 针对组合设备调度问题，本文首先介绍了基于库所变迁时延 Petri 网 (Place Transition Time Petri Net, PTTPN) 的组合设备建模方法。PTTPN 充分融合了库所时延 Petri 网和变迁时间 Petri 网的优点，将变迁和库所均与时间因素相结合，显著增强了对系统时效性的表达能力。通过对现有组合设备 PTTPN 模型的深入分析，本文明确了模型中各变迁和库所的语义，为后续研究奠定了坚实的理论基础。
2. 在上述 PTTPN 模型的基础上，本文进一步引入强化学习算法，以求解系统调度方案，实现最小化完工时间的目标。具体而言，首先定义系统的状态空间与动作集合，并结合任务目标设计合理的奖励函数。随后，构建训练环境，并分别采用 Q-learning 和深度 Q 网络 (Deep Q Network, DQN) 算法对智能体进行训练。通过分析训练曲线与提取调度策略，最终得到了高效的调度方案，并对不同方案的调度结果进行了详细的对比与分析。此外，针对 Q-learning 与 DQN 在训练过程中存在的探索能力问题，本文在基础单步惩罚奖励函数的基础上，进一步设计了加权组合奖励函数，以优化智能体的学习效果并提升算法的调度性能。
3. 本文提出了一种基于时间自动机的组合设备建模方法。首先，根据实际加工流程构建相应的子自动机，以模拟加工任务的执行过程。随后，针对系统内的各个功能模块，分别建立对应的子自动机，以精确刻画各模块的工作逻辑及运行约束。最后，整合各个子自动机，构建用于描述系统整体状态的自动机，并通过时间自动机的时钟变量精准模拟系统中的时间演化过程。多个子自动机协同作用，形成完整的自动机网络，从而实现对组合设备运行状态的全面刻画和精确分析。
4. 基于已建立的时间自动机模型，本文利用模型检测方法对组合设备的调度方案进行验证与优化。借助 UPPAAL 模型检测工具，搭建了组合设备的时间自动机模型，并采用时间计算树逻辑 (Timed Computation Tree Logic, TCTL) 描述系统

的关键性质，如可调度性与调度方案最优性。通过 UPPAAL 的验证器及诊断路径生成功能，获得其最优调度方案。

本文的创新工作主要体现在两个方面：一方面，针对 Petri 网系统中状态空间较大的情况，本文采用 DQN 算法，有效解决了 Q-learning 算法无法得到正确调度结果的问题。此外，在传统单步时间惩罚奖励函数的基础上，设计了一种加权组合奖励函数。该方法通过引入加工进度和设备利用率奖励项，并根据实际生产需求动态调整各因素的权重，从而增强了强化学习算法在复杂状态空间中的探索能力，加速其收敛至更优调度策略。另一方面，本文提出了一种基于时间自动机的组合设备建模方法。该方法通过构建多个子自动机精确刻画各功能模块的运行逻辑与时序约束，并将其整合为完整的时间自动机网络，为调度方案的验证与优化提供了坚实的理论支撑。

1.4 本文结构

本文针对半导体制造中晶圆加工流程中的组合设备调度问题展开研究。通过构建时间 Petri 网和时间自动机两种形式化模型，并分别结合强化学习算法与模型检测方法，对调度方案进行探讨。论文共分为六章，各章节安排如下：

第一章：绪论。本章首先阐述了半导体行业当前的发展现状及未来前景，明确了本研究的背景与意义。随后，对组合设备的建模方法以及常用调度算法进行了简要介绍，最后概述了本文的主要工作内容和整体结构。

第二章：相关基础理论和方法介绍。本章主要介绍了 Petri 网与自动机这两种形式化模型，并详细论述了引入时间因素的时间 Petri 网模型及时间自动机模型。接着，介绍了本文所采用的两种核心方法：强化学习算法和模型检测方法，为后续章节的研究奠定理论基础。

第三章：基于时间 Petri 网和强化学习的组合设备调度。本章首先介绍了组合设备的 Petri 网建模思路，并在此基础上详细阐述了强化学习训练环境的构建过程，包括状态空间设计、动作集构建以及奖励函数的设计。实验部分利用 MATLAB 平台及其强化学习工具箱，分别采用 Q-learning 算法和 DQN 算法对智能体进行训练，最终获得相应的调度方案。

第四章：基于时间自动机和模型检测方法的则和设备调度。本章主要提出了一种组合设备的时间自动机建模方法，并介绍了模型检测工具 UPPAAL 的使用方法及 TCTL 的基本概念。基于上述理论，生成了相应的组合设备调度方案。

第五章：实验结果和分析。本章通过两个组合设备案例，分别采用两种方法获得调度方案，并对比分析其调度效果。实验结果验证了所提出方法的有效性，同时也展示了其在实际工程应用中的潜在价值。

第六章：总结与展望。本章对全文研究工作进行了总结，归纳了研究中的不足，并对未来可能的优化方向和进一步拓展工作进行了展望。

第二章 相关基础理论和方法

本章是本论文的理论基础，主要介绍了离散事件系统中常用的两种形式化模型 Petri 网和自动机的相关概念及定义，在此基础上介绍了两种模型加入时间元素后的模型时间 Petri 网和时间自动机，最后介绍了强化学习和模型检测的相关理论。

2.1 Petri 网及时间 Petri 网基础概念

2.1.1 Petri 网基础概念

Petri 网是一种抽象的、形式化的信息流模型，主要用于对离散事件系统进行建模。通过研究网模型的相关属性，我们能够有效地描述和分析信息流，并对系统进行控制，特别是在处理具有大量并发事件的系统时，Petri 网表现出独特的优势，尤其适用于并发性、优先级或发生频率受到一定约束的复杂系统^[42]。

从图论来看，Petri 网是双枝有向图，由四大基本部分组成：库所 (place)，变迁 (transition)，有向弧 (arc) 和托肯 (token)。库所抽象地表示系统的状态或资源，可容纳一定数量的托肯，用以反映特定的状态或条件，用圆圈表示。变迁代表系统中的事件或动作，其触发需满足特定的输入条件，用矩形或粗实线表示。有向弧连接库所与变迁，明确状态与事件之间的流向关系，用箭头表示。托肯则作为一种标记或实体，分布于库所中，用以指示某种资源的存在或权限的赋予用黑点表示，当托肯数量过多时，用数字代替黑点以节省空间。

定义 2.1 ^[43] 一个 Petri 网 N 可以形式化定义为一个四元组：

$$N = (P, T, F, W)$$

其中：

- $P = \{p_1, p_2, \dots, p_m\}$ 是库所的有限集合，表示系统的状态。
- $T = \{t_1, t_2, \dots, t_n\}$ 是变迁的有限集合，表示系统中的事件或动作。
- $F \subseteq (P \times T) \cup (T \times P)$ 是有向弧的集合，表示库所与变迁之间的连接关系。
- $W : (P \times T) \cup (T \times P) \rightarrow \mathbb{N}^+$ 是一个 F 的权重映射，为每个有向弧分配一个正整数权值。

满足以下条件：

1. $P \neq \emptyset, T \neq \emptyset$: 库所集合和变迁集合非空。
2. $P \cap T = \emptyset$: 库所集合和变迁集合有限且互不相交。

定义 2.2 ^[43] Petri 网 $N = (P, T, F, W)$ 的标识 M 是一个从库所集合 P 到自然数集合 $\mathbb{N}$ 的映射。我们将 (N, M_0) 称为网系统或标识网，其中 M_0 表示 N 的初始标识。在不产生歧义的情况下， (N, M_0) 通常直接简称为 Petri 网。为了更明确地表达， (N, M_0) 有时也表示为 (P, T, F, W, M_0) 。

定义 2.3 ^[43] 一个 Petri 网 $N = (P, T, F, W)$ ，若 $\forall f \in F, W(f) = 1$ ，则称 N 为普通网，否则称 N 为一般网，记作 $N = (P, T, F)$ 。

定义 2.4 ^[43] 网 N 的标识（或状态） M 是一个映射 $M: P \rightarrow \mathbb{N}$ ，其中 $M(p)$ 表示在标识 M 下库所 p 中的托肯数量。当且仅当 $M(p) > 0$ 时，库所 p 被称为由 M 标记。集合 P 的任意子集 $S \subseteq P$ 被称为由 M 标记的子集，当且仅当 S 中至少存在一个库所被 M 标记。子集 S 中所有库所的托肯总数表示为 $M(S)$ ，定义为 $M(S) = \sum_{p \in S} M(p)$ 。通常， (N, M_0) 被称为网系统或标记网，其中 M_0 是网 N 的初始标识。

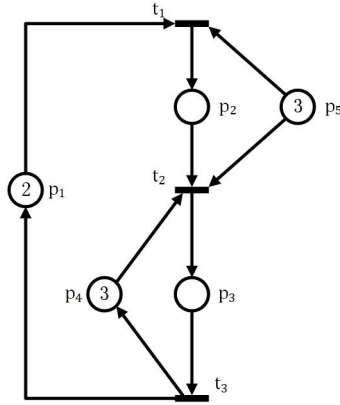


图 2.1 一个 Petri 网 (N, M_0)

例 2.1 图 2.1 给出了一个 Petri 网 (N, M_0) ，其中包括 5 个库所和 3 个变迁。库所集合 $P = \{p_1, p_2, p_3, p_4, p_5\}$ ，变迁集合 $T = \{t_1, t_2, t_3\}$ ，有向弧集合 $F = \{(p_1, t_1), (t_1, p_2), (p_2, t_2), (t_2, p_3), (p_3, t_3), (t_3, p_1), (t_3, p_4), (p_4, t_2), (p_5, t_1), (p_5, t_2)\}$ ，弧的权值为 $W(p_1, t_1) = W(t_1, p_2) = W(p_2, t_2) = W(t_2, p_3) = W(p_3, t_3) = W(t_3, p_1) = W(t_3, p_4) = W(p_4, t_2) = W(p_5, t_1) = W(p_5, t_2) = 1$ 。图中库所和变迁分别用圆圈与粗实线表示，库所中的数字代表该库所种的托肯数。由于该网中所有有向弧的权值均为 1，故该网为普通网，该网的初始标识为 $M_0 = [2, 0, 0, 3, 3]^T$ ，也可写为 $M_0 = 2p_1 + 3p_4 + 3p_5$ 。

定义 2.5 ^[43] 设网 $N = (P, T, F, W)$ 中任一节点 $x \in P \cup T$ ，则其前置集定义为 $\bullet x = \{y \in P \cup T \mid (y, x) \in F\}$ ，表示所有指向 x 的节点；对应地，后置集定义为 $x \bullet = \{y \in P \cup T \mid (x, y) \in F\}$ ，即所有从 x 出发的节点。该定义可进一步推广至任意

节点集合 $X \subseteq P \cup T$ ，其前置集与后置集分别为 $\bullet X = \bigcup_{x \in X} \bullet x$ 和 $X \bullet = \bigcup_{x \in X} x \bullet$ 。

例 2.2 在图 2.1 所示的 Petri 网中，各变迁和库所的前置集与后置集如下：变迁 t_1 的前置集为 $\bullet t_1 = \{p_1, p_5\}$ ，后置集为 $t_1 \bullet = \{p_2\}$ ；变迁 t_2 的前置集为 $\bullet t_2 = \{p_2, p_4, p_5\}$ ，后置集为 $t_2 \bullet = \{p_3\}$ ；变迁 t_3 的前置集为 $\bullet t_3 = \{p_3\}$ ，后置集为 $t_3 \bullet = \{p_1, p_4\}$ 。类似地，对于库所 p_1 和 p_2 ，它们的前置集与后置集分别为： $\bullet p_1 = \{t_3\}$ ， $p_1 \bullet = \{t_1\}$ ； $\bullet p_2 = \{t_1\}$ ， $p_2 \bullet = \{t_2\}$ ；若令 $S = \{p_1, p_2\}$ ，则其前置集和后置集分别为： $\bullet S = \bullet p_1 \cup \bullet p_2 = \{t_2, t_3\}$ ， $S \bullet = p_1 \bullet \cup p_2 \bullet = \{t_1, t_2\}$ 。

定义 2.6 ^[44] 在标识 M 下，若变迁 $t \in T$ 满足 $\forall p \in \bullet t, M(p) \geq W(p, t)$ ，则称 t 是使能的，记作 $M[t]$ 。当使能变迁 t 被触发后，将产生一个新的标识 M' ，记作 $M[t]M'$ ，且新标识 M' 满足 $\forall p \in P, M'(p) = M(p) - W(p, t) + W(t, p)$ 。此时称 M' 是从 M 直接可达的标识。进一步地，若存在一个变迁序列 $\sigma = t_0 t_1 \cdots t_n$ 以及一系列标识 $M_1, M_2, \dots, M_n$ ，使得 $M[t_0]M_1[t_1]M_2 \cdots M_n[t_n]M'$ 成立，则称标识 M' 是从 M 可达的。从标识 M 出发，网 N 所能到达的所有标识的集合称为网 N 的可达标识集合，记作 $R(N, M)$ 。

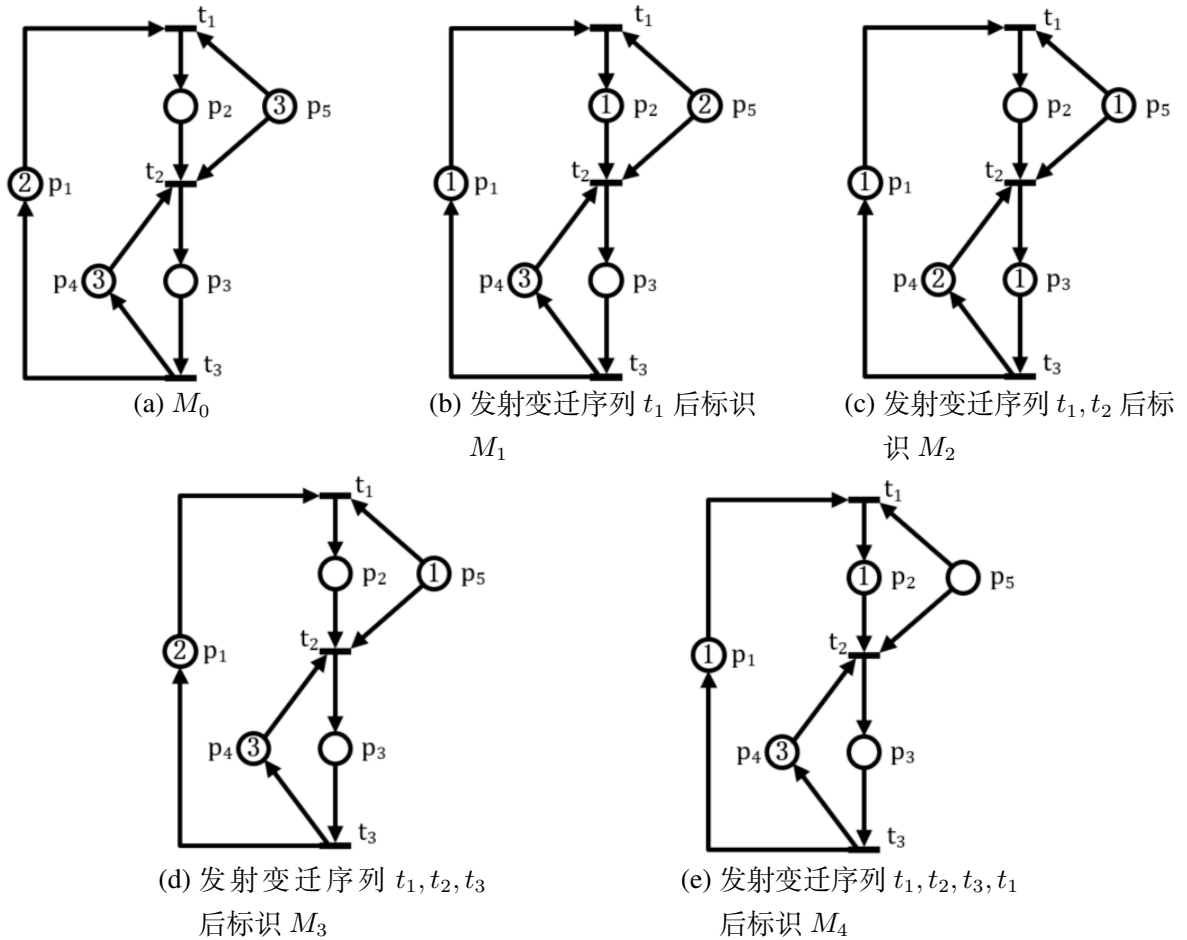


图 2.2 Petri 网发射变迁序列后的标识变化

例 2.3 在图2.1的 Petri 网系统中, 其初始标识为 $M_0 = 2p_1 + 3p_4 + 3p_5$ 。图2.2展示了该网模型发射变迁序列 $\sigma = t_1 t_2 t_3 t_1$ 时的标识变化。图2.2(a)标识 M_0 下使能的变迁集合为 t_1 , 当变迁 t_1 发射后, 得到新的标识 $M_1 = p_1 + p_2 + 3p_4 + 2p_5$ 。在图2.2(b)标识 M_1 下, 使能变迁集合为 $\{t_1, t_2\}$ 。变迁 t_2 发射后, 新的标识为 $M_2 = p_1 + p_3 + 2p_4 + p_5$ 如图2.2(c)。此时, 使能变迁集合为 $\{t_1, t_3\}$ 。当变迁 t_3 发射后, 新的标识为 $M_3 = 2p_1 + 3p_4 + p_5$ 如图2.2(d)。此时, 可发生变迁集合为 $\{t_1\}$ 。当变迁 t_1 发射后, 新的标识为 $M_4 = p_1 + p_2 + 3p_4$ 如图2.2(e), 此时系统没有可发生的变迁。

定义 2.7 ^[44] 称一个 Petri 网 $N = (P, T, F, W)$ 是纯的或无自环的, 当且仅当 $\nexists x, y \in P \cup T$, 使得 $(x, y) \in F$ 和 $(y, x) \in F$ 同时成立。

定义 2.8 ^[44] 一个纯网 $N = (P, T, F, W)$ 可以由它的关联矩阵 $[N]$ 来表示, 其中: $[N]$ 是一个 $|P| \times |T|$ 的整数矩阵, 而且 $[N](p, t) = W(t, p) - W(p, t)$ 。对于库所 (变迁) p (t) 来说, 它的关联向量记作 $[N](p, \cdot)$ ($[N](\cdot, t)$)。

根据关联矩阵定义, 图2.1中的 Petri 网系统的关联矩阵如下:

$$[N] = \begin{bmatrix} -1 & 0 & 1 \\ 1 & -1 & 0 \\ 0 & 1 & -1 \\ 0 & -1 & 1 \\ -1 & -1 & 0 \end{bmatrix} \quad (2-1)$$

关联矩阵中元素 $[N](p, t)$ 的物理含义为: 在变迁 t 发射之后, 如果 $[N](p, t) > 0$ (即, $[N](p, t) < 0$), 则库所 p 将得到 (失去) $|[N](p, t)|$ 个托肯; 如果 $[N](p, t) = 0$, 则库所 p 中的托肯数量保持不变。向量 $[N](p, \cdot)$ 描述了网中每个变迁发射一次时, 库所 p 中托肯数量变化的情况。而向量 $[N](\cdot, t)$ 则描述了变迁 t 发射后, 网中各个库所托肯数量变化的情况。例如, 在公式2-1中, $[N](\cdot, 1) = [-1, 1, 0, 0, -1]^T$ 表示在图2.1的网模型中, 变迁 t_1 发射后, 库所 p_1, p_2, p_3, p_4, p_5 托肯的数量变化。 $[N](2, \cdot) = [1, -1, 0]$ 表示在变迁 t_1, t_2, t_3 分别发射后 p_2 托肯数量的变化。

根据需要, $[N]$ 可以进一步分为两部分 $[N] = Post - Pre$, 其中, $Post$ 称为后置关联矩阵 (也称为输出矩阵或函数), Pre 称为前置关联矩阵 (也称为输入矩阵或函数)。具体而言, $Pre(p, t) = W(p, t)$, $Post(p, t) = W(t, p)$ 。假定 Petri 网 N 有 n 个库所和 m 个变迁, $Pre(P, t) = [W(p_1, t), W(p_2, t), \dots, W(p_n, t)]^T$, $[N](p, T) =$

$[[N](p, t_1), [N](p, t_2), \dots, [N](p, t_m)]$ 。图2.1中网模型的前置和后置关联矩阵如下：

$$[N]^+ = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}, \quad [N]^- = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 1 & 0 \end{bmatrix} \quad (2-2)$$

定义 2.9 ^[45] 设 $N = (P, T, F, W)$ 是一个网, σ 是变迁的一个有限发射序列, 则 σ 的 Parikh 向量 $\vec{\sigma} : T \rightarrow \mathbb{N}$ 是以 T 为序的向量, 且与 t 对应的元素的取值等于 t 在 σ 中出现的次数。

定义 2.10 ^[45] 对于 Petri 网 N 中每个变迁 t , 显然有 $[N](\cdot, t) = [N]_t \vec{\sigma}$ 。由 $M[t]M'$ 可得 $M' = M + [N](\cdot, t)$, 因而有: 如果 $M[t]M'$, 则 $M' = M + [N]_t \vec{\sigma}$ 。进一步可以推出: 对于一个任意的有限变迁序列 σ , 若 $M[\sigma]M'$, 则下式成立:

$$M' = M + [N]\vec{\sigma} \quad (2-3)$$

公式2-3称为 Petri 网的状态方程, 它是 Petri 网标识变化的代数描述方式。也就是说, 它是对标识与变迁序列中变迁的发生次数之间的关系的一种简洁的表达方式。

例 2.4 根据图2.1中的 Petri 网模型, 当可发射变迁序列为 $\sigma = t_1 t_2 t_3 t_1$ 时, $\vec{\sigma} = t_1 + t_2 + t_3 + t_1 = [2, 1, 1]^T$, 由式2-3可得:

$$M' = M_0 + [N]\vec{\sigma} = \begin{bmatrix} 2 \\ 0 \\ 0 \\ 3 \\ 3 \end{bmatrix} + \begin{bmatrix} -1 & 0 & 1 \\ 1 & -1 & 0 \\ 0 & 1 & -1 \\ 0 & -1 & 1 \\ -1 & -1 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 3 \\ 0 \end{bmatrix}$$

变迁序列发射后的新标识为: $M' = p_1 + p_2 + 3p_4$, 与图2.2(e)相符。

定义 2.11 ^[46] Petri 网 (N, M_0) 的可达图 $R(N, M_0) = (V, E)$ 是一个有向图, 其中: $V = R(N, M_0)$ 是定点的集合, $E = \{(M, t, M') \mid M, M' \in R(N, M_0), M[t]M'\}$ 是边的集合。

定义 2.12 ^[46] 在 Petri 网 (N, M_0) 的可达图 $R(N, M_0)$ 中, 如果某个标识下没有任何变迁能够使能, 则称该标识为死锁标识。如果系统进入某个标识后, 必然会导致死锁状态, 则该标识称为坏标识。那些可能通过发射变迁进入坏标识的标识被称为危险标识。除以上标识外, 其他标识被称为好标识。

Petri 网可达图生成算法如下:

算法 2.1 Petri 网可达图生成算法

输入: 一个 Petri 网 (N, M_0)

输出: 可达图 $R(N, M_0)$

将初始标识 M_0 作为可达图的初始点, 对 M_0 不做标记

while 存在未标记的标识 **do**

$M \leftarrow$ 未标记的标识

for 标识 M 下的使能变迁 **do**

$M' = M + [N](\bullet, t)$

if 可达图中不存在标识 M' **then**

 在可达图中添加 M'

 添加从 M 到 M' 的有向弧 t

 标记 M'

根据算法2.1, 可计算出图2.1中 Petri 网的可达图如下图2.3:

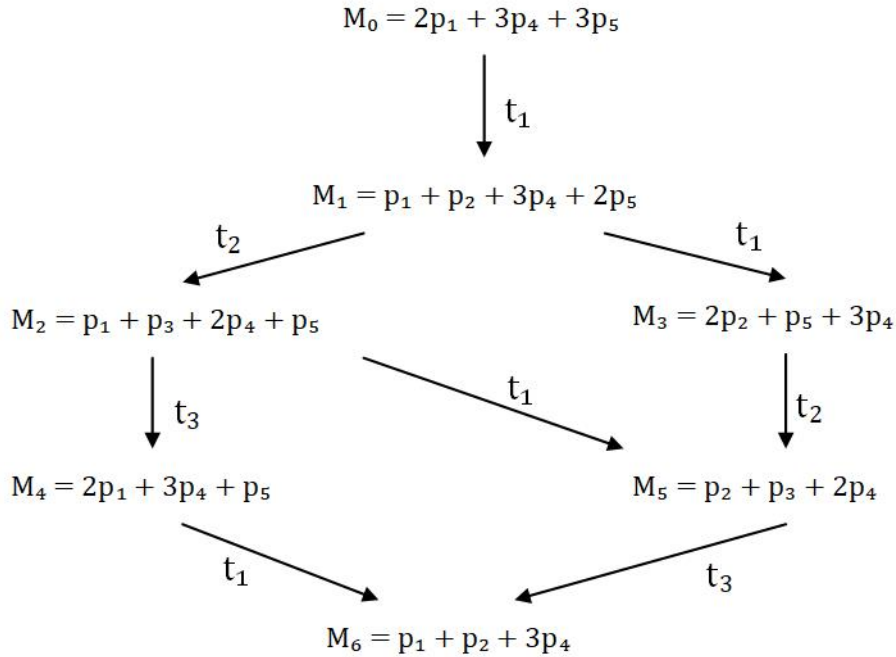


图 2.3 图2.1Petri 网的可达图

在图2.3的可达图中, 由定义2.12可知: 当发射变迁序列 $\sigma = t_1 t_2 t_3 t_1$ 后, 系统标识 M_6 , 此时无可发生变迁为死锁标识; 由于标识 M_4 下, 使能变迁只有 t_1 , 且变迁 t_1 发射后, 标识变为 M_6 , 故 M_3 为坏标识, M_2 为危险标识, M_0, M_1 为好标识。

除 Petri 网的基本定义和可达图外, Petri 网还具有一些重要特性。对这些特性的研究, 有助于我们更好地利用 Petri 网来解决调度问题。通过深入了解这些特性, 我们能够精确地建模系统的动态行为、并发控制、资源分配等关键问题, 从而优化调度策略, 提高系统效率。Petri 网的常用特性如下:

(1) 可达性

可达性是 Petri 网动态特性分析中的一个关键性质。具体来说, 假设从一个 Petri 网的初始标识 M_0 出发, 根据某一变迁发射序列 σ , 经过变迁的触发后可以得到新的标识 M' , 则称 M' 是从 M_0 可达的, 记作 $M_0[\sigma\rangle M'^{[47]}$ 。

(2) 有界性

在 Petri 网中, 有界性表示每个库所中托肯数量的限制。如果一个 Petri 网某个库所的托肯数量都不超过一个有限常数 k , 则称该库所是 k 有界的; 若该网从初始标识 M_0 可达的每一个标识中, 每一个库所的托肯数都不会超过一个有限常数 k , 则称该 Petri 网是 k 有界或有界的^[48]。

(3) 安全性

在有界性的定义中, 如果某个库所的 k 满足 $k = 1$, 则称该库所是安全的, 如果所有库所的托肯数都不超过 k 且 $k = 1$, 则称这个 Petri 网是安全的^[49]。

(4) 活性

σ 是一个变迁序列, 如果对于任意一个标识 $M \in R(N, M_0)$, σ 的触发可以使得此 Petri 网中的变迁 t 使能, 则该变迁是具有活性的。如果模型中的每个变迁都是具有活性的, 则该 Petri 网是具有活性的; 如果一个 Petri 网有活性的且网模型正确, 则系统中不会出现死锁^[50]。

这些特性在将 Petri 网与晶圆加工流程调度问题结合时具有重要意义。例如, 在研究晶圆加工流程调度问题时, 系统能否完成加工任务, 实际上是判断在 Petri 网模型中完工状态的标识是否可达; 此外, 整个晶圆加工流程中每个资源模块都有其容量限制, 因此在实际系统建模时必须考虑有界性; 在实际生产中, 死锁状态通常表示由于资源分配不合理等因素导致的生产停滞, 往往会带来严重的后果, 因此在生产调度中需要尽量避免死锁的发生。

2.1.2 时间 Petri 网基础概念

加入时间因素的 Petri 网相较于传统 Petri 网具有显著优势。首先, 时间 Petri 网通过为变迁或库所引入时间约束, 使得它能够更加真实地模拟系统的动态行为, 特别是涉及时间延迟、任务执行时长、资源占用时间等因素的系统^[51]。与传统 Petri 网仅能描述状态转换和并发行为不同, 时间 Petri 网能够处理实时系统中的时限约束和时间依赖性, 从而更准确地模拟任务的开始和结束时间、资源分配等。其次, 时间 Petri 网能够提供更加精细的动态行为描述, 通过在状态变化中引入时间延迟, 帮助捕捉任务执行之间的时间间隔和系统性能的关系, 这对调度问题的求解至关重要。在调度问题中, 时间 Petri 网能够更好地解决任务执行时间不确定、资源占用时间等问题, 从而提高模型的精确度和现实性。

时间 Petri 网的时间信息通常表示为实数，并与库所或变迁相关联，用来描述托肯在该节点经过的时延。根据时间信息的作用位置，时间 Petri 网可以分为三种类型：变迁时间 Petri 网 (Transition Time Petri Net, TTPN)、库所时间 Petri 网 (Place Time Petri Net, PTPN) 和库所变迁时间 Petri 网 (Place Transition Time Petri Net, PTTPN)。根据具体的应用需求，选择不同类型的时间 Petri 网可以更精确地模拟系统的时序特性，下文会对上述三种类型的时间 Petri 网进行介绍。

(1) 变迁时间 Petri 网

在变迁时间 Petri 网中，每个变迁都与一个时间变量相关联，该时间变量被称为变迁时延，它代表了从变迁激活到其实际发射所需的时间。在这种类型的 Petri 网中，变迁在满足触发条件后不会立即发射，而是需要经过一段时间的延迟才能执行。

定义 2.13 ^[52] TTPN 是一个六元组 $Z_t = \{P, T, I, O, m_0, d_t\} = \{N, m_0, d_t\}$ ，其中包括以下元素：

- $N = \{P, T, I, O\}$ 为一个 Petri 网，其中 I 和 O 分别是输入函数和输出函数，定义了库所与变迁之间的有向弧连接关系；
- $m_0 : P \rightarrow \mathbb{N}$ 为 Petri 网初始标识，表示每个库所上初始的托肯数量；
- $d_t : T_t \rightarrow \mathbb{R}^+ \cup \{0\}$ 是分配给变迁的延迟函数，表示变迁发射所需的时间延迟。

在 TTPN 中，变迁的使能需满足普通 Petri 网的使能条件，变迁准备发射时，变迁关联的时间变量逐渐减小，同时可并发的变迁时间变量也同时减小，当减小至零时，变迁发射。下文将通过例子来演示该过程。

例 2.5 如图 2.4 所示为一个简单 TTPN 模型，模型包含六个库所和四个变迁，模型中变迁与时间变量关联。初始标识 $M_0 = [1, 0, 0, 0, 0, 0]^T$ ， t_1 到 t_4 各个变迁的发射时间分别为 5 单位时间、4 单位时间、10 单位时间、0 单位时间，目标标识为 $M_e = [0, 0, 0, 0, 0, 1]^T$ 。

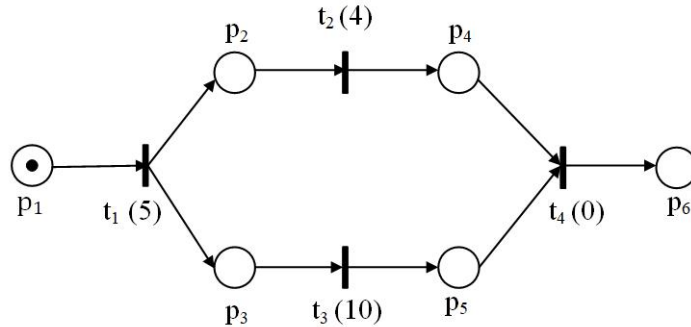


图 2.4 变迁时间 Petri 网示例

在初始标识 M_0 下，变迁 t_1 是使能的。经过 5 个单位时间的等待后，变迁 t_1 发射，新的标识为 $M_1 = [0, 1, 1, 0, 0, 0]^T$ ，此时全局时间为 5 单位时间。接着，在标识 M_1 下，变迁 t_1 和 t_2 都是使能的。在这两个变迁能够并发的情况下，经过 4 个单位时间后，变迁 t_2 发射，标识更新为 $M_2 = [0, 0, 1, 1, 0, 0]^T$ ，全局时间为 9 单位时间，此时变迁 t_3 仍剩余 6 个单位时间可发射。随着全局时间达到 15 单位时间，变迁 t_3 发射，新的标识为 $M_3 = [0, 0, 0, 1, 1, 0]^T$ 。此时变迁 t_4 使能，由于变迁 t_4 的时间延迟为 0，因此可以立即发射，发射后系统到达目标标识 $M_e = [0, 0, 0, 0, 0, 1]^T$ 。

(2) 库所时间 Petri 网

库所时间 Petri 网将时间信息与库所相结合。在晶圆加工系统中，通过为库所添加时延，可以有效地模拟加工模块对晶圆的加工过程。对于某些用来描述资源或中间缓存的库所，则不需要添加时延。

定义 2.14 ^[53] PTPN 网是一个二元组 $P = (N, I)$ ，其中：

- N 是一个经典的 Petri 网， P 是其库所集合。
- $I : P \rightarrow \mathbb{Q}_0^+ \times (\mathbb{Q}_0^+ \cup \{\infty\})$ 是一个函数，表示每个库所 $p \in P$ 对应的时间窗口 $I(p) = (l_p, u_p)$ ，其中 $l_p \leq u_p$ 。

库所时间 Petri 网的变迁使能需满足普通 Petri 网的发射条件，且其前置库锁中的托肯需满足库所的时间约束。变迁发射后，系统的全局时间的增量为该变迁前置库所中托肯的最大时钟。下文将通过例子来演示该过程。

例 2.6 如图 2.5 所示为一个简单的 PTPN 模型，结构与图 2.4 相同，但所有变迁均未设置时间约束，均默认瞬间发射，各个库所均设置了时间约束。初始标识 $M_0 = [1, 0, 0, 0, 0, 0]^T$ ， p_1 到 p_6 各个变迁的发射时间分别为 0 单位时间、3 单位时间、5 单位时间、5 单位时间，3 单位时间、0 单位时间，目标标识为 $M_e = [0, 0, 0, 0, 0, 1]^T$ 。

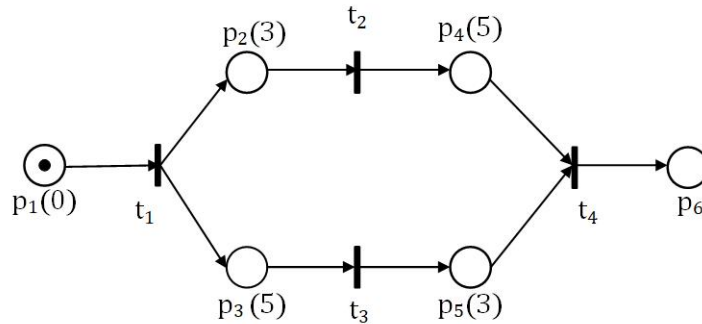


图 2.5 库所时间 Petri 网示例

在初始标识 M_0 下，变迁 t_1 处于使能状态。由于库所 p_1 的时间约束为 0， t_1 立即发射，更新后的标识变为 $M_1 = [0, 1, 1, 0, 0, 0]^T$ ，此时全局时间仍为 0。接着，在标

识 M_1 下, 变迁 t_1 和 t_2 均变为使能状态。经过 3 个单位时间后, 库所 p_2 中托肯的时钟归零, 触发 t_2 发射, 标识更新为 $M_2 = [0, 0, 1, 1, 0, 0]^T$, 此时全局时间为 3, 而库所 p_3 中托肯的剩余时间为 2 个单位。随着全局时间达到 5, 变迁 t_3 发射, 新的标识为 $M_3 = [0, 0, 0, 1, 1, 0]^T$ 。随后, 变迁 t_4 变为使能状态, 并在全局时间达到 10 时发射, 最终使系统到达目标标识 $M_e = [0, 0, 0, 0, 0, 1]^T$ 。

(3) 库所变迁时间 Petri 网

在库所变迁时间 Petri 网中, 库所和时间都与时间相关联, 结合了 PTTN 和 TTPN 的特点。对于工作逻辑更加复杂的系统, PTTPN 能够更有效地描述系统的时序行为。

定义 2.15 PTTPN 是一个三元组, 满足 $\{N, d_p, d_t\}$, 其中:

1. N 为一个四元组 (P, T, F, W) , 表示一个普通 Petri 网;
2. $d_p : P \rightarrow \mathbb{R}^+ \cup \{0\}, p_i \rightarrow d_p(p_i) = [0, T(p_i)]$, 其中 $T(p_i)$ 表示托肯直接经过该库所的时延;
3. $d_t : T \rightarrow \mathbb{R}^+ \cup \{0\}, t_i \rightarrow d_t(t_i) = [0, T(t_i)]$, 其中 $T(t_i)$ 表示托肯直接经过该变迁的时延。

PTTPN 的库所和变迁均与时间紧密关联, 因此, 变迁的使能与传统的 PTPN 类似, 必须等待前置库所中的托肯时延为零后才能使能。变迁一旦使能, 其时钟便开始计时, 计时完成后, 变迁才会执行发射操作。在这一过程中, 其他库所中的托肯和变迁上的时延也会随着全局时间的推进而逐渐减少。下文将通过例子来演示其变迁发射过程。

例 2.7 如图 2.6 所示为一个 PTTPN 模型, 所有变迁与库所均设置了时间约束。初始标识 $M_0 = [1, 0, 0, 0, 0, 0]^T$, 目标标识为 $M_e = [0, 0, 0, 0, 0, 1]^T$ 。

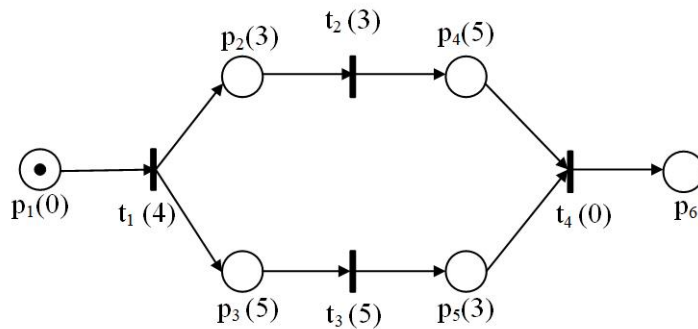


图 2.6 库所变迁时间 Petri 网示例

待发射变迁序列为 $\sigma = t_1 t_2 t_3 t_4$, 系统发射过程如下:

1. 在初始标识 M_0 下, 变迁 t_1 使能, 全局时间为 4 时, 变迁 t_1 发射, 新标识为 $M_1 = [0, 1, 1, 0, 0, 0]^T$ 。

2. 在 M_1 下, 全局时间增加 3, 变迁 t_2 使能, 此时库所 p_3 的剩余时间 2。全局时间增加 3 之后, 变迁 t_2 发射, 全局时间为 10, 新标识为 $M_2 = [0, 0, 1, 1, 0, 0]^T$, 此时变迁 t_3 还需 4 单位时间发射。
3. 在 M_2 下, 全局时间增加 4 后, 变迁 t_3 发射, 此时库所 p_4 剩余时间为 1, 全局时间为 14, 新标识为 $M_3 = [0, 0, 0, 1, 1, 0]^T$ 。
4. 在 M_2 下, 全局时间增加 3 后, 库所 p_4 和 p_5 时间均为 0, 此时变迁 t_4 使能, 可立即发射。发射后到达目标标识 M_e , 全局时间为 17。

2.2 自动机及时间自动机基础概念

2.2.1 自动机基础概念

在过去的几十年里, 自动机理论已发展成为计算机科学的一个重要基础, 并成为离散事件系统建模中的常用工具。自动机通常通过状态转换图来表示。在这种图中, 系统的状态以圆圈或矩形框的形式呈现, 状态之间通过有向边连接, 边上标有字母, 表示触发状态转换的事件。当事件发生时, 系统的状态将随之变化。初始状态通常通过一个指向该状态的箭头标识。根据不同的应用需求, 自动机可以分为多种类型, 如有限自动机 (Finite Automaton, FA)、无限自动机 (Infinite Automaton, IA)、概率自动机 (Probabilistic Automaton, PA)、细胞自动机 (Cellular Automaton, CA) 和抽象自动机 (Abstract Automaton) 等, 其中有限自动机又可分为确定性有限自动机 (Deterministic Finite Automaton, DFA)、非确定性有限状态自动 (Nondeterministic Finite Automaton, NFA) 机和 ϵ -非确定有限自动机 (Epsilon Nondeterministic Finite Automaton, ϵ -NFA)^[54]。本文中对于晶圆加工流程中组合设备的建模使用 DFA。

定义 2.16 ^[55] 一个确定性有限自动机 DFA 定义为 $G = \langle Y, \Sigma, \eta, y_0, Y_m \rangle$, 其中

- Y 是一个有限且非空的状态集;
- Σ 是一个有限且非空的事件集, 称为转换标签的字母表;
- $\eta: Y \times \Sigma \rightarrow Y$ 是部分转换函数;
- $y_0 \in Y$ 是初始状态;
- $Y_m \subseteq Y$ 是标记状态集。

例 2.8 图2.7为一个 DFA 模型, 其状态集合为 $Y = \{q_0, q_1, q_2, q_3\}$, 字母表为 $\Sigma = \{a, b, c\}$, 初始状态为 $y_0 = q_0$ 。其状态转换函数如表2.1所示。

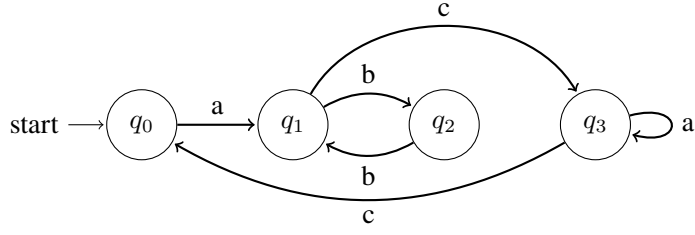


图 2.7 一个确定性有限状态自动机示例

δ	a	b	c
q_0	q_1	—	—
q_1	—	q_2	q_3
q_2	—	q_1	—
q_3	q_3	—	q_0

表 2.1 图2.7DFA 的转换函数

给定 $y \in Y$ 和 $\sigma \in \Sigma$, 如果 $\eta(y, \sigma)$ 被定义, 则写作 $\eta(y, \sigma)!$ 。为了方便起见, η 被扩展为一个偏函数: $Y \times \Sigma^* \rightarrow Y$, 定义为 $\eta(y, \epsilon) = y$ 且 $\eta(y, s\sigma) = \eta(\eta(y, s), \sigma)$, 其中 $s \in \Sigma^*$ 且 $\sigma \in \Sigma$ 。因此, 给定 $s \in \Sigma^*$, 若 $\eta(y, s)$ 被定义, 则写作 $\eta(y, s)!$ 。通过表2.1, 我们可以得到图2.7中 DFA 的状态转移函数。例如, 在状态 q_0 下, 事件 a 是可以触发的, 记作 $\eta(q_0, a)!$; 此时, 状态 q_0 可以通过事件 a 转移到状态 q_1 , 即 $\eta(q_0, a) = q_1$ 。状态 q_0 通过事件 a 到达状态 q_1 , 在通过时间 c 到达状态 q_3 记为 $\eta(q_0, ac) = q_3$ 。

2.2.2 时间自动机基础概念

时间自动机 (Timed Automata, TA) 是一类扩展了传统有限自动机的模型, 主要用于描述系统中随着时间变化的状态转移。与传统的自动机不同, 时间自动机不仅考虑事件的触发, 还包括时间的因素。时间自动机的状态转换通常伴随着时间约束, 即在特定的时间段内, 某些状态的转换可能会受到限制或者只能在特定时间发生。它们广泛应用于实时系统、嵌入式系统以及时间敏感的控制系统等领域。时间自动机的关键特点是引入了时间变量或时钟, 用于表示状态之间的时间依赖关系, 从而能够更精确地建模和分析动态系统中随着时间变化的行为。

定义 2.17 ^[56] 时间自动机 TA 是一个六元组 $\langle L, L^0, \Sigma, X, I, E_i \rangle$, 其中:

- L 是一个有限的状态集合;
- $L^0 \subseteq L$ 是初始状态集合;
- Σ 是一个有限的标签集合;
- X 是一个有限的时钟集合;

- I 是一个映射，为每个状态 s 分配一个时钟约束，属于 $\Phi(X)$ ；
- $E \subseteq L \times \Sigma \times 2^X \times \Phi(X) \times L$ 是一组转换，其中每个转换 $\langle s, a, \varphi, \lambda, s' \rangle$ 表示从状态 s 到状态 s' 的边，符号 a 为转换标签。 φ 是一个关于时钟集合 X 的时钟约束，指定了转换何时被启用，集合 $\lambda \subseteq X$ 给出了在转换发生时重置的时钟。

例 2.9 图2.8为一个时间自动机模型。其状态集合 $L = \{S_0, S_1, S_2, S_3\}$ ，初始状态 $L_0 = \{S_0\}$ ，事件标签集合为 $\Sigma = \{a, b, c\}$ ，时钟集合为 $X = \{x, y\}$ ，转换集合为 $E = \{\langle S_0, a, \emptyset, x \in (0, 1), S_1 \rangle, \langle S_1, b, x \in (5, 8), y, S_2 \rangle, \langle S_2, c, \emptyset, \emptyset, S_3 \rangle, \langle S_3, a, y \in (3, \infty), \emptyset, S_0 \rangle\}$

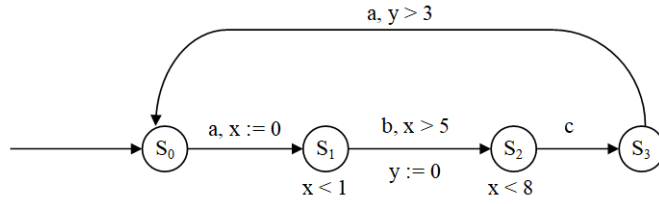


图 2.8 时间自动机示例

时间自动机 A 的语义通过将其与转换系统 S_A 关联来定义。在 S_A 中，每个状态均表示为有序对 (s, ν) ，其中 s 是 A 中的一个位置，而 ν 是定义在时钟集合 X 上的时钟赋值，且该赋值满足位置 s 的不变式 $I(s)$ 。所有状态构成的集合记为 Q_A 。当状态 (s, ν) 中的 s 属于 A 的初始位置，并且对于任意时钟 x 都有 $\nu(x) = 0$ 时，该状态被视为初始状态。

在 S_A 中有两种类型的转换：

1. 时间流逝：对于状态 (s, ν) 和一个实值时间增量 $\delta \geq 0$ ， $(s, \nu) \xrightarrow{\delta} (s, \nu + \delta)$ ，当且仅当对于所有 $0 \leq \delta_0 \leq \delta$ ， $\nu + \delta_0$ 满足不变式 $I(s)$ 。

2. 位置切换：对于状态 (s, ν) 和一个切换 $\langle s, a, \varphi, \lambda, s_0 \rangle$ ，当 ν 满足 φ 时， $(s, \nu) \xrightarrow{a} (s_0, \nu[\lambda := 0])$ 。

因此， S_A 是一个带有标签集 $\Sigma \cup \mathbb{R}$ 的转换系统。例如，图2.8中的时间自动机，关联的转换系统的状态空间是 $\{S_0, S_1, S_2, S_3\} \times \mathbb{R}^2$ ，标签集是 $\{a, b, c\} \cup \mathbb{R}$ ，可发生转换为：

$$(S_0, 0, 0) \xrightarrow{1.5} (S_0, 1.5, 1.5) \xrightarrow{a} (S_1, 0, 1.5) \xrightarrow{6.0} (S_1, 6.0, 1.5) \xrightarrow{b} (S_2, 6, 0)$$

2.3 强化学习

强化学习 (Reinforcement Learning, RL) 是机器学习领域中的一类重要算法，其目标是让智能体 (Agent) 通过与环境 (Environment) 的交互学习如何选择行为^[57]。与其

他学习方法不同，强化学习通过一个单一的奖励信号提供反馈，智能体根据这些信号调整其行为。强化学习不仅仅是一种具体的学习方法，更是一种学习范式或问题框架，智能体的核心任务是采取一系列行动，旨在长期内最大化累计的奖励^[58]。

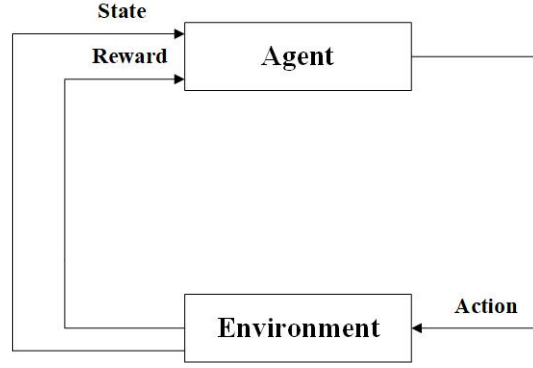


图 2.9 强化学习中智能体与环境的交互

如图2.9所示，在强化学习中，智能体在某个环境中运行，通过观测环境的状态 (State)，结合自身的历史经验，选择一个动作 (Action) 并施加于环境。环境在接收到该动作后，以一定的概率发生状态转移，同时向智能体反馈一个奖励 (Reward)。通过反复执行获取状态、选择动作、影响环境和接收奖励这一交互循环，智能体逐步学习一个决策策略，旨在最大化累计奖励的期望值，从而更好地适应当前环境。

在强化学习中，智能体与环境交互的试错过程通常具备马尔可夫性质，即在任意时刻 t ，下一个状态 s_{t+1} 的概率分布仅依赖于当前状态 s_t ，而与之前的所有历史状态 $\{s_0, s_1, \dots, s_{t-1}\}$ 无关。马尔可夫决策过程 (Markov Decision Process, MDP) 具备这一特性，用于描述有限、随机且连续的决策问题。

定义 2.18 ^[59] MDP 通常被定义为一个五元组 $\langle S, A_i, P_{ij}^a, R_{ij}^a, \gamma \rangle (i, j \in S, a \in A_i)$ ，具体描述如下：

- S : 环境的状态空间集合，其中 i 和 j 表示状态集合中的任意状态；
- A_i : 智能体在状态 i 下可选的动作集合；
- P_{ij}^a : 从状态 i 执行动作 a 转移到状态 j 的转移概率，状态转移函数定义为 $T: S \times A \rightarrow \mathcal{P}(S)$ ，其中 $\mathcal{P}(S)$ 表示状态的概率分布；
- R_{ij}^a : 在状态 i 执行动作 a 转移到状态 j 所获得的奖励值；
- γ : 折扣因子，用于权衡即时奖励和未来奖励的重要性， $\gamma \in [0, 1]$ 。

在每次迭代的时间步 t 中，智能体会感知当前状态 $s_t \in S$ ，然后选择一个动作 $a \in A$ 并执行。执行动作后，环境会根据状态转移概率将智能体转移到下一个状态 $s_{t+1} \in S$ ，并生成相应的奖励值 r_t 。

强化学习具有很多分类，根据是否依赖环境模型可以分为有模型学习和无模型学习。对于已知环境模型的情况，强化学习的过程类似于动态规划，目标是寻找最优解，常用的方法包括值迭代算法和基于 T 步累积奖励的策略迭代。对于无模型学习，强化学习主要分为时序差分学习和蒙特卡罗学习，蒙特卡罗学习需要在整个采样序列完成后才能更新估值，而时序差分学习则可以在每一步交互后即时更新估值^[60]。

强化学习中常用的算法包括 Q 学习 (Q-Learning)^[61]、深度 Q 网络 (Deep Q-Network, DQN)^[62] 和策略梯度方法 (Policy Gradient)^[63]。Q 学习是一种无模型强化学习^[64] 方法，通过迭代更新状态-动作值函数 (Q 值) 来学习最优策略，适用于离散状态和动作空间的问题。深度 Q 网络是 Q 学习的扩展，通过结合深度神经网络来近似 Q 值函数，能够有效处理高维和复杂状态空间，并利用经验回放和目标网络等技术提升训练的稳定性。策略梯度方法则直接优化策略，通过参数化策略并计算梯度来改进性能，尤其适用于连续动作空间的问题。在本文中，我们主要采用 Q 学习和 DQN 算法根据组合设备的 Petri 网模型训练智能体，以获得优化的调度方案。

2.4 模型检测

模型检测 (Model Checking) 是一种自动化技术，用于验证有限状态的反应性系统。通过命题时序逻辑 (Propositional Temporal Logic, PTL) 表达系统的规范，而反应性系统则通过状态转移图进行建模^[65]。模型检测器是一种用于判断给定结构 M 是否满足逻辑公式 φ 的决策过程，通常记为 $M \models \varphi$ 。直观上， M 代表待验证系统的抽象模型，一般为一个有限自动机，而 φ 则通常取自时序逻辑或模态逻辑，用以刻画系统应具备的理想性质^[66]。模型检测器能够一键验证系统模型 M 是否符合规范 φ 所描述的要求。这种全自动化的验证流程结合了高效检测工具在处理复杂逻辑方面的能力，使得模型检测技术在理论研究和实际应用中均具有广泛的吸引力。

模型检测基本流程包括建模，检测和分析三个阶段^[67]：

- 建模阶段：使用当前模型检测器的模型描述语言对系统进行建模，并通过仿真测试进行初步检查和快速评估。使用适当的逻辑公式来描述系统的性质，如线性时序逻辑 (Linear Temporal Logic, LTL) 和计算树逻辑 (Computation Tree Logic, CTL)。
- 检测阶段：也称为运行阶段。此阶段通过运行模型检测器来验证系统模型中属性的有效性。
- 分析阶段：根据系统属性的检测结果进行分析。如果属性满足要求，记录相关信息并继续检测其他属性；若不满足，则通过仿真分析模型检测器生成的反例，

对系统模型或属性描述进行优化，并重复整个过程。在检测过程中，可能出现内存不足的情况，通常是由于系统状态空间过大所致。此时，需要尝试简化模型，缩小系统状态空间。

模型检测流程框图如下图2.10所示：

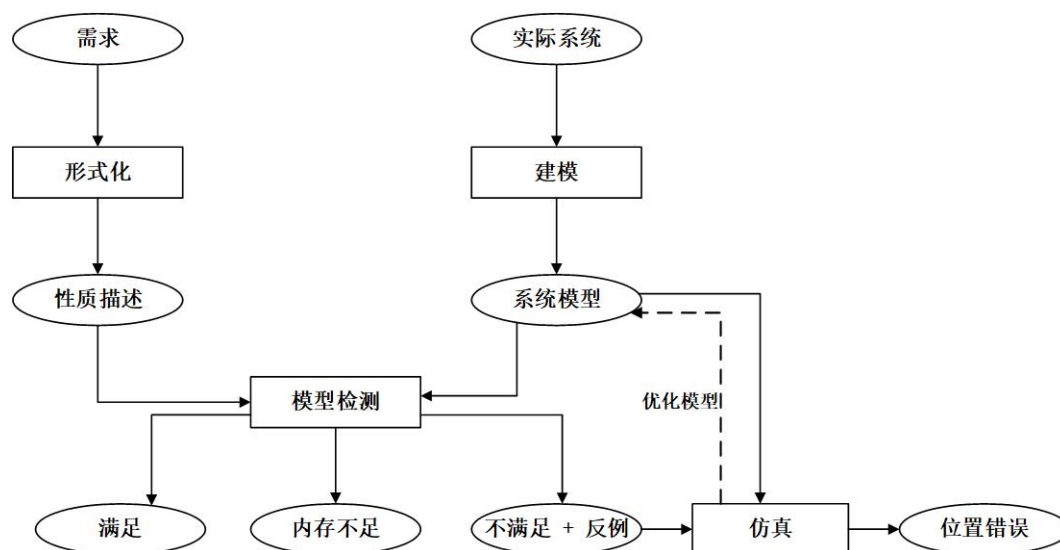


图 2.10 模型检测流程框图

本文中使用时间自动机来作为系统建模工具,对组合设备进行建模,使用时间计算树逻辑 (Timed Computation Tree Logic, TCTL) 作为系统性质描述语言,使用 UP-PAAL 模型检测工具作为模型检测器,对组合设备的系统性质和调度方案进行研究。

2.5 本章小结

本章主要介绍了两种常用的形式化模型：Petri 网和自动机模型，并探讨了在这两种模型中引入时间因素后衍生出的时间 Petri 网和时间自动机的理论基础。通过具体例子，详细解释了这些模型中各个元素的含义与作用。接着，介绍了强化学习的相关知识，包括其基本概念、分类以及常见的强化学习算法。最后，本章还阐述了模型检测方法的概念和基本流程。

第三章 基于时间 Petri 网和强化学习算法的组合设备调度

第二章主要介绍了两种常用的形式化模型 Petri 网和自动机，以及论文中使用到的调度算法相关的基础知识。本章将在此基础上根据实际系统对组合设备进行 Petri 网模型的构建，并在此基础上利用强化学习算法生成调度策略来解决组合设备的调度问题。

3.1 组合设备的 Petri 网建模

3.1.1 组合设备

在半导体制造业中，组合设备是集成多个功能模块的高精度自动化系统，专用于从晶圆加工到芯片封装等关键环节的生产。这类设备通常由清洗、光刻、刻蚀、沉积、抛光和检测等模块组成，每个模块承担特定的工艺任务，并通过复杂的调度算法和精密的控制系统实现高效协同工作。组合设备的设计目标是满足半导体制造中对高精度、高产能以及高可靠性的严苛要求，为提升生产效率和产品质量提供了有力支持。

组合设备通常由多个功能模块组成，包括加工模块 (Processing Module, PM)、校准模块 (Aligner, AL)、冷却模块 (Cooling Module, CL)、真空锁 (Load Lock, LL) 以及运输模块 (Transfer Module, TM)。如图3.1为晶圆加工流程中的组合设备示意图。

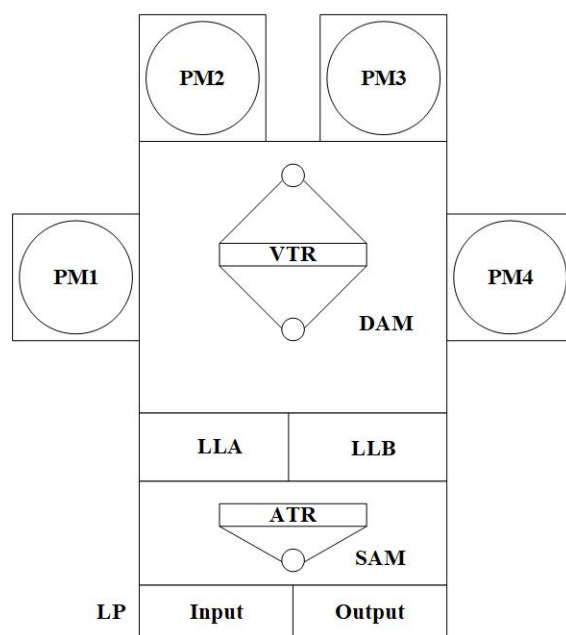


图 3.1 组合设备示意图

图中每个模块的含义与作用如下：

1. 装卸模块 LP>Loading Port, LP): 用于存放待加工和加工完毕的晶圆;
2. 大气端运输机械臂 ATR(Atmospheric Transport Robot, ATR): 属于运输模块, 主要作用是在半导体制造设备的大气环境中完成晶圆的搬运任务。它通常位于设备外部或大气环境模块之间, 用于将晶圆从晶圆盒中取出, 并转移至真空锁或其他模块的入口。图3.1中使用单臂机械手 (Single-Arm Robot, SAM) 作为 ATR。
3. 真空端运输机器人 VTR(Vacuum Transport Robot, VTR): 属于运输模块, 主要作用是在真空环境中实现晶圆的高精度搬运。它通常连接在加工模块之间, 如刻蚀、沉积和清洗等模块, 负责将晶圆从一个加工模块搬运到另一个加工模块。图3.1中使用双臂机械手 (Dual-Arm Robot, DAM) 作为 VTR。
4. 真空锁 LLA, LLB: 主要作用是作为一个中介区域, 用于在大气环境和真空环境之间转换。
5. 加工模块 PM: 实现晶圆加工具体工艺操作的核心设备, 它直接影响到产品的质量、产量和生产效率。

一个完整的晶圆加工流程可表示为: $LP \rightarrow ATR \rightarrow LL \rightarrow VTR \rightarrow PM \rightarrow LL \rightarrow ATR \rightarrow LP$ 。即待加工的晶圆首先由 ATR 从 LP 中夹取, 运送至 LL 进行大气环境到真空环境的转换。接着, VTR 从 LL 中夹取晶圆并运输到 PM 进行加工。根据不同的加工工艺, 晶圆会按照特定的顺序依次被夹取到各个 PM 进行处理。加工完成后, VTR 将晶圆从 PM 夹取并送回 LL, 实现从真空环境到大气环境的转换。最终, ATR 将晶圆从 LL 夹取并返回至 LP, 完成整个加工过程。

3.1.2 Petri 网建模方法

Petri 网模型因其能够清晰地表现系统中的并发行为, 已被广泛应用于自动化制造系统 (Automated Manufacturing System, AMS) 的建模、分析、仿真与控制中。在将非形式化需求转化为形式化描述时, Petri 网有助于有效识别潜在的歧义与矛盾。借助形式化模型, 系统开发可以实现半自动化甚至全自动化, 从而显著提升开发效率。此外, 通过数学验证方法, 可以确保模型的准确性, 避免逐一测试所有情况所带来的繁琐与低效。经过验证的子系统还可以便捷地集成到更大规模的系统中, 进一步增强系统的整体性能与可扩展性。

Petri 网由两种节点组成: 库所和变迁, 分别用圆圈和矩形表示。库所与变迁通过有向弧相连, 定义了 Petri 网的动态运行规律。在对制造系统进行建模时, 库所通常用来表示资源或工序的状态, 变迁则用于表示工序事件的发生或起止。库所中的小黑点被称为托肯, 用于表示相应资源的数量或状态。对于 AMS 来说, Petri 网的建模方

式通常分为面向过程的 Petri 网 (Process-Oriented Petri Net, POPN) 和面向资源的 Petri 网 (Resource-Oriented Petri Net, ROPN), 下文将对两种建模方式进行介绍。

(1) POPN

由于 AMS 具有灵活性, 系统中可以同时加工多种类型的工件。每种工件类型都需要完成一系列操作, 并按照规定顺序和工艺路线进行加工。因此, 为了对这些生产过程进行建模, 可以根据每种工件类型的生产顺序直接建立 AMS 的模型, 从而得到一个 POPN 模型。建立 AMS 的 POPN 模型通常包括以下几个步骤^[68]:

1. 建立工件生产过程的顺序模型。首先, 识别工件生产过程中涉及各个流程, 并将每个流程对应的活动建模为一个库所, 称为操作库所。这些操作库所根据流程的顺序, 通过变迁连接起来, 从而形成生产过程的序列模型。
2. 建立系统中资源的模型。对 AMS 中的资源进行建模, 为每种资源创建一个带有托肯的库所, 此类库所被称为资源库所, 用于表示资源的可用状态。
3. 建立操作所需的资源需求。当某个操作需要使用特定资源时, 将对应的操作库所与资源库所通过有向弧连接起来, 以表示操作对资源的依赖关系, 从而模拟资源需求。

下图3.2为一个 AMS 的 POPN 模型:

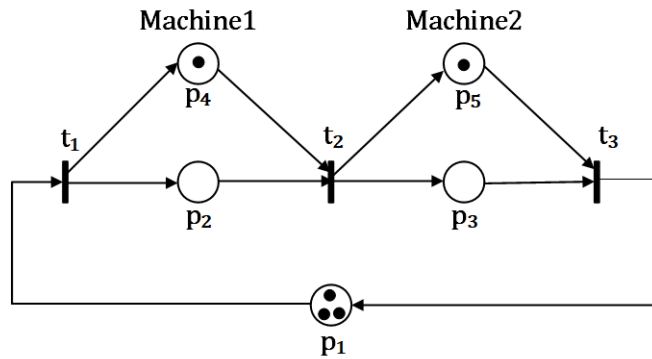


图 3.2 一个 AMS 的 POPN 模型示例

图中的 POPN 模型包含五个库所, 三个变迁。其中 p_1 为进出料口, p_2, p_3 为加工库所, p_4, p_5 为资源库所。工件的加工流程为: $p_1 \rightarrow t_1 \rightarrow p_2 \rightarrow t_2 \rightarrow p_3 \rightarrow t_3 \rightarrow p_1$ 。资源库所 p_4 和 p_5 初始状态各带有一个托肯, 表示两个机器均同一时间只能加工一个工件。

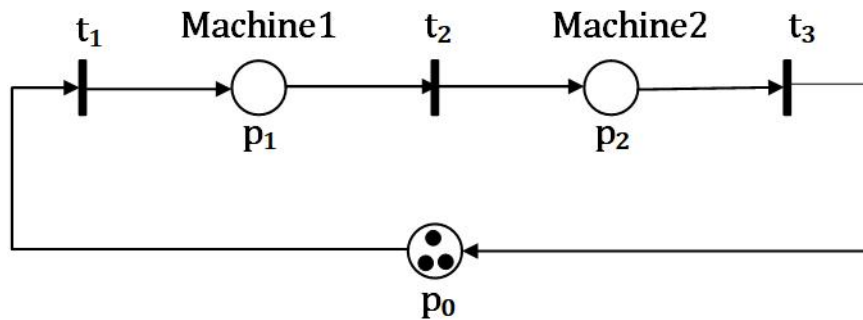
(2) ROPN

ROPN 建模方法将工件的生产过程视为资源动态分配的过程, 通过单一库所对系统中的每种资源进行建模。当待加工工件进入系统时, 它们会对这些资源进行竞争。

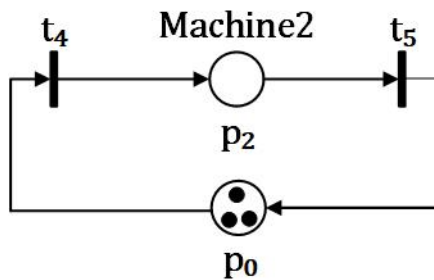
通过模拟工件在各资源之间的流动，可以直观地呈现整个加工流程。在 ROPN 中，机器和缓冲区被归为一类资源，而物料搬运设备则被视为另一类资源。通常建立 ROPN 模型分为以下几步：

1. 建立系统中资源的模型，并根据实际系统，设置资源库所的容量；
2. 针对各种类型的工件，将其生产流程建模为一个子网；
3. 合并各子网，形成 ROPN；
4. 若系统需要同时加工不同类型的工件，可引入带颜色的托肯，用于描述不同工件的流动过程。

下图3.3为一个 AMS 的两个机器的 ROPN 子网示例。假设该 AMS 需加工两种工件，工件 1 需要先由机器 1 加工和再由机器 2 加工，工件 2 则仅需机器 2 加工，机器 1 和机器 2 均只能同时加工一个工件。故用库所 p_1 和 p_2 模拟两机器且两库所的容量均为 1，工件 1 和工件 2 的 ROPN 子网如图3.3(a)和图3.3(b)所示，



(a) 机器 1 的 ROPN 子网



(b) 机器 2 的 ROPN 子网

图 3.3 机器 1 和机器 2 的 ROPN 子网

在建立好子网后，根据共用的资源库所建立系统整体的 ROPN 模型，如图3.4所示，表3.1为模型中各个元素所代表的含义。

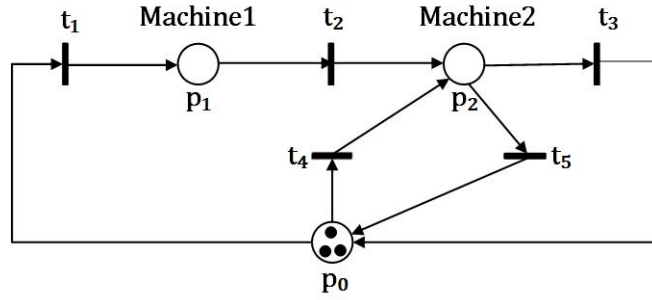


图 3.4 一个 AMS 的 ROPN 模型示例

表 3.1 图3.4所示的 ROPN 模型中元素含义

库所及变迁符号	含义
p_0	AMS 的进出料口，初始托肯数为待加工工件数量
p_1	机器 Machine1，其容量代表可同时加工工件数量
p_2	机器 Machine1，其容量代表可同时加工工件数量
t_1	工件 1 放入机器 1
t_2	由机器 1 加工后的工件 1 放入机器 2
t_3	将机器 2 加工后的工件 1 放入出料口
t_4	将工件 2 放入机器 2
t_5	将机器 2 加工后的工件 2 放入出料口

AMS 的 Petri 网建模方法各有特点。POPNN 在建模能力上优于 ROPNN，能够直观地对系统的生产流程进行建模，并详细刻画系统。当系统中的某些操作需要涉及多种资源时，POPNN 能轻松应对，而 ROPNN 则难以直接建立模型。然而，POPNN 模型使用了更多的库所和变迁，容易导致 Petri 网状态空间过大，从而增加系统分析与控制的难度。因此，在模型复杂度方面，ROPNN 更具优势。

3.1.3 组合设备的 Petri 网建模

由于晶圆加工流程需要组合设备与多个模块协同操作完成，并且涉及对系统调度问题的研究，因此系统模型需要清晰地展示晶圆的加工过程。为此，本文采用 POPNN 的建模方法对组合设备进行建模。在系统的 Petri 网模型中，变迁用于表示系统中机械手对晶圆在各个模块间的搬运操作，库所用于表示设备中的模块与资源。为了对系统的调度问题进行研究，本文采用 PTPNN 进行建模，即对 Petri 网的库所和变迁均添加时间因素。下面介绍主要建模流程。

首先，根据系统的加工流程，建立简单顺序过程的 Petri 网。以加工流程为 LP → LL → PM → LL → cooling → LP 的晶圆为例：晶圆从进料口 LP 被运输至 LL，完成从大气环境到真空环境的转换；随后进入 PM 进行加工；加工完成后，再次进入 LL，

完成从真空环境到大气环境的转换；最后，经过冷却后返回出料口 LP，完成整个加工流程。该加工过程对应的简单顺序 Petri 网如图 3.5 所示。

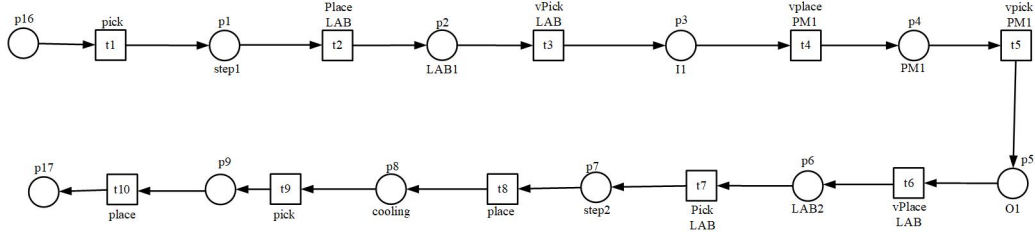


图 3.5 图 3.1 组合设备的简单顺序流程 Petri 网

在完成系统简单顺序过程的建模后，需要进一步根据组合设备的生产逻辑及生产能力限制（如设备容量）对系统进行细化建模。主要建模的模块包括：机械手模块、加工模块以及晶圆存储模块。

在该加工流程中，涉及两个机械手：ATR 和 VTR。ATR 为单臂机械手，容量为 1；VTR 为双臂机械手，容量为 2。为此，分别用初始托肯数为 1 和 2 的库所来表示两个机械手的状态。当进行取操作时，相应的机械手库所作为该操作变迁的前置库所，操作完成后容量减 1；进行放操作时，机械手库所作为后置库所，操作结束后容量加 1。图 3.6(a) 和图 3.6(b) 分别展示了 ATR 和 VTR 的库所示意图。



图 3.6 机械手模块示意图

根据加工模块的容量和工作逻辑，可建立其 Petri 网模型，如图 3.7 所示。其中， P_{pm} 表示加工操作， P_{pmcap} 表示加工模块的容量。在该模型中，库所 P_{pmcap} 的初始托肯数为 2，表示该模块可同时处理两个晶圆。当机械手将晶圆放入加工模块（PM）时，变迁 t_{in} 发射，加工模块容量减 1；晶圆加工完成后，变迁 t_{out} 发射，加工模块容量加 1。

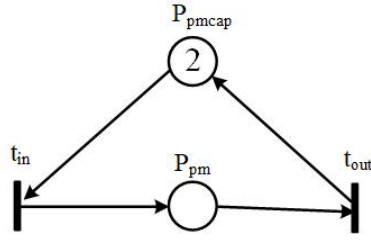


图 3.7 加工模块 Petri 网模型

晶圆存储模块仅用于存放待加工和完成加工的晶圆，故使用单个库所进行模拟。待加工晶圆存放库所的初始托肯数根据具体加工任务确定，加工完毕晶圆库所初始托肯数为 0。当完整的加工流程完毕后，待加工晶圆库所托肯数为 0，转移至已加工库所。如图 3.5 中的库所 P_{16} , P_{17} 所示。

在建立加工模块、机械手模块以及晶圆存储模块后，可以组合得到整个设备的完整 Petri 网模型，如图 3.8 所示。在该模型中，库所 P_{16} 的托肯数 m 表示当前待加工晶圆的数量。模型中，各模块的逻辑通过变迁和有向弧连接，系统地表达了晶圆在设备中的加工流程及资源调度关系。

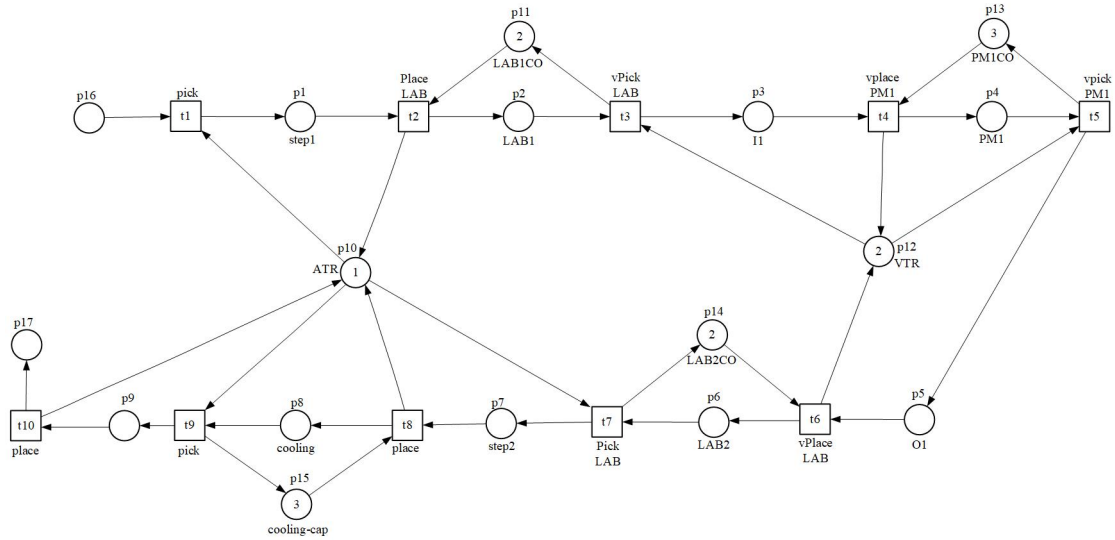


图 3.8 加工系统的 Petri 网模型

在建立系统整体的 Petri 网模型后，需要为模型中的库所和变迁引入时间因素，以准确表示组合设备中加工模块的加工过程、机械手的移动及抓取等操作。引入时间信息后，可以进一步对系统的调度方案进行研究。本文采用 PTPN 来建模组合设备系统的时间特性。图 3.8 中的 Petri 网模型包含的库所与变迁的具体含义以及相应的时间信息详见表 3.2。

表 3.2 图3.8的库所及变迁含义

库所与变迁编号	含义	时间信息 (s)
$p1$	晶圆大气端移动至 LL	1.5
$p2$	LL 抽气, 大气转换至真空	27
$p3$	晶圆真空端移动至 PM	2.5
$p4$	PM 加工	271
$p5$	晶圆真空端移动至 LL	2.5
$p6$	LL 放气, 真空转换至大气	12
$p7$	晶圆大气端移动至冷却模块	15
$p8$	冷却	64
$p9$	晶圆加工完毕, 放入 LP	15
$p10$	ATR 机械手模块	15
$p11$	真空锁抽气复位	12
$p12$	VTR 机械手模块	25
$p14$	真空锁放气复位	27
$p13, p15$	PM 与冷却模块容量	0
$p16, p17$	晶圆加载模块 LP	0
$t1, t7, t9$	ATR 取操作	4.5
$t2, t8, t10$	ATR 放操作	4.5
$t3, t5$	VTR 取操作	8.6
$t4, t6$	VTR 放操作	8.6

3.2 基于 Petri 网和 Q-learning 算法的组合设备调度算法

3.2.1 Q-learning 算法

Q-learning 是一种经典的强化学习算法, 主要用于解决离散状态空间中的马尔可夫决策过程问题, 该算法通过探索环境并不断更新状态-动作值函数 $Q(s, a)$, 逐步优化策略以接近最优^[69]。具体而言, Q-learning 通过评估在特定状态下执行某一动作所能获得的预期回报, 结合试探性探索与经验积累, 不断调整 Q 值, 从而逼近最优策略。该算法无需依赖环境的先验知识, 并能够在充分探索的基础上保证收敛至最优值函数, 因此广泛应用于自动化控制、路径规划和资源调度等领域, 具有重要的实用价值。算法中 $Q(s, a)$ 的迭代公式为:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (3-1)$$

在式3-1中, $Q(s, a)$ 表示在当前状态 s 下执行动作 a 所对应的 Q 值; α 为学习率, 控制每次更新 Q 值时新信息的权重, 取值范围通常为 $0 < \alpha \leq 1$ 。当 α 较大时, 新信息对 Q 值更新的影响较大, 使模型能够快速适应环境变化; 而当 α 较小时, 模型

更倾向于保留历史经验，从而使更新过程更加平稳。 r 为即时奖励 (reward)，表示执行当前动作 a 所带来的直接回报； γ 为折扣因子，用于衡量未来奖励对当前决策的影响，其取值范围通常为 $0 \leq \gamma \leq 1$ 。当 γ 接近 1 时，Q 值的更新更加强调长期回报，即未来奖励对当前决策的影响较大；而当 γ 接近 0 时，更关注即时回报，适合短期决策场景。 $\max_{a'} Q(s', a')$ 表示在下一状态 s' 下可能执行的所有动作中最大的 Q 值，即最优动作对应的 Q 值。

Q-learning 算法的步骤通常为：

1. 初始化。初始化 Q 表，表中的 Q 值可以设置为任意值，然后设置学习率 α ，折扣因子 γ 和探索策略。
2. 迭代过程。从初始状态 s 开始，根据当前状态与探索策略选择一个动作 a ，执行动作 a 转移到下一状态 s' ，获得与之对应的奖励 r ，根据式 3-1 更新 $Q(s, a)$ 。当 s' 满足终止条件式，停止更新。
3. 收敛判定。如果所有状态和动作的 $Q(s, a)$ 值基本稳定，则认为算法已收敛，策略 $\pi^*(s) = \arg \max_a Q(s, a)$ 是最优策略。

在 Q-learning 算法的初始化阶段，选择合适的探索策略是至关重要的，因为它直接关系到算法在探索与利用之间的平衡。探索指的是尝试环境中的新动作，目的是发现潜在的高回报动作，避免陷入局部最优解；而利用则是基于已有的信息，选择当前看似最优的动作，以实现即时回报的最大化。强化学习的核心挑战之一就在于如何有效平衡探索与利用：过度探索可能导致学习过程收敛缓慢，增加计算资源的浪费，而过度利用则可能让算法过早陷入局部最优解，无法找到全局最优策略。

本文采用 ϵ -greedy 算法^[70]作为探索策略，该算法通过引入参数 ϵ 来控制随机选择与贪婪选择的比例。在每个决策步骤中，算法以概率 ϵ 随机选择一个动作（探索），以概率 $1 - \epsilon$ 选择当前状态下 Q 值最大的动作（利用）。当 ϵ 较大时，算法倾向于更多地进行探索，从而有机会发现潜在的最优动作，但收敛速度可能较慢；而当 ϵ 较小时，算法更倾向于利用已有的最优策略，收敛速度较快，但可能陷入局部最优解而无法找到全局最优策略。其具体步骤如算法 3.1 所示。

算法 3.1 ϵ -greedy 动作选择算法

- 1: 初始化 ϵ 的值 (如 $\epsilon = 0.1$)
 - 2: 在当前状态 s 下，根据以下规则选择动作 a :
 - 3: **if** 随机数 $Rand \in [0, 1)$ 满足 $Rand > \epsilon$ **then**
 - 4: 选择当前状态 s 下使 $Q(s, a)$ 最大的动作 (利用)
 - 5: **else**
 - 6: 随机选择一个动作 (探索)
-

3.2.2 基于 Q-learning 算法的组合设备调度算法设计

在获得晶圆加工流程中组合设备的 Petri 网模型后，为利用 Q-learning 算法获得有效的调度方案，需要针对系统特性合理设计算法的关键要素，包括状态空间、动作集和奖励函数。

状态空间需要准确描述系统的动态状态信息，例如设备的使用情况、各模块的任务分布以及晶圆的加工进度。通过全面刻画这些信息，算法能够准确感知当前系统的运行状态，为后续决策提供依据。根据 Petri 网的特点，本文使用 Petri 网的标识作为观测量，为 $1 \times n$ 的向量，其中 n 为 Petri 网中库所数量。

动作集对应系统中可执行的调度操作，例如机械手的抓取与放置动作、晶圆的加工步骤选择等。这些动作直接决定了系统状态的演变，并对调度方案的优化效果产生重要影响。因此，在设计动作集时需要充分考虑设备能力和工艺约束，确保其符合实际运行逻辑。结合组合设备的 Petri 网模型，可选择 Petri 网中的变迁集合作为算法的动作集。假设 Petri 网中包含 m 个变迁，则动作集可表示为 $Action = \{t_1, t_2, t_3, \dots, t_m\}$ 。Petri 网中的每个变迁对应组合设备中的具体操作，算法通过执行变迁并观察变迁后的系统状态，根据设定的奖励函数为该操作提供奖励值，从而更新 Q 表。经过多次训练后，Q 表中记录的 Q 值即可用于指导最终的调度方案，实现调度优化目标。

奖励函数是指导 Q-learning 算法优化的核心，通过为每个动作分配即时奖励，衡量其对系统性能的影响。奖励函数的设计应与优化目标一致，例如提升生产效率、减少设备空闲时间或降低能源消耗，从而有效引导算法快速收敛至全局最优调度策略。针对本文中组合设备的调度问题，目标是找到完工时间最短的调度方案，因此奖励函数设计以时间因素为核心，同时考虑系统可能进入的死锁状态等问题。具体而言，智能体在 Petri 网的当前标识 s 下，从可发生变迁集合中随机选择一个动作 a ，当变迁发生后，系统状态转移至下一标识 s' 。若 s' 为死锁状态，则分配一个极小的奖励值；若 s' 为系统完工状态，则分配一个极大的奖励值；对于其他状态，奖励值取变迁所用时间的负值，从而最小化调度总时间。奖励函数如式3-2所示，这样的奖励函数设计能够在满足系统约束的同时，引导算法优化调度策略，提升系统的整体效率。

$$R = \begin{cases} R_{\min}, & s' \in DeadlockStates, \\ R_{\max}, & s' = TargetState \\ -t, & \text{其他, 其中 } t \text{ 为变迁使能和发射所用时间。} \end{cases} \quad (3-2)$$

综合设计以上要素，可以有效利用 Q-learning 算法对 Petri 网模型进行调度优化，确保调度方案能够在满足工艺需求的前提下，实现资源利用率和生产效率的最大化。综上所述，基于组合设备 Petri 网模型的 Q-learning 算法步骤如算法3.2。

根据该算法进行训练后，生成训练曲线，并得到训练后的 Q 表，根据训练后的

算法 3.2 基于 Petri 网的 Q-learning 算法

输入: 学习率 α , 折扣因子 γ , 探索权重因子 ϵ , 迭代次数 $maxEpisode$, 每次迭代的最大步长 $maxStepPerEpisode$, 目标标识 $TargetState$, 奖励函数 R , 状态集 $State$, 动作集 $Action$

输出: 训练后的 Q 表

```

1: 初始化 Q 表, 将 Q 值赋为 0, 智能体的初始状态  $s_0$  即 Petri 网中的初始标识  $M_0$ , 当前状态  $s_{cur}$ 
2: while  $maxEpisode \geq 0$  do
3:   令  $s_{cur} = s_0$ ,  $maxEpisode--$ ;
4:   for  $i = 1$  to  $maxStep$  do
5:     if Q 表中不存在  $s_{cur}$  then
6:       | 在 Q 表中添加  $s_{cur}$ 
7:       | 随机生成一个在  $[0, 1]$  范围内的随机数  $Rand$ 
8:       if  $Rand > \epsilon$  then
9:         | 选取状态  $s_{cur}$  对应的 Q 值最大的动作  $a$ 
10:      else
11:        | 随机在当前状态  $s_{cur}$  下可发生动作集合中选取一动作  $a$ 
12:        | 智能体执行动作  $a$ , 得到下一个状态  $s_{i+1}$  即下一个标识  $M_{i+1}$ 
13:        | 通过奖励函数  $R$  计算对应的奖励  $R_i$ 
14:        | 更新 Q 值:  $Q(s_{cur}, a) \leftarrow Q(s_{cur}, a_t) + \alpha(R_i + \gamma \max_a Q(s_{i+1}, a) - Q(s_{cur}, a))$ 
15:        if  $s_{i+1} == TargetState$  then
16:          | break;
17:        | 更新当前状态  $s_{cur} = s_{i+1}$ 
    
```

Q 表得到组合设备的调度方案的具体步骤如算法 3.3

算法 3.3 根据 Q 表的调度方案生成算法

输入: 训练后的 Q 表, 初始状态 s_0 , 目标状态 $TargetState$

输出: 变迁序列 σ

```

1: 初始化当前状态  $s_{cur} = s_0$ 
2: while  $s_{cur} \neq s_0$  do
3:   选择当前状态  $s_{cur}$  下 Q 值最大的动作:  $a = \arg \max_a Q(s_{cur}, a)$ 
4:   将动作添加至变迁序列  $\sigma$ 
5:   执行变迁得到下一个状态  $s_{next}$ 
6:   if  $s_{next} == TargetState$  then
7:     | break;
8:   | 更新状态:  $s_{cur} = s_{next}$ 
    
```

3.3 基于 Petri 网和 DQN 算法的组合设备调度算法

上一节介绍了通过组合设备的 Petri 网模型和 Q-learning 算法训练智能体, 生成系统调度方案的方法。然而, 随着系统复杂度的提升, Petri 网模型可能面临状态空

间急剧膨胀的问题。当待加工晶圆数量增加时，系统的可达状态数目会显著增多，导致 Q 表的存储和更新开销呈指数级增长。这种“维度灾难”现象会导致训练曲线难以收敛，从而无法生成正确的调度方案。为应对此问题，本节引入深度强化学习中的 DQN (Deep Q-Network) 算法。DQN 算法通过神经网络对 Q 值函数进行近似，用参数化函数（如神经网络）替代传统的 Q 表。神经网络以系统状态作为输入，并输出每个动作对应的 Q 值，从而能够高效处理高维和连续状态空间，显著提升训练效率和算法适应性。

3.3.1 DQN 算法

深度强化学习 (Deep Reinforcement Learning, DRL) 是强化学习与深度神经网络相结合的一种方法，专注于解决高维复杂问题。DRL 通过利用深度神经网络强大的特征提取能力，将传统强化学习中基于表格或线性函数的值函数近似替换为神经网络表示，从而克服高维状态和动作空间的挑战。与传统强化学习相比，DRL 能够有效应对连续和高维环境下的复杂任务，适用于机器人控制、游戏 AI、资源调度和自动驾驶等实际场景。典型的 DRL 算法包括基于值函数的 DQN (Deep Q-Network) 及其改进版本（如 Double DQN、Dueling DQN），以及基于策略优化的 PPO (Proximal Policy Optimization) 和 A3C (Asynchronous Advantage Actor-Critic) 等。本节将使用 DQN 算法基于组合设备的 Petri 网模型对系统的调度方案进行研究。

DQN 算法的核心是使用一个深度神经网络来近似 Q-learning 中的 Q 值函数 $Q(s, a)$ ，并通过梯度下降进行更新。DQN 涉及的关键技术有经验回放机制 (Experience Replay) 和目标网络 (Target Network)。由于在传统的强化学习中，智能体每次与环境交互后立即使用最新的经验进行更新，这可能会引起训练不稳定和数据利用率低等问题。因此，引入经验回放机制，在智能体与环境交互后，将生成的每个经验 (s, a, r, s') 存储到一个固定大小的缓冲区 (Replay Buffer) 中，再从缓冲区中随机抽取一个小批量的经验用于训练神经网络，并使用抽样的经验计算损失函数，通过梯度下降更新神经网络的参数，从而提高了强化学习算法中的数据利用率。

由于神经网络参数的不断更新，可能会导致训练过程中的不稳定，引起训练过程中的振荡甚至发散。为解决该问题，DQN 中引入了目标网络。目标网络是与主网络结构相同但参数独立的网络，且两者更新方式不同。主网络负责实时更新，用于环境交互和训练，而目标网络用于计算目标值，更新缓慢，避免目标值频繁更新带来的不稳定。主网络通过梯度下降更新其网络参数，目标网络以较低的频率从主网络复制参数。这样，通过固定目标网络，减少了目标值的频繁波动，稳定了训练过程，使神经网络的梯度更新更加平稳，有助于训练收敛。

3.3.2 基于 DQN 算法的组合设备调度算法设计

上文中对组合设备的建模思路以及具体步骤已进行介绍，并得到了实际系统对应的 Petri 网模型。下面将对使用 DQN 算法与组合设备 Petri 网模型对系统调度方案的研究进行介绍。本节选用 MATLAB 平台的强化学习工具 (Reinforcement Learning Toolbox) 箱作为实验平台，对实验环境进行搭建，对系统进行研究。

MATLAB 的强化学习工具箱为设计、训练和验证强化学习算法提供了一个高效且直观的平台。该工具箱集成了多种经典与前沿算法 (如 Q-learning、DQN、PPO 等)，支持用户灵活定义代理和环境，以适应多样化的应用需求。借助 MATLAB 强大的数值计算和可视化能力，工具箱能够清晰地呈现训练过程中关键指标 (如累计奖励、损失函数变化等)，为用户分析算法性能、优化模型参数提供了直观的支持。

使用强化学习算法训练智能体解决问题一般分为以下几个步骤：提出问题，创建环境，定义奖励，创建智能体，训练智能体，模拟智能体，部署策略。

1. 提出问题。定义智能体需要学习的任务。本文需要训练智能体得到系统完工时间最短的调度策略；
2. 创建环境。定义智能体运行的环境，包括智能体与环境之间的接口以及环境的动态模型。
3. 定义奖励。指定智能体用于衡量其任务目标表现的奖励信号，并定义如何从环境中计算该信号。由于待解决的问题相同，故本节的奖励定义与前文中使用 Q-learning 算法时的定义相同；
4. 创建智能体。包括创建默认智能体，并配置智能体的学习算法。本节使用的算法为 DQN 算。
5. 训练智能体。使用定义的环境、奖励和智能体学习算法训练智能体的近似器。
6. 模拟智能体。通过模拟智能体与环境的交互评估训练后智能体的性能。
7. 部署策略。通过训练好的智能体得到对应的策略，并进行应用。

(1) 环境创建

在强化学习工具箱中搭建环境时，通常包括观测量定义、动作集定义、步进函数 (Step Function) 设计和重置函数 (Reset Function) 设计。观测量是环境当前状态的表征，智能体通过这些信息了解环境的情况，用以决策并选择合适的动作，以最大化累积奖励。动作集则是智能体可以执行的所有可能动作的集合，定义了其与环境之间交互的方式。步进函数用于描述环境如何根据智能体的动作更新状态，并返回下一状

态、奖励值以及终止标志。重置函数用于在新一轮训练或评估开始时，将环境初始化为起始状态，并返回初始观测值。

通常，环境内部的状态或辅助信息会存储在结构体 `LoggedSignals` 中。这些信息不直接作为智能体的观测量，但为环境的状态更新和奖励计算提供支持。在本实验中，`LoggedSignals` 被用于存储 Petri 网状态转移过程中需要迭代更新的关键变量，例如 Petri 网的当前标识、每个库所中托肯的剩余时间、每个状态下变迁的可发射剩余时间以及系统的全局时间等。这些内部信息支持环境动态变化的模拟，同时为智能体的训练和决策提供了基础。观测量定义为 Petri 网的当前标识，表示为一个长度为库所数量的行向量，而动作集对应 Petri 网的变迁集合。通过 `LoggedSignals` 的内部状态，智能体间接提取关键特征以准确表征系统可能发生的情况。在决策过程中，智能体仅依赖观测量，而无需关注先前的变迁序列，从而满足 MDP 的条件。这种设计使智能体能够有效学习和优化复杂离散事件系统的行为。

对于步进函数的设计，在本实验中智能体选择动作的过程，即为 Petri 发射变迁的过程，因此状态的更新为 Petri 网中信息的更新。步进函数的结构如算法3.4。步进函数在训练智能体的每个 Episode 中，负责在每一步选择动作并进行状态转移，同时根据下一个观测值计算当前的奖励。

算法 3.4 Step Function

输入: 环境信息 `LoggedSignals`, 智能体当前选择动作 `Action`;

输出: 下一观测量 `NextObs`, 对应奖励 `Reward`, 结束标志 `IsDone`, 环境信息 `LoggedSignal`;

```

1: 从环境信息 LoggedSignals 中获取当前观测量 obs = LoggedSignal.state;
2: 获取当前状态 obs 可发生变迁集合 EnabledTransitionSet
3: if Action  $\in$  EnabledTransitionSet then
4:   执行该动作, 更新 LoggedSignals;
5:   NextObs = LoggedSignals.state;
6: else
7:   NextObs = obs;                                     // 不更新环境信息
8: Reward = RewardFunction(NextObs);                   // 根据 NextObs 获取奖励 Reward
9: if NextObs  $\in$  目标状态 then
10:  IsDone = true;
11: else
12:  IsDone = false;
```

重置函数主要用于训练过程中每个 Episode 开始时，将环境恢复到初始状态。在本实验中，每个 Episode 开始时，需要将系统状态重置为初始状态，同时初始化每个库所中托肯的时间、每个变迁的剩余触发时间以及全局时间。

(2) 奖励函数

对于事件驱动的系统，给定策略 $\pi(a|s)$ ，智能体在与环境交互过程中的累计奖励

回报 $G(t)$ 可表示为式 3-3。

$$G(t) = \sum_{k=0}^{\infty} \gamma^k r(s_k, a_k, s_{k+1}) \quad (3-3)$$

其中：

- k 表示离散时间步索引，每次智能体执行动作后，时间步加一。
- γ 为折扣因子，通常取值范围为 $0 < \gamma \leq 1$ ，用于权衡短期和长期奖励。
- 奖励 $r(s_k, a_k, s_{k+1})$ 由环境反馈给智能体，衡量在状态 s_k 执行动作 a_k 后的收益。

在标准 DQN 训练中，智能体通过最大化期望累计奖励来优化策略。Q 值函数的更新遵循贝尔曼方程：

$$Q(s_k, a_k) = r(s_k, a_k, s_{k+1}) + \gamma \max_{a'} Q(s_{k+1}, a') \quad (3-4)$$

其中， $\max_{a'} Q(s_{k+1}, a')$ 表示在下一状态 s_{k+1} 选择最优动作的 Q 值估计，从而引导智能体朝长期最优方向学习。由上述公式可知，智能体的累计奖励回报与折扣率 γ 密切相关。由于本文旨在研究系统的最小完工时间，需要对整个加工过程进行全面评估，因此 γ 的取值应更接近于 1，以强调对长期奖励的关注，从而优化整体性能。

本文的调度算法以实现系统最小化完工时间为目标，同时有效规避死锁的发生。为此，奖励函数的设计具有明确的针对性：对于死锁状态，奖励函数会分配一个较大的负奖励，明确惩罚系统进入不可恢复的状态。在智能体训练过程中，由于其每一步可能随机选择当前状态下不使能的变迁，为避免这种情况，奖励函数同样会给予较大的负奖励，从而引导智能体避开选择不使能的变迁。对于系统中可执行的动作，则依据其发生所需的时间代价赋予负值作为奖励，以激励智能体选择更短的操作时间，从而实现时间最小化目标。当系统完成加工任务时，奖励函数则会给予一个较大的正奖励，以强化智能体对最终完成任务的偏好。奖励函数如式3-5。

$$R = \begin{cases} -r_1, & s_{t+1} \in \text{死锁状态;} \\ -r_2, & a_t \notin \text{当前状态使能变迁集合;} \\ r_3, & s_{t+1} = \text{完工状态;} \\ -(t_{enabling} + t_{firing}), & \text{其他情况} \end{cases} \quad (3-5)$$

其中， r_1 、 r_2 、 r_3 均为正数，用于表示智能体在不同状态下的奖励与惩罚。 $t_{enabling}$ 和 t_{firing} 分别表示当前变迁使能所需的时间代价和发射该变迁所需的时间，两者相加即为发射该变迁的总时间代价。

(3) 深度神经网络设计

在深度强化学习中，深度神经网络作为函数逼近器，发挥着核心作用。它主要用于逼近复杂的非线性函数，例如状态值函数 $V(s)$ 、动作值函数 $Q(s, a)$ 或策略函数 $\pi(a | s)$ 。传统强化学习方法在面对高维连续状态空间时，往往难以有效建模和优化。而深度神经网络凭借其强大的表达能力，能够从高维输入中自动提取特征，并精准地建模状态与动作之间的复杂非线性关系，从而有效提升算法在复杂场景下的性能。因此，深度神经网络已成为深度强化学习不可或缺的核心。

在 DQN 等值函数强化学习方法中，深度神经网络通过输入状态或状态-动作对，输出相应的值函数 $Q(s, a)$ ，并通过训练学习最优值函数。多层神经网络的结构能够提取高维原始输入的特征，简化状态表示，提高学习效率。通常，深度神经网络由以下三个部分组成：

1. 输入层：用于接收环境的状态向量或原始数据，作为网络的输入。
2. 隐藏层：由多个神经元组成，通常包括卷积层或全连接层，并通过激活函数引入非线性特性。卷积层通常用于图像处理任务，全连接层则广泛应用于一般数据处理场景。
3. 输出层。根据具体任务输出值函数、策略分布或其他结果。在 DQN 中，输出层的节点数等于动作集的大小，每个节点对应一个动作的 $Q(s, a)$ 值。

这种网络结构结合了特征提取与非线性映射能力，为强化学习提供了强大的表达能力和灵活性。

在本实验中，深度神经网络设计为四层结构，包括输入层、两层隐藏层和输出层，如图3.9所示。输入层用于接收环境的观测量，其神经元数量等于观测量的维度。隐藏层由全连接层和激活函数层组成。全连接层负责对输入状态特征进行初步提取，其神经元数量通常选择为 2 的幂次，这种设计既能提升模型性能，又有助于保持计算的高效性。增加神经元数量可以增强模型的表达能力，但同时也会显著增加训练时间和存储需求。激活层采用 ReLU (Rectified Linear Unit) 激活函数，引入非线性，以使网络能够学习更复杂的状态特征映射，同时确保计算的高效性。输出层的神经元数量等于动作空间的大小，每个输出神经元对应一个动作的 $Q(s, a)$ 值。

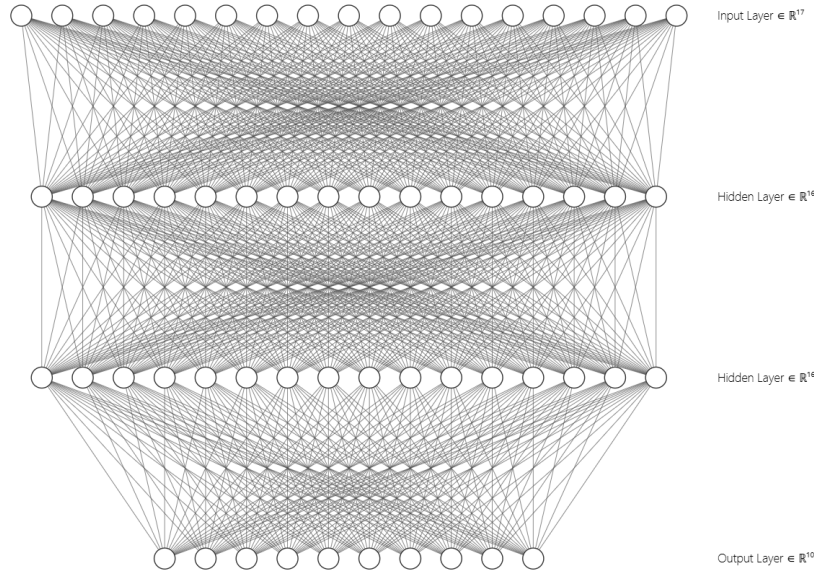


图 3.9 深度神经网络结构示意图

在创建好环境和智能体参数后，就可以使用 DQN 算法开始训练智能体，得到系统的完工时间最小的调度方案。基于 DQN 的智能体训练流程算法如算法3.5所示。

算法 3.5 基于 DQN 的组合设备调度模型训练算法

输入：最大 Episode 数 $MaxEpisodes$ ，每个 Episode 的最大步数 $MaxStepsPerEpisode$ ，未训练的智能体 $agent$ ，主网络参数 θ ，目标网络参数 θ^- ，更新频率 f ；

输出：训练后的智能体 $agent$

```

1: for  $i = 1$  to  $MaxEpisodes$  do
2:   调用  $ResetFunction$ ，进行初始化；
3:   for  $j = 1$  to  $MaxStepsPerEpisodes$  do
4:      $agent$  根据 epsilon-greedy 策略，基于当前观测值  $obs$  选择动作  $Action$ ；
5:     调用  $StepFunction$  执行动作  $Action$ ，状态转移
6:      $[NextObs, Reward, IsDone, LoggedSignals] = StepFunction(Action, LoggedSignals)$ ；
7:     生成经验  $(obs, Action, Reward, NextObs, IsDone)$ ，并存储到经验缓冲区；
8:     if  $IsDone == true$  then
9:       当前 Episode 结束
10:    从经验缓冲区随机采样一批经验  $(obs, Action, Reward, NextObs, IsDone)$ 
11:    计算目标 Q 值  $y = r + \gamma * \max_{Action'} Q(NextObs, Action'; \theta^-)$ ， $\theta^-$  为目标网络参数
12:    根据损失函数  $L(\theta) = \frac{1}{N} \sum (y - Q(s, a; \theta))^2$  更新 Q 网络的参数；
13:    根据更新频率  $f$  更新主网络参数到目标网络： $\theta^- \leftarrow \theta$ 
    
```

3.3.3 加权组合奖励函数

在传统的奖励设计中，奖励函数通常仅考虑单步时间惩罚，并在特定目标状态（如完工状态或死锁状态）施加额外的奖励或惩罚。前文所述的奖励函数即基于此原

则，如式 3-5 所示。这种设计直观且易于实现，但在处理复杂的调度问题时存在一定的局限性。由于奖励信号较为单一，智能体在训练过程中可能难以获取足够的信息来优化调度策略，从而导致收敛速度较慢、策略陷入局部最优，甚至难以全面提升系统整体性能。。

为了解决这些问题，本文在基础奖励的基础上，进一步引入进度奖励和设备利用率奖励，并通过加权组合的方式优化奖励设计。这种改进能够为智能体提供更丰富的反馈信息，使其在学习过程中不仅关注任务完成时间，还能够兼顾资源利用率与任务推进情况，从而更高效地优化调度方案，提升系统整体性能。

具体而言，加权组合奖励函数如式3-6所示。在此基础上，该函数充分利用了中间状态的信息，通过计算加工进度与设备利用率，并结合适当的权重，将其纳入奖励值的计算。当当前状态 s_t 选择的动作 a 不可执行时，给予较小的惩罚 r^- ；若动作可执行但导致系统进入死锁状态，则施加更大的惩罚 r^{--} 。当任务正确完成时，奖励值的大小取决于完工时间 t_{finish} 与训练过程中已记录的最小完工时间。若 t_{finish} 小于历史最优完工时间，则给予极大的奖励值 r^{++} ，以鼓励更优的调度方案；否则，给予较小的奖励 r^+ 。在训练初始化阶段，最小时间通常可设置为其他算法的结果。对于其他中间状态（即动作可执行但尚未完成全部任务），奖励值综合考虑单步时间惩罚、加工进度以及设备利用率的贡献，使调度策略不仅追求较短的完成时间，同时优化资源分配，提高系统整体效率。

$$R = \begin{cases} r^{--}, & s_{t+1} \in \text{死锁状态}; \\ r^-, & a_t \notin \text{可发生变迁集合}; \\ r^+, & t_{\text{finish}} > \text{minTime}; \\ r^{++}, & t_{\text{finish}} \leq \text{minTime}; \\ -(t_{\text{enabling}} + t_{\text{firing}}) + \omega_1 \cdot \text{Progress} + \omega_2 \cdot \text{Utilization}, & \text{其他情况}. \end{cases} \quad (3-6)$$

权重系数 ω_1 和 ω_2 分别用于平衡设备利用率与进度奖励在 Q 值更新中的影响，其取值需结合实验数据调优，以确保各奖励项对优化目标的有效性。加工进度 *Progress* 通常定义为已完成任务数量与总任务数量的比值，反映系统整体的生产进展情况。而设备利用率的计算公式如式 3-7 所示，该指标综合考虑设备负载、容量和权重，以衡量系统资源的利用情况。其中，单台设备的利用率由其当前负载与容量比值决定，而设备权重基于标准加工时间归一化，以确保加工时间较长的设备对整体利用率的影响更为显著。最终，通过加权平均计算整体设备利用率，使其更准确地反映系统运行状态，从而为调度优化提供合理依据。

$$Utilization = \frac{\sum_{i=1}^N \left(\frac{L_i}{C_i} \cdot W_i \right)}{\sum_{i=1}^N W_i} \quad (3-7)$$

设备权重 W_i 计算如下：

$$W_i = \frac{T_i}{\max(T)} \quad (3-8)$$

其中：

- U 为整体设备利用率；
- N 为设备数量；
- L_i 为第 i 台设备的当前负载；
- C_i 为第 i 台设备的容量；
- W_i 为第 i 台设备的权重；
- T_i 为设备的标准加工时间；

相较于传统的单步奖励函数，加权组合奖励函数充分考虑了实际生产中的多种关键因素。此外，对于像 Petri 网这种状态空间较大的模型，加权组合奖励函数通过对中间状态提供有效引导和反馈，有助于防止模型在优化过程中过早陷入局部最优，从而提升整体系统性能，确保在长期目标驱动下局部优化不会对整体调度效果产生负面影响。如果调度问题还需要考虑系统能耗、交付率等其他因素，可以根据这些因素的重要性设计相应的权重，以获得同时满足多个调度目标的综合调度方案。

3.4 本章小结

本章首先介绍了组合设备的相关概念，并阐述了利用 Petri 网对组合设备进行建模的方法。随后，在此基础上，设计了一种基于强化学习和 Petri 网的组合设备调度算法。该算法利用 MATLAB 平台的强化学习工具箱构建了环境，并设计了状态空间、动作空间、步进函数、重置函数以及基础奖励函数。在此基础上，还设计了考虑多因素的加权组合奖励函数，以进一步优化调度效果。最后，分别采用 Q-learning 算法和 DQN 算法训练智能体，研究其在系统调度方案中的应用效果。

第四章 基于时间自动机和模型检测方法的组合设备调度

本章提出了一种基于时间自动机的组合设备建模方法，并针对实际的组合设备进行建模。通过使用 UPPAAL 模型检测工具构建时间自动机模型，结合 TCTL 语言描述系统性质，并对其进行验证，从而得到最优的调度方案。

4.1 UPPAAL 模型检测工具

传统的设备调度方法主要包括规则算法、启发式方法和优化算法。这些方法在不同场景中各具优势，但在解决复杂的组合设备调度问题时存在一定的局限性。规则算法以其简单直观、易于实现的特点适用于问题规模较小的场景，能够快速给出决策方案。然而，规则算法的局限性在于其缺乏全局性，通常只能提供次优解，并且依赖于人工设计的规则，难以推广到复杂或动态的调度问题中。启发式方法通过引入启发规则，在大规模调度问题中展现了良好的求解能力和灵活性。然而，这种方法可能因为启发规则的不完备而陷入局部最优，同时对时间约束和系统动态变化的应对能力较弱。优化算法则凭借数学模型的严格性，在理论上可以提供全局最优解，但其高计算复杂度使得在实际应用中往往难以满足实时性需求，尤其在处理大规模问题时表现出计算资源的瓶颈。此外，优化算法对不确定性因素的处理能力有限，使其适用性受到一定限制。

总体来看，传统方法在处理复杂调度问题时暴露出多方面的不足。首先，许多方法在全局性上表现较差，难以综合考虑多个目标或全局约束，尤其是在动态环境中缺乏统一的优化能力。其次，这些方法通常缺乏形式化的验证手段，无法保证生成的调度方案是否满足系统的关键性质，例如无死锁、任务按时完成等。此外，时间约束是设备调度中的核心问题之一，但传统方法往往难以精确处理复杂的时间依赖性，如任务之间的优先级和设备切换时间。最后，传统方法对动态性和不确定性的适应能力较弱，例如在面对设备故障或任务变化时，可能无法快速调整并重新验证调度方案。

相较之下，基于模型检测的方法弥补了传统方法的诸多不足。模型检测能够全面遍历状态空间，保证解的全局性，并结合时序逻辑对系统性质进行形式化描述与验证。这使得调度方案不仅满足全局最优或次优要求，还可以通过工具支持验证其是否符合时间约束、资源分配等复杂规则。同时，模型检测对时间自动机等模型的良好支持，使其在处理调度问题中的时间约束时具有显著优势。此外，通过动态分析状态变化，模型检测能够适应系统的突发情况，为在线调度提供有效支持。在工具支持方面，UPPAAL、SPIN 等模型检测工具具备强大的建模、仿真和验证能力，大幅降低了

复杂问题的实现难度。基于这些优势，模型检测方法为组合设备调度问题提供了一种系统化、高效且可验证的解决方案。

UPPAAL 是由丹麦 Aalborg 大学和瑞典 Uppsala 大学联合开发的一种专用于建模、仿真和验证实时系统的模型检测工具^[71]。它聚焦于具有时间约束的离散事件动态系统的分析，基于时序自动机理论，为用户提供了强大的工具支持，用于定义状态、事件和时间约束，从而精准地表达系统的行为逻辑和实时性需求。UPPAAL 提供直观的图形化界面，用户可以通过绘制自动机以及定义变量、时钟等元素，便捷地构建系统模型。

作为一种高效的实时系统验证工具，UPPAAL 在通信协议验证、嵌入式系统设计、调度优化和工业控制等领域被广泛应用。在本研究中，UPPAAL 被用于构建系统模型并验证所设计方案的正确性与时序性能，以确保其符合具体应用场景的需求。

UPPAAL 主要由编辑器，模拟器和验证器三个部分组成。编辑器是 UPPAAL 的核心建模工具，提供了直观的图形化界面，用于构建时序自动机模型。在 UPPAAL 模版中的自动机模型，通常包含下以下两个关键元素：

1. 位置 (Location)：是 UPPAAL 模型中的核心构件，用于描述自动机的具体位置或状态。在可视化编辑器中，位置以节点的形式呈现，用于表示系统当前的运行状态。根据功能和特性，UPPAAL 中的位置分为普通位置、初始位置、提交位置和紧急位置。普通位置是系统的一般状态，无特殊属性；初始位置表示系统的起始状态，通常在建模中作为运行的起点；提交位置要求系统必须立即离开该位置，不能停留，用于保证某些操作的优先执行或避免不必要的延迟；紧急位置则表示系统在此位置不能流逝时间，需立即完成状态迁移，常用于建模关键路径中的强制执行逻辑。这些位置类型通过表达不同的动态行为和时间约束，为实时系统的精确建模和验证提供了强有力的支持。
2. 边 (Edge)：用于描述自动机从一个位置到另一个位置的状态变化。边用带箭头的连线表示，连接两个位置，表示系统在满足特定条件时可以从源位置移动到目标位置。在 UPPAAL 中，边通常由选择 (Select)、条件 (Guard)、同步 (Synchronization) 和更新 (Update) 等要素构成。Select 用于定义非确定性选择变量，从而建模系统中的不确定性；Guard 通过布尔表达式或时钟约束限制迁移的触发条件，确保边在特定情况下发生；Synchronization 通过信道实现自动机之间的通信，用于协调多个自动机的迁移行为，其中发送！与接收？必须成对出现；Update 则是在状态转移过程中执行的操作，用于修改系统变量或时钟的值，以反映系统状态的动态变化。这些元素共同描述了边的触发条件、执行逻辑和系统状态的变化方式，是建模动态行为的核心组成部分。

用户可以通过拖放操作添加位置、边、时钟、变量和同步信号，并为转换定义条件和更新操作。编辑器还支持全局变量和模板功能，通过模块化建模简化了复杂系统的构建过程，使用户能够快速创建清晰且结构化的模型。

模拟器是一个行为分析工具，用于动态验证模型的正确性和可行性。用户可以通过模拟器以交互方式探索模型的执行路径，直观地观察状态和变量的变化。模拟器支持逐步仿真和随机仿真两种模式，帮助用户验证系统行为是否符合预期。此外，模拟器还提供时间进程的可视化功能，便于分析系统的时序特性，从而为模型优化和设计改进提供参考依据。

验证器是一个强大的模型检测工具，利用时 TCTL 对模型的属性进行验证。用户可以通过输入查询语句，如“某状态是否可达”或“某行为是否在规定时间内发生”，来检查系统是否满足安全性、活性和时序约束等特定要求。验证器基于高效的符号算法，能够处理大规模复杂模型并快速生成验证结果。如果某些属性未被满足，验证器还会生成反例路径，帮助用户分析问题根源并改进模型设计。

4.2 组合设备的时间自动机建模

时间自动机因其对时间约束的精准表达能力，在自动化制造系统建模中备受关注。它能够直接引入时间变量，为每个状态和转换定义明确的时间约束和触发条件，从而便捷地描述加工时间、等待时间以及任务超时等关键因素。时间自动机的建模过程既直观又灵活，尤其在多任务协同和复杂动态行为的控制中，通过时间条件对系统状态实现精细化管理。此外，当时间自动机与强化学习等智能算法结合时，可以充分利用时间维度优化调度方案，进一步提升系统运行效率。

与时间 Petri 网相比，时间自动机的优势主要体现在建模的简洁性和时间控制的直观性。尽管时间 Petri 网通过扩展引入了时间约束，其模型仍以标记流动为核心，时间的表达方式相对间接。在复杂系统中，往往需要额外的逻辑来管理时间行为。相比之下，时间自动机通过时钟变量直接实现时间管理，避免了附加复杂性的引入。同时，时间自动机在状态转换规则中能够直接嵌入时间约束条件，使其在动态调度和实时优化场景中具有更大的优势。然而，时间 Petri 网在处理并发事件和离散行为建模方面仍然表现突出，作为一种补充工具，具有重要价值。

在本章中，利用时间自动机模型对 AMS 进行建模的思路与 POPN 的建模方法有相似之处，但在实现上存在显著差异。与 POPN 仅通过单一模型对系统进行建模不同，AMS 的时间自动机模型由多个子模型构成。这些子模型各自承担不同的功能，例如描述工作流程、定义系统约束等。通过这种模块化建模方式，各子模型之间通过同步通道进行通信与协调，形成相互制约的关系，从而完整地表达整个 AMS 的动态行为。这种多子模型的设计提高了建模的灵活性和扩展性，使其能够更精确地刻画复杂

系统的运行机制。

对于组合设备的时间自动机建模，主要包括晶圆加工流程建模、系统机械臂模块建模、各类资源模块建模以及系统运行状态建模。针对3.1.3中描述的组合设备系统，其组成包括单臂机械手、双臂机械手、真空锁模块、加工模块和冷却模块。以下将以加工配方 $LP \rightarrow LL \rightarrow PM \rightarrow LL \rightarrow \text{cooling} \rightarrow LP$ 的晶圆为例，详细阐述其时间自动机的建模方法。

4.2.1 加工任务流程自动机

与 POPN 的建模思路相似，首先建立晶圆加工流程的自动机 $Task(i)$ ，来表示晶圆加工的顺序流程。其中 i 表示待加工晶圆数量，整个时间自动机网络中会有 i 个 $Task$ 自动机，来表示系统同时对 i 个晶圆进行加工。如图4.1所示，为系统的晶圆加工流程任务自动机 $Task(i)$ ， i 代表第 i 个待加工晶圆。

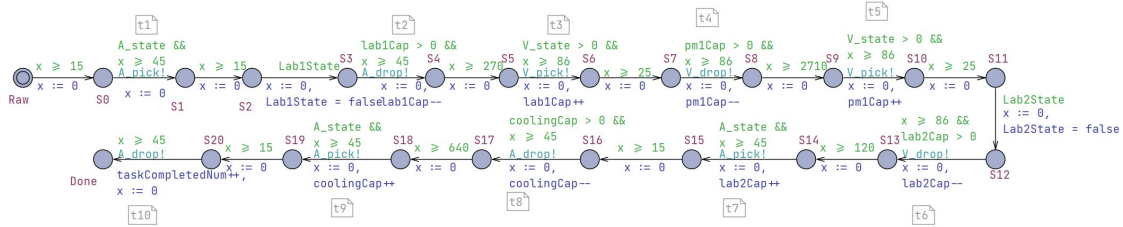


图 4.1 晶圆加工流程自动机

在该模型中，实心圆圈表示时间自动机的位置，有向箭头表示状态转移的边。边上标注有守卫条件、同步通道和变量更新，分别以绿色、青色和蓝色字体表示。当守卫条件满足时，若该边设有同步通道，则会发送相应的同步信号，并在完成变量更新后，触发位置的转移。在该模型中，从位置 $Task(i).Raw$ 转移到 $Task(i).Done$ 表示第 i 个晶圆的加工已完成。图 4.1 所示的变量及其含义详见表 4.1。

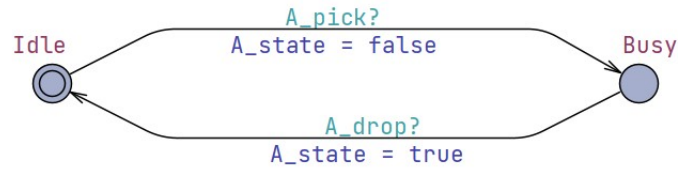
该模型的工作逻辑如下：当系统开始加工任务时，每个 $Task$ 自动机对应一个待加工晶圆的完整加工流程。晶圆由两台机械臂协作搬运，依次进入各个加工场所进行加工。在晶圆进入加工场所后，系统会调度空闲的机械臂前往该场所待命，确保加工完成后能迅速进行转移。在机械臂执行搬运操作的过程中，相应的同步信号会被触发，从而协调不同自动机的并行运行，保证整个系统的高效协同工作。

4.2.2 机械手自动机

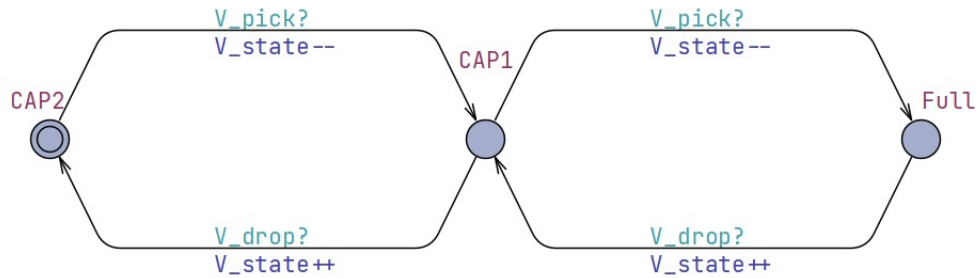
对于单臂机械手，其同时仅能抓取一个晶圆，因此在进行一次抓取操作后，必须再进行放回操作，才能进行下一次抓取。而对于双臂机械手，其同时可抓取两个晶圆。针对两种机械手的容量与工作逻辑，分别建立其自动机模型如图4.2(a)和图4.2(b)所示。

表 4.1 $Task(i)$ 自动机变量及含义

变量名	含义
x	$Task(i)$ 局部时钟
A_{state}, V_{state}	机械手容量
$lab1Cap, lab2Cap, coolingCap, pm1Cap$	加工模块容量
$A_{drop}, A_{pick}, V_{drop}, V_{pick}$	机械臂动作同步信号
$Lab1State, Lab2State$	Lab1, Lab2 是否准备就绪标志
$taskCompletedNum$	加工完成的晶圆数量



(a) 单臂机械手 ATR 自动机



(b) 双臂机械手 VTR 自动机

图 4.2 机械手自动机模型

在图 4.2(a) 所示的时间自动机模型中，初始状态为 *Idle*，表示 ATR 处于空闲状态。当 $Task(i)$ 自动机发送同步信号 A_{pick} 时，ATR 由空闲状态 *Idle* 转移至工作状态 *Busy*。在 *Busy* 状态下，ATR 机械臂无法执行其他移动操作，直至其在工作状态下接收到同步信号 A_{drop} ，此时 ATR 由 *Busy* 状态恢复至空闲状态 *Idle*，等待下一次任务请求。

如图 4.2(b) 所示，VTR 自动机包含三个状态：*CAP2*、*CAP1* 和 *Full*，分别表示 VTR 剩余容量为 2、1 以及已满的状态。初始状态为 *CAP2*，即系统启动时 VTR 处于最大容量状态。当 $Task(i)$ 自动机发送同步信号 V_{pick} 或 V_{drop} 时，VTR 自动机根据当前状态进行相应的转换。当 VTR 处于 *Full* 状态时，仅允许执行放回操作，

以确保系统资源的正确管理和整体运行的稳定性。

4.2.3 资源场所自动机

在晶圆加工系统中，资源场所通常分为两类：需要准备时间的加工场所和无需准备时间的加工场所。对于需要准备时间的加工场所（例如，大多数对晶圆进行工艺处理的场所），通常在实际操作之前必须进行必要的准备工作。其工作逻辑是：当当前场所的一项资源被上一个晶圆占用结束后，场所便会立即开始为下一次加工进行准备工作。其工作逻辑可表示为图4.3，图中变量名称及含义如表4.2。

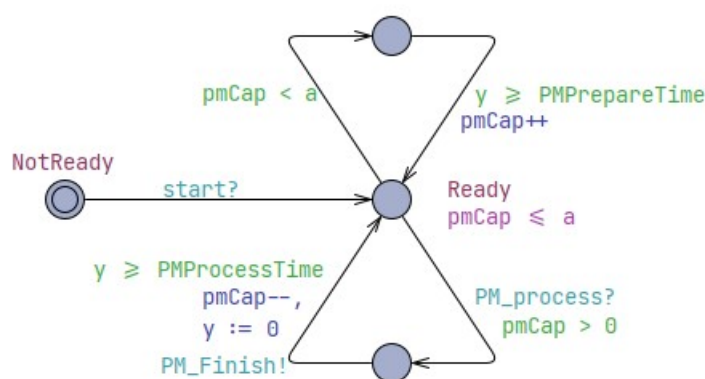


图 4.3 需要准备时间的加工场所工作逻辑

表 4.2 需要准备时间的加工场所自动机变量及含义

变量名	含义
<i>start ?</i>	同步通道，接收信号后，系统开始运行
<i>pmCap</i>	设备当前剩余容量
<i>y</i>	自动机局部时钟
<i>PMPrepareTime</i>	准备所需时间
<i>PMProcessTime</i>	加工时间
<i>PM_Process ?</i>	同步通道，接收信号后，PM 开始进行加工操作
<i>PM_Finish !</i>	同步通道，当 PM 完成加工后，发送该信号

在图4.3 中，共包含四个位置，其中未就绪状态 *NotReady* 和就绪状态 *Ready* 为主要状态，其余两个未命名状态为过渡状态，主要用于在发射同步信号之间实现条件满足时更新资源场所的容量信息。假设场所的容量为 a ，整个系统的工作逻辑如下：

系统初始处于未就绪状态 *NotReady*，当接收到开始工作信号 *start* 后，状态转移至就绪状态 *Ready*。此时，所有资源尚未完成准备工作，因此变量 *pmCap* 初始化

为 0。此后，系统在满足 $pmCap < a$ 的条件下进行状态转移，进入准备工作流程。准备工作完成后，更新剩余容量变量 $pmCap$ 。由于设备容量为 a ，系统在 *Ready* 位置处设置不变量，以确保 $pmCap$ 不超过设备容量 a 。

在需要进行加工时，系统会先检查是否存在准备完成的场所。如果有，则进行加工并更新相关变量；如果没有，则等待场所的准备工作完成后开始加工。这种设计保证了当场所剩余容量小于设备总容量时，系统能够持续进行准备工作，直到所有资源均已准备完毕。

然而，这种建模方式会导致各加工场所自动机中的局部变量数量过多。在 UP-PAAL 中，整个系统的全局状态空间可视为各自动机状态及其变量值的笛卡尔积，因此，每增加一个局部变量，尤其是取值范围较大的变量，都会显著增加状态组合的数量，使状态空间呈指数级增长，从而影响验证效率和可扩展性。为在保持模型正确性和清晰性的前提下优化状态空间，我们尽量将局部变量集中在加工流程自动机 *Task* 中。修改后的加工场所自动机（如图 4.4 所示）仅用于模拟准备过程，而加工过程本身则通过 *Task* 自动机的一条边来表示。优化后的模型逻辑如下：当已准备好的资源小于设备容量时，触发准备操作，直至所有资源准备完毕。此设计不仅降低了加工场所自动机的复杂度，还减少了变量的分散存储，有效控制了状态空间增长，提高了模型验证的效率。

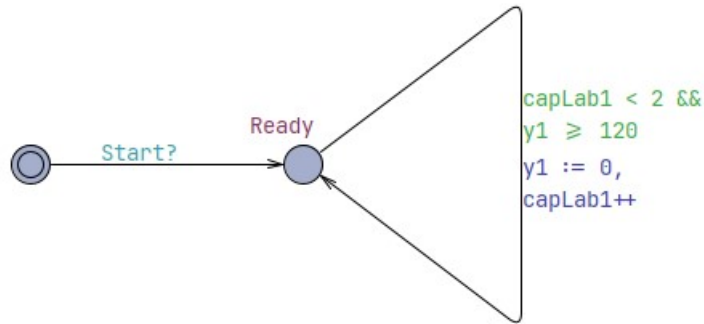


图 4.4 需要准备时间的加工场所自动机

对于无需准备时间的加工场所，可以使用变量直接表示设备容量，从而避免建立子自动机模型，进而减少状态空间的大小。例如，在图 4.1 所示的加工流程自动机中， $Task(i).S17 \rightarrow Task(i).S18$ 这条边表示冷却模块的加工过程。假设冷却模块的容量为 3，且工作时无需额外准备，即晶圆放入后便直接开始加工。因此，可以通过变量 $coolingCap$ 表示冷却模块的剩余容量，其工作逻辑如下：当 $coolingCap > 0$ 时，可以执行下一步加工操作，同时将 $coolingCap$ 减 1。通过这种方式，简化了对冷却模块的建模，同时显著优化了模型的状态空间规模。

4.2.4 系统状态自动机

系统状态自动机是用于表示当前系统运行状态的自动机，如图4.5所示。系统状态自动机包括四个状态：空闲 *Idle*，正常工作 *Working*，错误状态 *Error* 和完工状态 *AllTasksComplete*，其中 *Idle* 为初始状态。当系统开始工作时，会向各个加工场所发送 *start* 信号；在加工过程中，通过自定义函数 *isOverCapacity()* 来检测是否出现容量溢出或小于 0 的情况，若返回 *true* 系统将从正常工作状态转入错误状态。当所有晶圆的加工流程结束时，系统将转入完工状态。

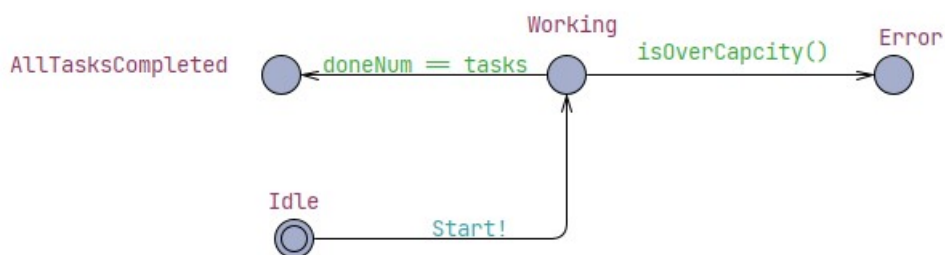


图 4.5 系统状态自动机

在完成各个子自动机的设计后，可以在 UPPAAL 编辑器中搭建完整的系统模型，其整体自动机网络如图 4.6 所示。通过同步通道和全局变量，各子自动机实现同步通信，从而有效模拟晶圆加工系统中各设备的协同工作过程。此外，对于同一设备的不同晶圆加工配方，仅需建模相应的加工流程自动机，并在系统声明中进行配置，即可使用同一 UPPAAL 模型对该设备的多个加工配方进行建模。基于前文的建模思路，其他实际组合设备的时间自动机模型构建如图 4.12 所示。

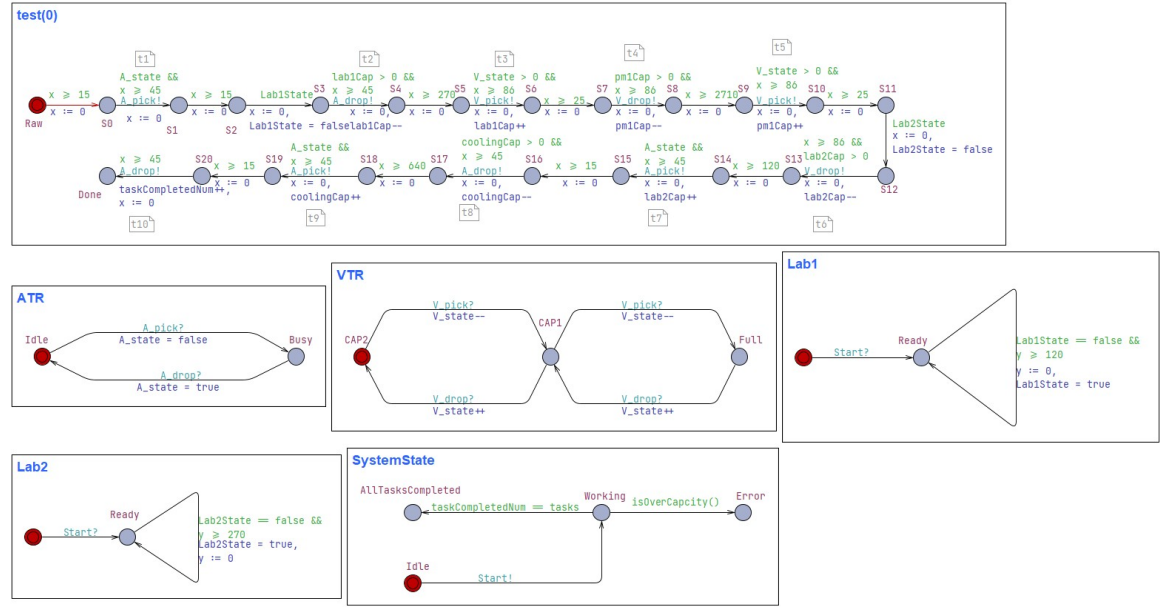


图 4.6 系统 UPPAAL 时间自动机网络

在介绍完组合设备时间自动机建模方法后，我们进一步对加工流程更为复杂的多组合设备展开建模，其示意图如图4.7所示。该加工流程为 $LA2 \rightarrow ChC \rightarrow ChA \rightarrow Ch1 \rightarrow Ch5 \rightarrow ChB \rightarrow ChF \rightarrow LA1$ ，其中，加工模块 CHA 和 CHB 分别作为两个组合设备之间的真空锁（即缓冲模块）来连接设备。依据前述建模思路，我们构建了相应的时间自动机网络，如图4.12所示，其变量与含义如表4.4。

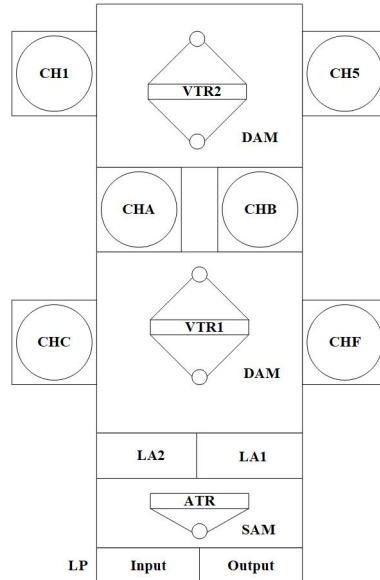


图 4.7 多组合设备示意图

4.3 基于模型检测的组合设备调度方法

模型检测是一种系统验证技术，通过遍历形式化模型（如本节中的时间自动机），来检查是否满足特定的时序性质。常见的时序逻辑包括线性时序逻辑 (Linear Temporal Logic, LTL)，计算树逻辑 (Computation Tree logic, CTL) 和时间计算树逻辑 (Timed Computation Tree Logic, TCTL) 等，UPPAAL 通常使用 TCTL 作为系统的性质描述逻辑。

4.3.1 UPPAAL 查询语言

模型检测器的核心目标是依据需求规范对模型进行验证。与模型本身类似，需求规范也必须采用一种形式上定义良好且能够被机器读取的语言来表达。在科学文献中，已经提出了多种适合此用途的逻辑语言，而 UPPAAL 使用了一种简化形式的 TCTL。与 TCTL 类似，UPPAAL 的查询语言 (Query Language) 由路径公式和状态公式组成^[72]。状态公式用于描述单一状态的特征，而路径公式则用于量化模型的路径或轨迹。路径公式可以进一步划分为三大类：可达性、安全性和活性，分别用于表达模型的状态是否可达、系统行为是否安全以及系统是否能够持续保持活跃性。

(1) 可达性

可达性通常用于模型设计过程中进行合理性检查。其目的是判断是否存在某个特定的状态公式 ϕ ，可能在任意可达状态中被满足。换句话说，可达性性质用于验证是否存在一条从初始状态出发的路径，使得给定的状态公式 ϕ 最终在该路径上得以满足。在 UPPAAL 中，使用 $E\Diamond\phi$ 表示某个可达状态满足 ϕ 。如图 4.8 所示，该图展示了满足 $E\Diamond\phi$ 的示意图，其中存在一条路径，该路径最终满足 ϕ 。

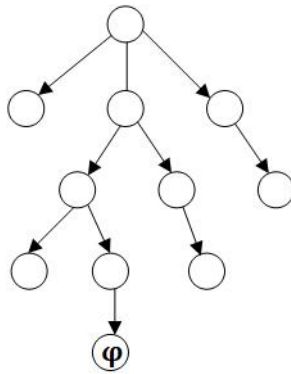


图 4.8 $E\Diamond\phi$

(2) 安全性

安全性通常用于验证系统是否能够始终避免某些不希望的行为或状态，换句话说，即确保某些不良事件永远不会发生，并保证系统始终满足特定的安全条件。常见的安全性性质包括：状态不变性（确保某个变量始终保持在指定范围内）和死锁检查等。在 UPPAAL 中，使用 $A\Box\phi$ 表示在所有可达状态中，给定状态公式 ϕ 均成立；使用 $E\Box\phi$ 表示存在一条路径，在该路径上状态公式 ϕ 始终成立。这两种语句的示意图分别如图 4.9(a) 和图 4.9(b) 所示。

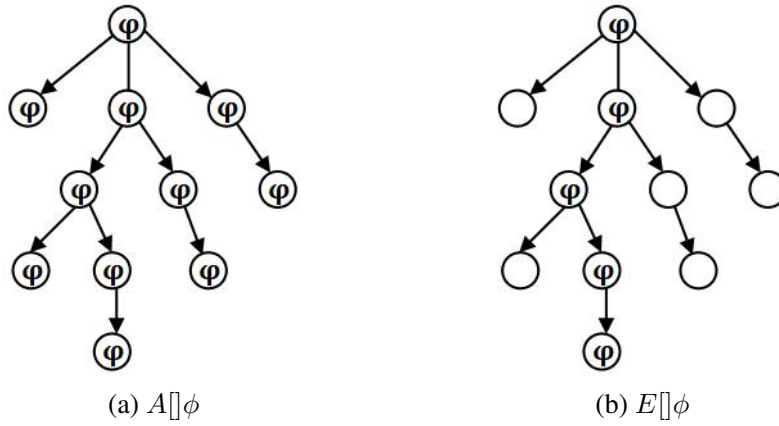


图 4.9 安全性逻辑示意图

(3) 活性

活性用于验证系统是否能够在未来某个时刻达到期望的状态或行为。它关注系统的“进展性”，确保“某些事情最终会发生”。在 UPPAAL 中，通常用 $A\Diamond\phi$ 和 $\phi \rightarrow \psi$ 来表示活性。前者表示对于所有路径，最终都会使状态公式 ϕ 满足，后者表示 ϕ 满足时， ψ 最终也满足，其示意图如图 4.10(a) 和 4.10(b) 所示。

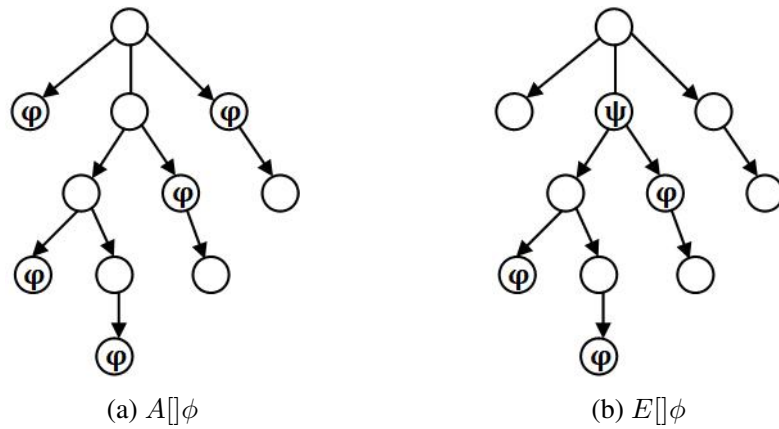


图 4.10 活性逻辑示意图

下面将以 Fischer 协议的 UPPAAL 模型为例，解释 UPPAAL 的查询语言。

例 4.1 Fischer 协议是一种经典的分布式系统算法，用于实现临界区的互斥访问。通过引入全局时钟和时间约束，该协议能够确保多个进程在访问共享资源时避免冲突，从而实现安全的资源共享。其 UPPAAL 模型如图4.11所示。

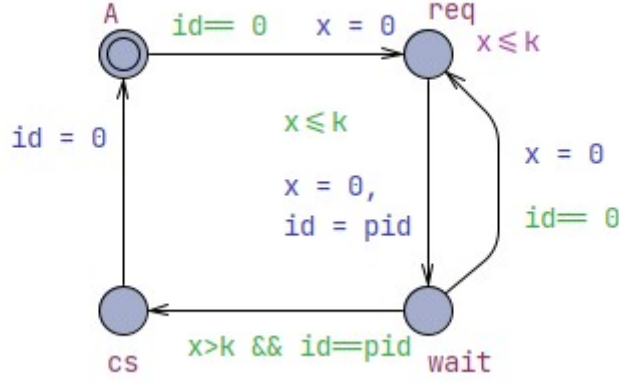


图 4.11 Fischer 协议自动机

Fischer 协议由四个位置组成：初始位置 A 、请求位置 req 、等待位置 $wait$ 和临界位置 cs 。系统可通过调整参数改变协议自动机的数量，以模拟不同的进程数量。以四个协议自动机 P_1, P_2, P_3, P_4 为例，当全局变量 $id == 0$ 时，进程进入 req ，并以非确定性方式停留 0 至 k 个单位时间，然后转移至 $wait$ ，将 id 设置为当前进程编号。在 $wait$ 位置，进程需至少等待 k 个单位时间，并在 id 等于自身编号时才能进入临界区 cs 。协议的核心在于：当 $id \neq 0$ 且经过 k 个时间单位后，所有等待进入临界区的进程均处于 $wait$ ，但只有一个进程的 id 匹配，可进入 cs 。进程退出临界区时会重置 id ，允许其他进程尝试重新进入 cs 。

在 UPPAAL 中搭建模型后，需要对其性质进行验证。首先使用 TCTL（时序计算树逻辑）描述其性质，然后通过 UPPAAL 验证器进行验证。待验证的性质包括以下内容：

1. 可达性

验证系统是否存在一条路径，使某个进程能够到达临界区 (cs)。其 TCTL 表达式为：

$$E\Diamond(P(1).cs + P(2).cs + P(3).cs + P(4).cs = 1)$$

2. 安全性

验证在所有路径上，临界区同时只能有一个进程，即满足互斥性。具体而言，对于任意 $i, j \in [1, 4]$ ，若 $P(i).cs \wedge P(j).cs$ 为真，则必有 $i == j$ 。其 TCTL 表达式

为:

$$A[\forall(i : id_t)\forall(j : id_t)(P(i).cs \wedge P(j).cs \implies i = j)]$$

此外, 还需检测系统中是否存在死锁路径。其 TCTL 表达式为:

$$A[not\ deadlock]$$

3. 活性

验证当某个进程请求进入临界区时, 最终能够进入等待位置。以进程 $P1$ 为例, 其 TCTL 表达式为:

$$P1.req \rightarrow P1.wait$$

通过以上性质验证, 可以确保系统的正确性和可靠性, 包括进程的可达性、安全性 (互斥性和无死锁) 以及活性。在 UPPAAL 中的验证结果如表4.3

表 4.3 Fischer 协议系统性质 TCTL 及验证结果

性质	结果
$E\Diamond(P(1).cs + P(2).cs + P(3).cs + P(4).cs = 1)$	满足
$A[\forall(i : id_t)\forall(j : id_t)(P(i).cs \wedge P(j).cs \implies i = j)]$	满足
$A[not\ deadlock]$	满足
$P1.req \rightarrow P1.wait$	满足

4.3.2 组合设备的性质验证及调度方案

本章已介绍了组合设备的时间自动机建模、模型检测工具 UPPAAL 以及 TCTL 语句。接下来, 将基于前述内容, 列举并验证组合设备时间自动机模型的待验证性质对应的 TCTL 表达式, 并利用 UPPAAL 诊断路径 (Diagnostic Trace) 生成功能, 推导出组合设备的调度方案。

UPPAAL 的诊断路径功能基于组合设备时间自动机模型构造的区域图, 通过状态空间搜索, 并采用正向或逆向的探索策略生成一条完整的执行轨迹。在这一过程中, 诊断路径生成方法类似于均匀成本搜索或 Dijkstra 算法, 对状态空间中每个节点的累计时间进行严格评估与剪枝, 从而确保最终获得的轨迹在满足模型性质的前提下, 实现了时间或步骤的最优化要求^[73]。该轨迹展示了从初始状态到达特定目标状态的状态转换序列。当待验证的 TCTL 性质成立时, 验证器会生成满足性质的诊断路径; 若性质不满足, 则生成反例路径。在此过程中, UPPAAL 不仅记录了每个状态对应的离散位置, 而且详细保存了各状态下时钟变量的取值范围以及转换发生的具体

时间，从而为调度方案提供了丰富的时序数据支持。

具体来说，模型检测完成后，针对某一待验证性质，UPPAAL 可生成多种类型的执行轨迹，例如最短路径（状态转换次数最少）、最快路径（实际耗时最短）以及随机路径。本文主要关注在最快路径模式下生成的诊断路径。该路径通过捕捉任务启动、任务完成以及关键状态到达等关键事件，并在所有满足性质的路径中选择耗时最短的一条，从而确保所得到的调度方案不仅符合验证要求，而且在实际运行中具有较高的时效性。借助这条诊断路径，我们可以清晰地识别各任务之间的前后依赖关系以及各调度点的具体时刻，进而提取出满足系统时间约束的最优调度策略。然而，由于 UPPAAL 采用的是穷尽式模型检查方法，验证过程中必须构造并存储整个状态空间。随着代理数量的增加，这个状态空间会呈现指数级膨胀，导致计算时间和内存消耗急剧上升，从而给验证过程带来明显的性能瓶颈。

在搭建好组合设备的 UPPAAL 模型后，我们需要对该模型的一些性质进行验证并在此过程中得到该系统的完工时间最短的调度方案。待验证性质如下：

(1) 可调度性

验证系统是否存在一条路径，使系统最终到达完工状态。其 TCTL 表达式为：

$$E \Diamond \text{SystemState.AllTasksCompleted} \quad (4-1)$$

或

$$E \Diamond \text{forall}(i : t_id) \text{Task}(i).Done \quad (4-2)$$

在验证可调度性时，我们将验证器的诊断路径模式设置为“最快”。验证结束后，如果满足调度条件，验证器便会生成一条全局时间最短且顺利完工的执行路径，其所需时间记为 $fastestTime$ ；反之，若验证结果不满足，则表明系统无法达到完工状态，从而无法实现调度。

(2) 调度方案的最优性

针对可调度性验证过程中生成的诊断路径，其全局时间记为 $fastestTime$ 。我们进一步检验这一指标的最优性，即验证所有达到完工状态的路径的全局时间是否均不低于 $fastestTime$ 。换句话说，我们需要确认是否存在任何一条完工路径，其全局时间小于 $fastestTime$ ，以确保 $fastestTime$ 真正代表了全局最优的调度方案。其 TCTL 为：

$$A [] (\text{SystemState.AllTasksCompleted} \implies \text{globalClock} \geq \text{fastestTime}) \quad (4-3)$$

或

$$E \Diamond (\text{SystemState.AllTasksCompleted} \text{ and } \text{globalClock} < \text{fastestTime}) \quad (4-4)$$

当式4-3验证结果为真或式4-4验证结果为假时，则证明 $fastestTime$ 即为最短完工时间。

(3) 安全性

验证系统中是否存在死锁状态以及系统是否会进入系统状态自动机中的错误状态，其 TCTL 如式4-5和式4-6。

$$A [] not deadlock \quad (4-5)$$

$$A [] not SystemState.Error \quad (4-6)$$

4.4 本章小结

本章针对组合设备调度问题，提出了一种基于时间自动机和模型检测的解决方案。首先，分析了传统调度方法在全局性、形式化验证及时序约束处理方面的不足，进而引入模型检测方法，利用其状态空间遍历和形式化验证的优势，确保调度方案的正确性和最优性。通过构建模块化的时间自动机模型，分别对晶圆加工流程、机械手操作、资源场所及系统状态进行精确建模，并借助 UPPAAL 工具的同步机制实现各子模型的协同。进一步采用 TCTL 语言描述系统性质，验证了可调度性、安全性及调度方案的最优性，并通过诊断路径功能生成时间最优的调度轨迹。

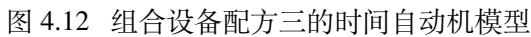


表 4.4 多组合设备配方三时间自动机模型变量及含义

变量名	含义
<i>start ?</i>	同步通道，接收信号后，系统开始运行
<i>capLA2, capLA1, capChC, capChA,</i> <i>capChB, capChF, capCh1, capCh5</i>	各加工场所剩余容量
<i>capV1, capV2, capA</i>	机械臂当前容量
<i>vmove, vpick, vdrop, amove, apick, adrop</i>	机械臂动作同步信号
<i>doneNum</i>	完成加工的晶圆数量
<i>x, xa, xv1, xv2</i>	局部时钟

第五章 实验结果与分析

在第三章中，我们采用强化学习算法中的 Q-learning 和 DQN 分别对组合设备的 Petri 网模型进行了研究，生成了相应的调度方案。第四章则提出了基于时间自动机和模型检测的组合设备调度方法。本章将分别应用上述方法对若干组合设备案例进行调度方案的计算，并对结果进行展示、对比和分析，以验证所提方法的有效性。

本实验均使用同一台计算机，其具体配置如表5.1:

表 5.1 实验环境配置

环境	配置
操作系统	Windows 11
内存	16G
处理器 (CPU)	i5-12500H
图形处理器 (GPU)	NVIDIA GeForce RTX3050 Laptop
开发环境	MATLAB R2024b, UPPAAL-5.0.0

5.1 组合设备案例介绍

组合设备案例一：使用第三章和第四章中的组合设备建模思路介绍中的单组合设备作为案例一，如图3.8所示，其加工配方为 $LP \rightarrow LL \rightarrow PM \rightarrow LL \rightarrow \text{cooling} \rightarrow LP$ 。该组合设备中的具体时间信息如表3.2所示。

组合设备案例二：如图5.14所示，为多组合设备案例二的 Petri 网模型，包含两个组合设备，三个机械手，其加工配方为 $LA2 \rightarrow ChC \rightarrow ChA \rightarrow Ch1 \rightarrow Ch5 \rightarrow ChB \rightarrow ChF \rightarrow LA1$ ，其库所变迁的含义如表5.17所示。

5.2 基于时间 Petri 网和强化学习算法的组合设备调度实验结果

本实验首先对组合设备在不同待加工晶圆数量下的调度方案进行了初步探索，以验证所提出算法的有效性。鉴于实际生产中晶圆盒通常装载 13 片晶圆，本研究进一步聚焦于待加工晶圆数量为 13 的情形，展开更深入的调研。通过多次实验和对训练超参数的细致调整，最终选取了综合性能最佳的调度方案，并对不同方案的效果进行了对比与分析。

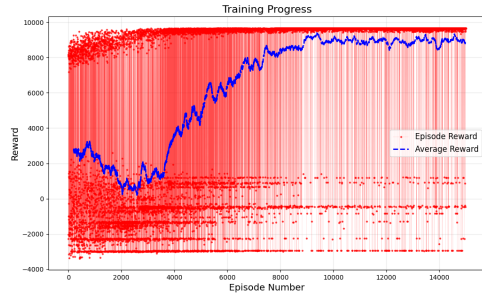
5.2.1 基础奖励函数

在案例一中，我们首先采用 Q-learning 算法对组合设备的 Petri 网系统进行训练，以生成有效的调度方案。实验平台选用 MATLAB，并编写相应程序实现 Q-learning 算法。由于本实验聚焦于调度问题，对长期回报尤为重视，因此折扣因子 γ 选取为 0.9。在 epsilon-greedy 探索策略中，我们将探索因子 ϵ 初始值设置为 0.9，并采用线性下降策略，使得训练初期偏向探索，后期逐渐转向利用。学习率则采用了常用值 0.1。

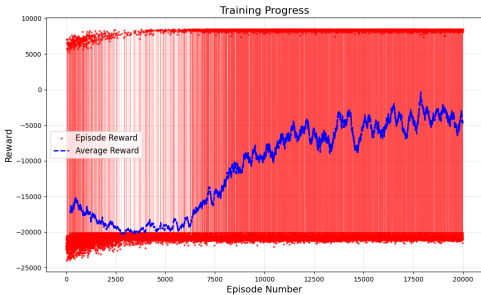
图 5.1 展示了在待加工晶圆数量分别为 5、10、13、15、20 和 25 时，采用 Q-learning 算法训练得到的曲线。其中，横坐标表示训练迭代次数，纵坐标为奖励值，红色线条为训练过程中每个 Episode 的奖励值，蓝色线条则代表窗口大小为 200 的滑动窗口平均奖励。基于相同训练参数下得到的 Q 表，我们进一步计算了不同加工任务的总调度时间，详见表 5.2。



(a) 待加工晶圆数量为 5



(b) 待加工晶圆数量为 10



(c) 待加工晶圆数量为 13



(d) 待加工晶圆数量为 15



(e) 待加工晶圆数量为 20



(f) 待加工晶圆数量为 25

图 5.1 不同待加工晶圆数量下的 Q-learning 训练曲线

表 5.2 组合设备案例一不同待加工晶圆下 Q-learning 算法调度结果

待加工晶圆数	Q-learning 算法调度时间	收敛代数
5	729.40	≈ 5000
10	1249.40	≈ 8000
13	1523.40	≈ 12000
15	1683.00	≈ 8000
20	2198.90	≈ 30000
25	2815.70	≈ 30000

从图中可以看出，所有训练曲线均已收敛，这表明 Q-learning 算法成功训练出了稳定的调度策略。由此可证明，该算法能够有效学习组合设备 Petri 网模型的调度方案，确保系统在训练过程中逐步优化并趋于稳定。在训练初期，由于 ϵ 值较大，智能体处于广泛探索阶段，采取了大量随机动作；随着训练的推进，曲线逐步上升，表明智能体通过不断与环境交互并更新策略，逐渐找到了较优的决策路径。经过若干训练周期后，各曲线趋于平稳，表明算法基本收敛，找到了稳定的控制策略。随着待加工晶圆数量的增加，任务的复杂度也随之提升，此时训练曲线的收敛速度明显减慢，需要更多的训练轮次才能达到收敛状态；同时，在晶圆数量较多的情况下，曲线波动也更为显著，反映出任务复杂性增加了智能体在探索过程中面临的不确定性。

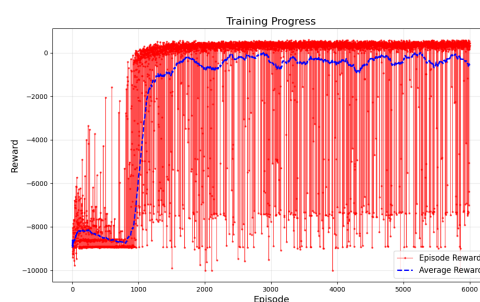
随着加工任务的增加，Q-learning 的训练曲线迭代速度显著减慢。这主要是由于状态空间呈指数级增长，导致系统复杂度提升，使得探索变得更加困难。此外，当状态空间过大时，Q 表的维度可能无法满足训练需求。为了解决高维度带来的问题，我们采用了 DQN 算法，利用神经网络来近似 Q 值函数。DQN 引入了经验回放和目标网络等机制，提高了训练的稳定性和收敛性。使用 MATLAB 强化学习工具箱进行实验，表 5.3 显示了实验中 DQN 算法的参数设置，图 5.2 展示了不同待加工晶圆数量下的训练曲线，训练结束后，通过智能体得到的调度结果如表 5.4。

表 5.3 DQN 训练参数

参数名	学习率 α	折扣因子 γ	初始 ϵ	ϵ 衰减系数	ϵ 最小值
参数值	0.0001	0.9500	0.9500	1e-5	0.0500
参数名	批量大小	目标网络更新频率	目标网络更新平滑系数	经验池大小	经验采样方式
参数值	256	100	0.01	1e5	随机采样



(a) 待加工晶圆数量为 5



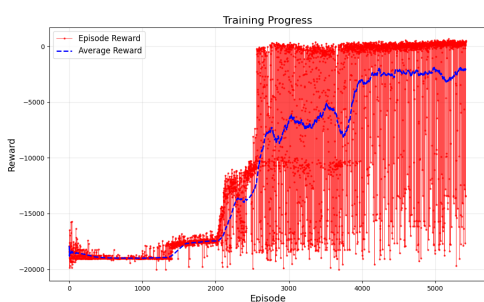
(b) 待加工晶圆数量为 10



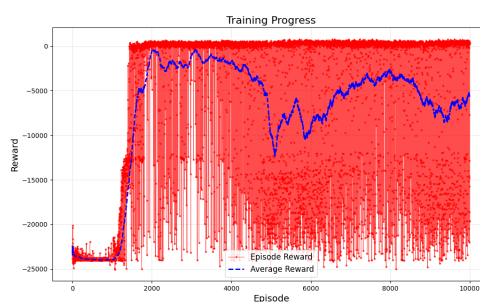
(c) 待加工晶圆数量为 13



(d) 待加工晶圆数量为 15



(e) 待加工晶圆数量为 20



(f) 待加工晶圆数量为 25

图 5.2 不同待加工晶圆数量下的 DQN 训练曲线

表 5.4 组合设备案例一不同待加工晶圆下 DQN 算法调度结果

待加工晶圆数	DQN 算法调度时间	收敛代数
5	783.50	≈ 1000
10	1255.70	≈ 1200
13	1573.50	≈ 1200
15	1749.50	≈ 1200
20	2304.00	≈ 2500
25	2750.50	≈ 4000

从训练曲线中可以看出,对于待加工晶圆数量较高的任务,DQN 算法的收敛速度明显快于 Q-learning 算法,但 DQN 训练的总体时长却更长。这主要是因为 Q-learning 采用表格更新方法,其每一步计算十分高效,适合状态空间较小的情况;然而,当状

态空间维度增大时，Q-learning 需要大量迭代才能遍历所有状态，从而导致整体收敛速度降低。相比之下，DQN 利用神经网络的泛化能力，在减少迭代次数上具有优势，但每一步计算涉及前向传播、经验回放和目标网络同步更新，导致单步耗时显著增加。

在组合设备案例一中，我们采用了 Q-learning 和 DQN 算法，并成功验证了这两种算法在该 Petri 网模型调度问题中的有效性，得到了相应的调度方案。为了进一步证明所提方法的广泛适用性，我们又选取了图 5.14 所示的组合设备案例二 Petri 网模型进行实验，相对于案例一其加工流程更加复杂，状态空间更大。在此实验中，我们同样应用 Q-learning 和 DQN 算法，对其调度方案进行了深入研究和分析。针对案例二，我们沿用了与案例一相同的 Q-learning 实验参数，经过训练后，得到的调度结果如表 5.5 和表 5.6 所示。

表 5.5 组合设备案例二不同待加工晶圆下 Q-learning 算法调度结果

待加工晶圆数	Q-learning 算法调度时间	收敛代数
5	5147.00	≈ 6000
10	11664.00	≈ 9000
13	死锁	未收敛
15	死锁	未收敛
20	死锁	未收敛
25	死锁	未收敛

表 5.6 组合设备案例二不同待加工晶圆下 DQN 算法调度结果

待加工晶圆数	调度时间	收敛代数
5	5207.00	≈ 2000
10	10827.00	≈ 3000
13	15296.00	≈ 4000
15	16066.00	≈ 6000
20	21362.90	≈ 6000
25	26340.00	≈ 8000

从表 5.5 和表 5.6 可以看出，当待加工晶圆数量超过 10 时，Q-learning 算法已无法训练出正确结果。这主要是由于待加工晶圆数量增加导致 Petri 网模型的状态空间呈指数级增长，使得 Q-learning 难以有效探索和覆盖所有状态。而采用 DQN 算法后，借助深度神经网络对状态价值函数的逼近能力，能够有效应对大规模状态空间，从而在高待加工晶圆数量下仍能获得正确的调度结果。案例一和案例二的算法收敛速度分别如图 5.3(a) 和 5.3(b)。

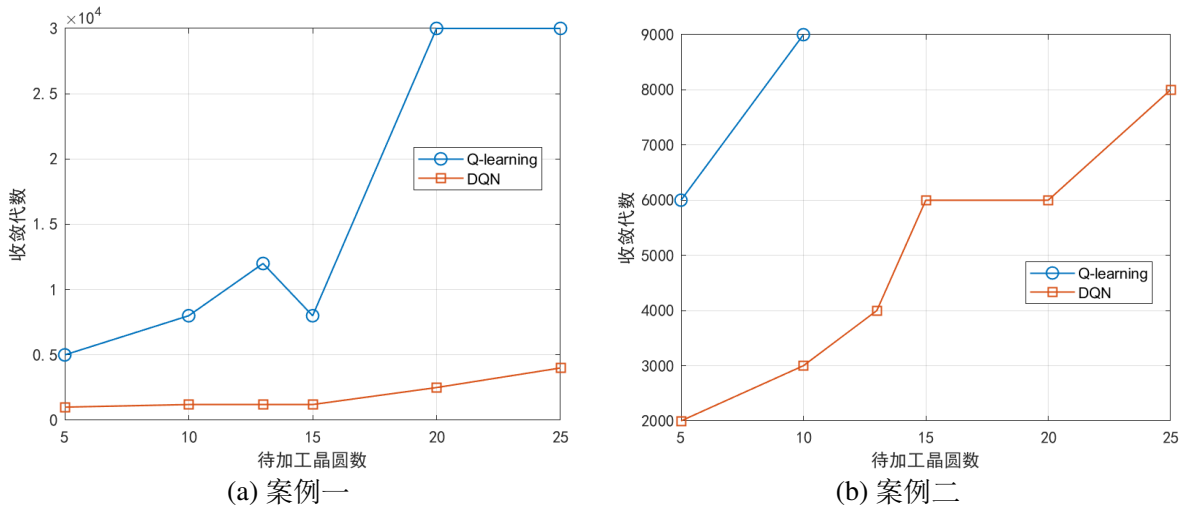


图 5.3 算法收敛速度对比

5.2.2 加权组合奖励函数

由于基础奖励函数在探索复杂状态空间时存在局限性，本节使用融合进度奖励与设备利用率奖励的加权组合奖励函数（如式3-6所示）。我们采用 Q-learning 和 DQN 算法，对两个组合设备调度案例进行研究，并将其训练过程及调度结果与基础奖励函数的对应表现进行展示与对比。

在案例一和案例二的组合设备调度问题中，当待加工晶圆数量设定为 13 时，我们分别采用基础奖励函数和多目标奖励函数，使用 Q-learning 和 DQN 算法进行了训练。其训练曲线分别展示在图5.4和图5.5中。

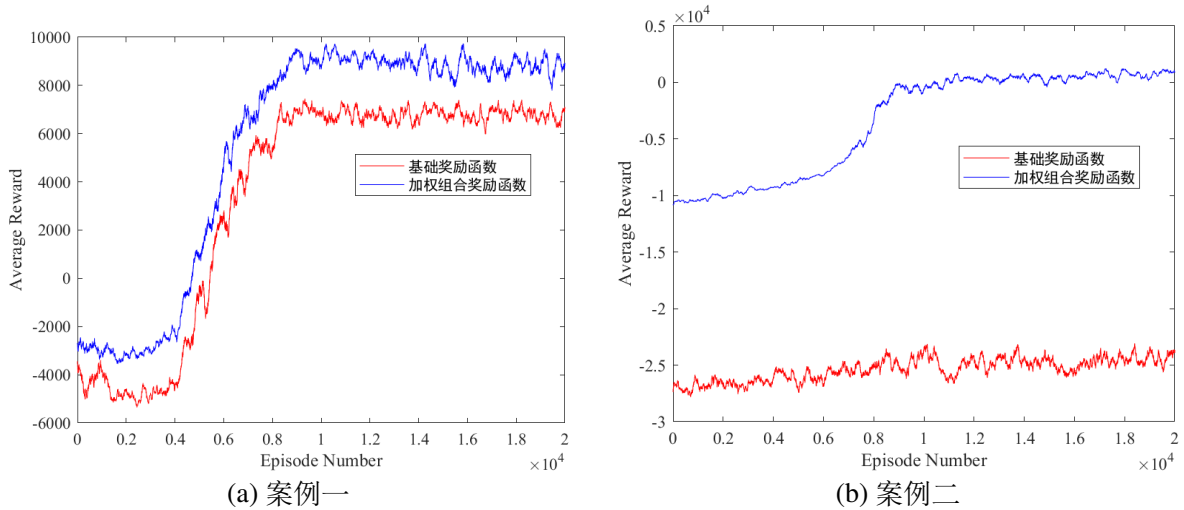


图 5.4 不同奖励函数下 Q-learning 训练曲线的对比

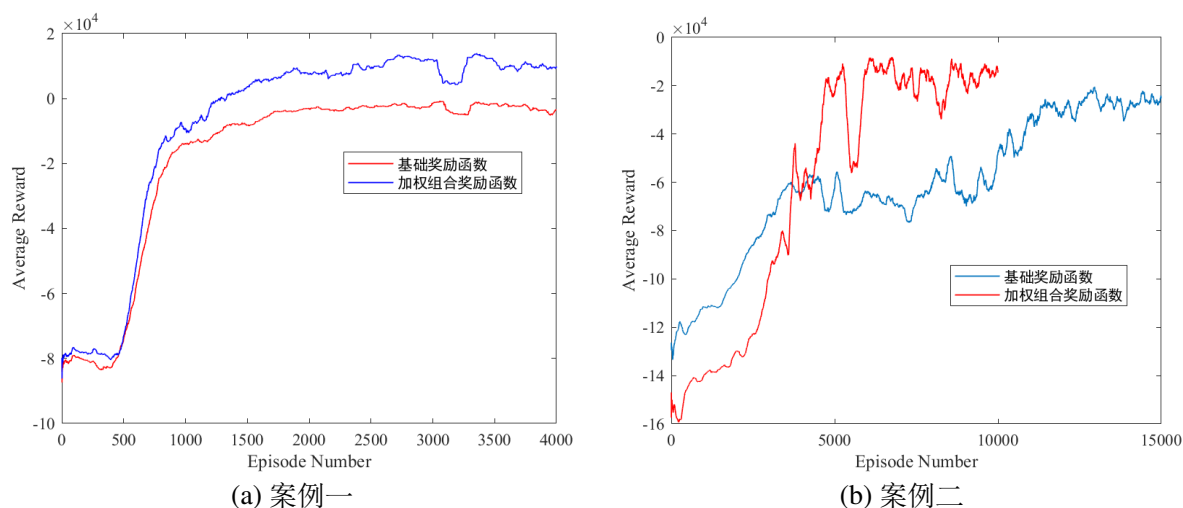


图 5.5 不同奖励函数下 DQN 训练曲线的对比

从图 5.4 和图 5.5 的训练曲线可以看出，在案例一中，基础奖励函数与加权组合奖励函数的训练曲线相差不大，无论是收敛代数还是曲线波动幅度，都未表现出显著的提升。而在案例二中，使用加权组合奖励函数有效解决了 Q-learning 算法在基础奖励函数下无法正确训练的问题，同时在 DQN 算法的训练过程中，收敛速度也有所提升。这种差异主要源于案例一和案例二在工艺流程上的复杂性不同。案例一的加工流程较为简单，中间状态较少，基础奖励函数已能提供足够的引导，确保算法收敛。然而，案例二涉及多组合设备，流程复杂，状态空间庞大，仅依赖基础奖励函数难以充分评估中间状态，导致训练难以收敛。而加权组合奖励函数能够更精确地对中间状态进行评估和引导，从而促进算法更快、更稳定地收敛，最终获得更优的调度方案。

5.2.3 比较和分析

表 5.7 和表 5.8 分别展示了在待加工晶圆数量为 13 的情况下，案例一和案例二中，不同调度算法（包括 Q-learning、DQN 及其他调度方法）在 10 次实验中的调度结果对比。与此同时，表 5.9 统计了 Q-learning 和 DQN 在训练过程中的训练时间及实际应用中的响应时间，以进一步评估两种强化学习算法的计算成本和实时性。

表 5.7 案例一调度结果对比

调度算法	平均调度时间	最优调度时间	平均收敛代数
Q-learning + 基础奖励函数	1550.11	1529.70	9000
Q-learning + 加权组合奖励函数	1556.45	1522.90	9000
DQN + 基础奖励函数	1625.27	1561.80	1500
DQN + 加权组合奖励函数	1545.93	1512.80	1500
A*	\	1521.80	\

表 5.8 案例二调度结果对比

调度算法	平均调度时间	最优调度时间	平均收敛代数
Q-learning + 基础奖励函数	死锁	死锁	未收敛
Q-learning + 加权组合奖励函数	14353.29	12889.00	10000
DQN + 基础奖励函数	14422.00	13513.00	12000
DQN + 加权组合奖励函数	13509.33	12968.00	10000
A*	\	13304.00	\

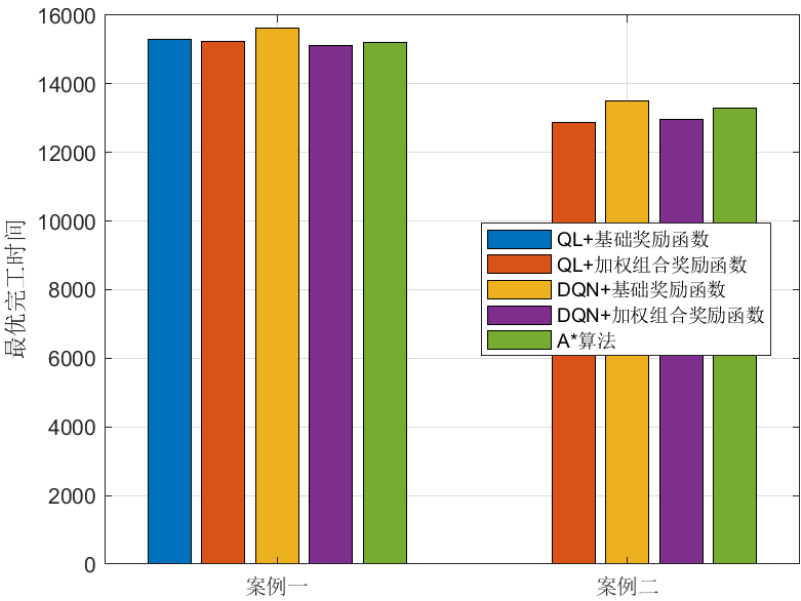


图 5.6 不同算法调度结果对比

表 5.9 案例二下训练时间和响应时间对比

时间类型	Q-learning	DQN
训练时间	1h 44min	8h 9min
响应时间	0.17s	0.78s

由表 5.7 可见，案例一的组合设备加工流程较为简单，Petri 网的状态空间较小。因此，各种调度算法均能正确生成完工调度方案。在此情况下，Q-learning 由于能够充分探索环境，获得了良好的调度效果，而 DQN 虽未显著提升调度质量，但因引入神经网络减少了重复探索，使得收敛速度明显加快。从奖励函数的影响来看，加权组合奖励函数对 Q-learning 的平均调度效果提升有限，但有助于智能体找到更优调度方案，从而缩短完工时间。对于 DQN，该奖励函数显著改善了平均调度时间和最优调度结果，主要原因在于 DQN 对奖励信号更为敏感，特别是在状态空间较小时，加权组合奖励能提供更丰富的中间反馈，促进策略学习。

表 5.8 显示, 案例二涉及多组合设备, 加工流程更复杂, Petri 网状态空间大幅扩展。在此情境下, Q-learning 仅依赖基础单步奖励函数难以有效探索, 训练过程难以收敛, 最终易陷入死锁。但在加权组合奖励函数的引导下, Q-learning 能够正确训练并找到可行调度方案。相比之下, DQN 在两种奖励函数下均能生成有效调度结果, 但采用加权组合奖励时, 不仅平均调度时间和最优调度时间均有所改善, 且收敛速度更快, 表明加权组合奖励机制对优化调度质量和提升算法收敛性均有积极作用。此外, 与 A* 算法相比, 在两个案例中, DQN 结合加权组合奖励函数能够进一步优化调度方案, 实现更短的完工时间。然而, 强化学习方法在训练过程中计算成本较高, 其训练时间远超 A* 算法的运行时间。

综合实验结果表明, Q-learning 和 DQN 均适用于组合设备 Petri 网调度问题。对于状态空间较小的场景, Q-learning 训练效率较高, 且能获得较好的调度效果。然而, 在复杂调度任务中, DQN 依托其较强的泛化能力、更快的收敛速度以及对大规模状态空间的适应性, 表现出更高的可靠性, 尤其在资源丰富、任务规模较大的情况下更具优势。相比之下, Q-learning 在大规模任务中的探索效率受限, 难以找到有效调度方案。此外, 在基础单步惩罚奖励函数的基础上引入进度奖励等因素, 并合理调整权重, 可有效优化探索过程, 提高调度质量和收敛性能, 同时相较于常规调度算法展现出显著优势。因此, 在高维复杂调度问题中, DQN 结合加权组合奖励函数兼顾了收敛速度与调度质量, 是更优的选择。

5.3 基于时间自动机和模型检测方法的组合设备调度实验结果

5.3.1 组合设备案例一

针对组合设备案例一的系统, 我们依据其 Petri 网模型和实际工作逻辑, 参照第四章中组合设备的时间自动机建模方法, 构建了相应的自动机网络, 并在 UPPAAL 平台上进行建模与验证, 如图4.6所示。其待验证性质如表5.10

表 5.10 组合设备待验证性质及其 TCTL

待验证性质	TCTL
是否存在死锁	$A [] not deadlock$
是否会进入错误状态	$A [] not SystemState.Error$
可调度性	$E \Diamond SystemState.AllTasksCompleted$
调度方案最优性	$E \Diamond (SystemState.AllTasksCompleted \ and \ globalClock < fastestTime)$

对于系统是否存在死锁状态和系统会进入错误状态这两种关于安全性的性质, 以待加工晶圆数量为 1 为例, 使用 UPPAAL 验证器结果如图5.7。

```
A[] not deadlock
A[]not SystemState.Error
```



图 5.7 安全性性质验证结果

```
A[] not deadlock
验证费时/kernel费时/总费时: 0.234s / 0.015s / 0.256s.
常驻内存/虚拟内存的使用峰值: 22,668KB / 71,108KB.
不满足该性质.
```

图 5.8 是否存在死锁性质验证详细信息

```
A[]not SystemState.Error
验证费时/kernel费时/总费时: 2.719s / 0.047s / 2.914s.
常驻内存/虚拟内存的使用峰值: 74,480KB / 175,020KB.
满足该性质.
```

图 5.9 是否进入错误状态性质验证详细信息

从验证结果来看,对于性质4-5,验证结果显示为红色,表明该性质不满足,即系统存在死锁状态。在第四章提出的方法中,我们主要关注组合设备系统的调度方案,因此需要借助模型检测方法寻找一条完工时间最短的可行路径,以确保调度方案能够正确执行且不会发生死锁。由于本方法未涉及整个系统的死锁预防控制策略设计,因此系统整体可能存在死锁状态,但所生成的调度方案的执行序列不会导致死锁。

对于性质4-6,验证结果显示为绿色,表明该性质满足。这是因为在建模过程中,我们对各资源场所的容量进行了严格限制,确保资源不会超载或溢出,从而避免了系统进入错误状态,保证了该安全性性质的满足。此外,我们针对不同的待加工晶圆数量进行了多次验证,结果均与待加工晶圆数量为1时一致,进一步验证了系统调度方案的稳定性和正确性。

在研究组合设备的调度方案时,首先需要验证系统的可调度性,即性质4-2。为此,我们利用UPPAAL验证器的诊断路径生成功能,并将诊断路径选项设置为“最快路径”,以确保生成的调度方案在满足可行性的同时,尽可能缩短完工时间。随后,我们针对不同待加工晶圆数量的系统进行验证,其验证结果如表5.11所示。

表 5.11 不同待加工晶圆数下的可调度性验证结果

待加工晶圆数	验证结果	验证时间	诊断路径时间
1	满足	0.003s	4509
2	满足	0.016s	4629
3	满足	0.639s	4865
4	满足	51.46s	7305

```

E◇SystemState.AllTasksCompleted
验证费时/kernel费时/总费时: 49.813s / 0.547s / 51.46s.
常驻内存/虚拟内存的使用峰值: 1,305,620KB / 2,645,712KB.
满足该性质.

```

图 5.10 可调度性验证详细信息

由表5.11可知,对于案例一中的组合设备系统,当待加工晶圆数量小于5时,UPPAAL验证器均能成功验证出所需性质,并生成一条用时最短的诊断路径。然而,随着待加工晶圆数量的增加,系统可调度性的验证时间及内存占用量也呈现上升趋势。这主要归因于UPPAAL验证器采用穷尽式模型检测策略,即需遍历系统完整的状态空间;而待加工晶圆数量的增加将导致系统状态空间呈指数级扩张,从而显著增加验证时长和内存需求。当待加工晶圆数量达到或超过5时,由于状态空间过于庞大,现有计算资源已无法满足验证要求,此时需要借助系统模型简化等手段来缩减状态空间规模。针对这一问题,后续研究可进一步探讨高效状态空间约简技术或分层验证方法。

在理论上,UPPAAL在“最快路径”模式下生成的诊断路径是在所有满足验证性质的路径中用时最短的,即具有全局最优性。这是因为UPPAAL采用了类似均匀成本搜索或Dijkstra算法的策略,在遍历整个状态空间时,对每个状态节点都记录了从初始状态出发的累计时间,并始终优先扩展累计延时较小的节点。因此,只要系统中存在满足验证性质的路径,算法找到的第一条路径就必然具有最短的累计延时,从而保证了其最优性。

为了进一步验证UPPAAL在这一特性上的有效性,我们对诊断路径的最优性进行了验证,如性质4-4所示。待验证性质及相应的验证结果均如表5.12所示。

表 5.12 不同待加工晶圆数下的诊断路径最优性验证结果

待加工晶圆数	TCTL	验证结果
1	$E◇(SystemState.AllTasksCompleted \text{ and } globalClock < 4509)$	满足
2	$E◇(SystemState.AllTasksCompleted \text{ and } globalClock < 4629)$	满足
3	$E◇(SystemState.AllTasksCompleted \text{ and } globalClock < 4865)$	满足
4	$E◇(SystemState.AllTasksCompleted \text{ and } globalClock < 7305)$	满足

```

E◇ SystemState.AllTasksCompleted and global_clock < 7305
验证费时/kernel费时/总费时: 52.266s / 0.39s / 53.913s.
常驻内存/虚拟内存的使用峰值: 1,038,248KB / 2,104,592KB.
不满足该性质.

```

图 5.11 调度方案最优性验证详细信息

5.3.2 组合设备案例二

根据图5.14的案例二组合设备的 Petri 网模型和其实际加工逻辑，依照第四章中的组合设备时间自动机建模思路，建立系统各模块的时间自动机模型，并搭建在 UP-PAAL 中如图4.12所示。其待验证性质与案例一相同

表 5.13 案例二不同待加工晶圆数下的可调度性验证结果

待加工晶圆数	验证结果	验证时间	诊断路径时间
1	满足	0.008s	6893
2	满足	0.064s	7434
3	满足	5.669s	8909
4	满足	621.963s	9450

表 5.14 案例二待验证性质及结果

待验证性质	TCTL	验证结果	验证时长
是否存在死锁	$A[] \text{ not deadlock}$	不满足	0.023s
是否会进入错误状态	$A[] \text{ not SystemState.Error}$	满足	65.27s
可调度性	$E\Diamond \text{SystemState.AllCompleted}$	满足	547.25s
调度方案最优性	$E\Diamond (\text{SystemState.AllCompleted and globalClock} < 9450)$	满足	533.67s

A[] not deadlock
验证费时/kernel费时/总费时: 0s / 0.015s / 0.023s.
常驻内存/虚拟内存的使用峰值: 19,100KB / 65,008KB.
不满足该性质.

(a) 是否存在死锁

A[]not SystemState.Error
验证费时/kernel费时/总费时: 48.375s / 14.515s / 65.27s.
常驻内存/虚拟内存的使用峰值: 2,956,820KB / 10,864,480KB.
满足该性质.

(b) 是否进入错误状态

E◇SystemState.Complete
验证费时/kernel费时/总费时: 519.734s / 3.86s / 533.67s.
常驻内存/虚拟内存的使用峰值: 10,584,240KB / 21,439,812KB.
满足该性质.

(c) 可调度性

E◇ SystemState.Complete and global_clock < 9450
验证费时/kernel费时/总费时: 532.359s / 4.421s / 547.254s.
常驻内存/虚拟内存的使用峰值: 10,240,404KB / 20,570,892KB.
不满足该性质.

(d) 调度方案最优性

图 5.12 案例二待验证性质验证结果详细信息

5.3.3 比较与分析

为对比模型检测方法在调度问题的优点，通过不同算法对同一模型进行测试，实验结果如表5.15和表5.16所示，各算法的调度结果对比如图5.13。

表 5.15 案例一调度结果对比

加工晶圆数量	模型检测	A*	蚁群算法
1	4509	4509	4509
2	4629	4629	4629
3	4865	5030	5030
4	7305	7701	7391

表 5.16 案例二调度结果对比

加工晶圆数量	模型检测	A*	蚁群算法
1	6893	6893	6893
2	7557	7557	7557
3	8909	9002	9511
4	9450	9904	10171
5	\	10453	12819

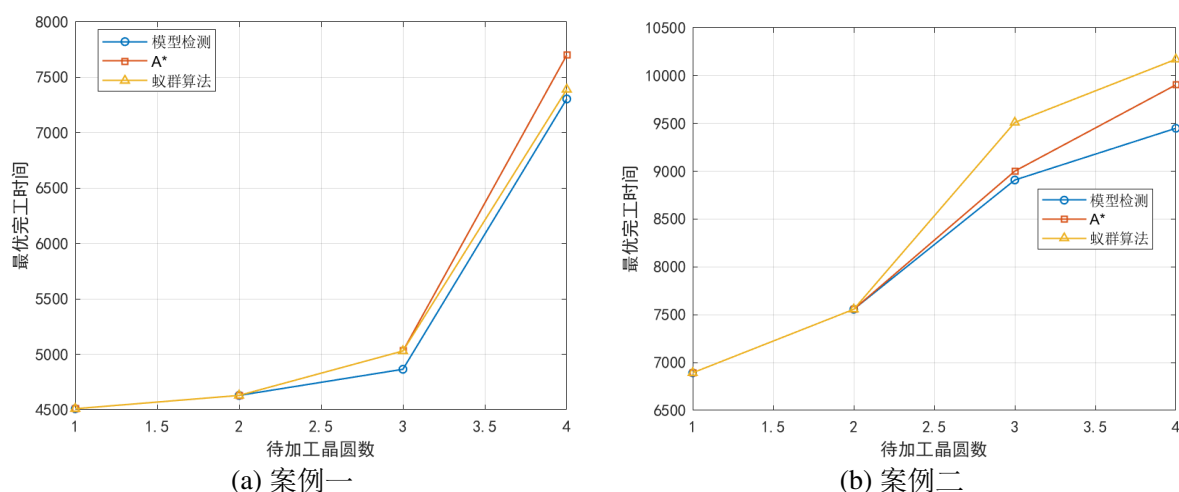


图 5.13 算法收敛速度对比

由于模型检测方法在验证过程中遍历整个状态空间，因此能够保证调度方案的全局最优性。在相同待加工晶圆数量下，案例一和案例二的诊断路径均等于或优于两种启发式搜索算法的调度方案。然而，由于模型检测需要穷尽整个状态空间，当晶圆数量增加时，状态空间急剧扩展，导致在有限的计算资源下可能无法获得正确的验证结果；而两种启发式算法仍能输出有效方案。尽管存在上述局限，模型检测方法在工程实践中具有重要意义：它不仅为调度问题提供了最优性保障，还能作为启发式算法性能的基准，为实际生产中的调度方案设计和优化提供有力参考。未来工作可以进一步探索模型简化和状态空间缩减技术，以应对大规模状态空间问题，从而在保持验证最优性的同时，提高计算效率。

5.4 本章小结

本章在统一的实验环境下，分别采用 Q-learning 和 DQN 算法，对多个组合设备的 Petri 网模型调度问题展开了深入研究。通过大量训练，我们获得了不同加工任务下的调度方案，并从训练曲线、调度结果以及与其他调度算法的对比中，验证了这两种算法在组合设备调度问题上的有效性及各自的优势与不足。此外，我们在基础单步惩罚奖励函数中引入多因素加权组合，对中间状态进行引导，从而在一定程度上提升了调度效果和收敛性能。为进一步寻求调度问题的最优解，本章还引入了模型检测方法，构建了组合设备的时间自动机网络模型，并利用 UPPAAL 模型检测工具对系统调度方案进行了研究与分析。实验结果表明，该方法在优化调度方案方面同样具备良好的有效性，同时揭示了其在实际应用中的优势与局限性。

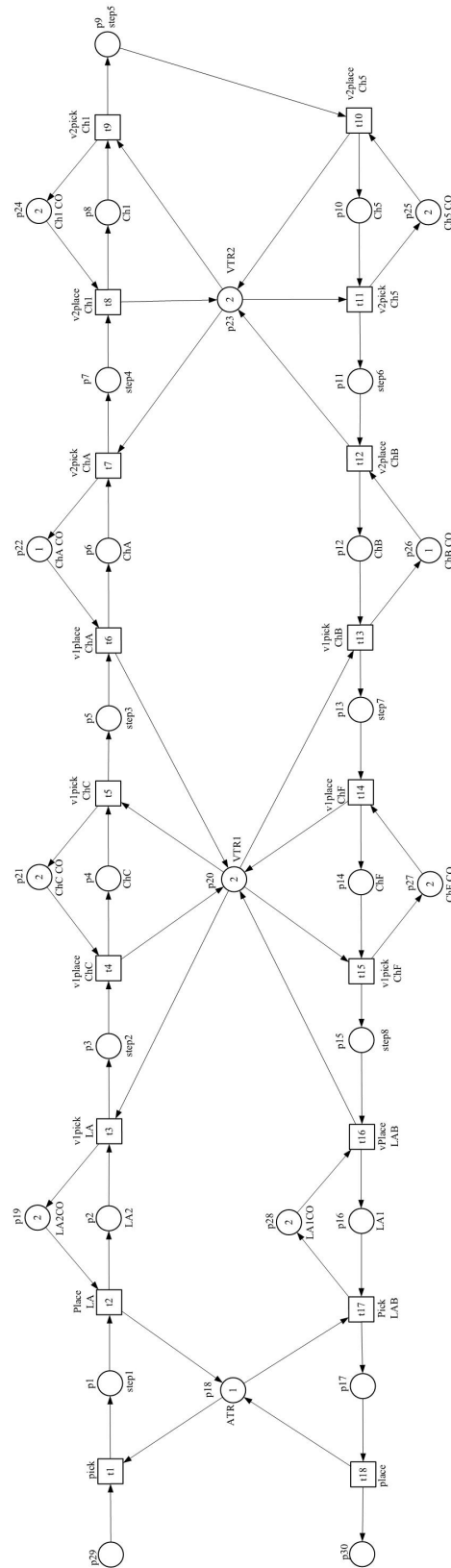


图 5.14 组合设备案例二的 Petri 网模型

表 5.17 图5.14组合设备 Petri 网库所变迁含义

库所变迁符号	含义
$t1, t17$	ATR 取操作
$t2, t18$	ATR 放操作
$t3, t5, t7, t9, t11, t13, t15$	VTR1, VTR2 取操作
$t4, t6, t8, t10, t12, t14, t16$	VTR1, VTR2 放操作
$p1, p17$	ATR 大气端移动
$p3, p5, p7, p9, p11, p13, p15$	VTR1, VTR2 真空端移动
$p2, p4, p6, p8, p10, p12, p14, p16$	加工场所进行加工
$p18, p20, p23$	ATR, VTR1, VTR2 容量
$p19, p21, p22, p24, p25, p26, p27, p28$	加工场所容量
$p29, p30$	进料口、出料口

案例一使用 Q-learning 算法的最优调度方案的变迁序列为: $t1 \rightarrow t2 \rightarrow t1 \rightarrow t2 \rightarrow t1 \rightarrow t3 \rightarrow t4 \rightarrow t2 \rightarrow t1 \rightarrow t3 \rightarrow t4 \rightarrow t2 \rightarrow t1 \rightarrow t3 \rightarrow t2 \rightarrow t1 \rightarrow t5 \rightarrow t4 \rightarrow t6 \rightarrow t5 \rightarrow t3 \rightarrow t4 \rightarrow t3 \rightarrow t6 \rightarrow t2 \rightarrow t1 \rightarrow t2 \rightarrow t5 \rightarrow t7 \rightarrow t4 \rightarrow t8 \rightarrow t3 \rightarrow t7 \rightarrow t8 \rightarrow t6 \rightarrow t7 \rightarrow t8 \rightarrow t9 \rightarrow t10 \rightarrow t1 \rightarrow t2 \rightarrow t9 \rightarrow t10 \rightarrow t9 \rightarrow t5 \rightarrow t6 \rightarrow t3 \rightarrow t4 \rightarrow t10 \rightarrow t7 \rightarrow t8 \rightarrow t1 \rightarrow t2 \rightarrow t5 \rightarrow t1 \rightarrow t6 \rightarrow t3 \rightarrow t4 \rightarrow t2 \rightarrow t7 \rightarrow t8 \rightarrow t1 \rightarrow t5 \rightarrow t6 \rightarrow t3 \rightarrow t2 \rightarrow t9 \rightarrow t10 \rightarrow t7 \rightarrow t8 \rightarrow t9 \rightarrow t10 \rightarrow t4 \rightarrow t5 \rightarrow t6 \rightarrow t4 \rightarrow t3 \rightarrow t7 \rightarrow t8 \rightarrow t1 \rightarrow t2 \rightarrow t9 \rightarrow t10 \rightarrow t5 \rightarrow t6 \rightarrow t4 \rightarrow t3 \rightarrow t7 \rightarrow t8 \rightarrow t9 \rightarrow t5 \rightarrow t6 \rightarrow t10 \rightarrow t3 \rightarrow t4 \rightarrow t7 \rightarrow t8 \rightarrow t5 \rightarrow t4 \rightarrow t6 \rightarrow t7 \rightarrow t8 \rightarrow t5 \rightarrow t6 \rightarrow t9 \rightarrow t10 \rightarrow t5 \rightarrow t9 \rightarrow t10 \rightarrow t7 \rightarrow t6 \rightarrow t8 \rightarrow t9 \rightarrow t10 \rightarrow t5 \rightarrow t6 \rightarrow t7 \rightarrow t8 \rightarrow t7 \rightarrow t8 \rightarrow t9 \rightarrow t10 \rightarrow t9 \rightarrow t10 \rightarrow t9 \rightarrow t10$ 。

案例一使用 DQN 算法的最优调度方案的变迁序列为: $t1 \rightarrow t2 \rightarrow t1 \rightarrow t3 \rightarrow t4 \rightarrow t2 \rightarrow t3 \rightarrow t4 \rightarrow t1 \rightarrow t2 \rightarrow t1 \rightarrow t3 \rightarrow t4 \rightarrow t2 \rightarrow t1 \rightarrow t3 \rightarrow t2 \rightarrow t5 \rightarrow t1 \rightarrow t4 \rightarrow t6 \rightarrow t2 \rightarrow t1 \rightarrow t3 \rightarrow t5 \rightarrow t4 \rightarrow t2 \rightarrow t6 \rightarrow t7 \rightarrow t8 \rightarrow t1 \rightarrow t3 \rightarrow t5 \rightarrow t6 \rightarrow t2 \rightarrow t4 \rightarrow t1 \rightarrow t3 \rightarrow t2 \rightarrow t7 \rightarrow t5 \rightarrow t8 \rightarrow t4 \rightarrow t3 \rightarrow t6 \rightarrow t5 \rightarrow t9 \rightarrow t10 \rightarrow t1 \rightarrow t2 \rightarrow t4 \rightarrow t7 \rightarrow t8 \rightarrow t9 \rightarrow t10 \rightarrow t6 \rightarrow t1 \rightarrow t3 \rightarrow t2 \rightarrow t7 \rightarrow t5 \rightarrow t4 \rightarrow t8 \rightarrow t9 \rightarrow t10 \rightarrow t1 \rightarrow t6 \rightarrow t3 \rightarrow t2 \rightarrow t7 \rightarrow t5 \rightarrow t4 \rightarrow t8 \rightarrow t9 \rightarrow t10 \rightarrow t1 \rightarrow t6 \rightarrow t3 \rightarrow t5 \rightarrow t2 \rightarrow t7 \rightarrow t4 \rightarrow t8 \rightarrow t9 \rightarrow t10 \rightarrow t7 \rightarrow t8 \rightarrow t9 \rightarrow t6 \rightarrow t3 \rightarrow t5 \rightarrow t10 \rightarrow t4 \rightarrow t7 \rightarrow t8 \rightarrow t9 \rightarrow t10 \rightarrow t6 \rightarrow t3 \rightarrow t7 \rightarrow t5 \rightarrow t4 \rightarrow t6 \rightarrow t8 \rightarrow t9 \rightarrow t10 \rightarrow t7 \rightarrow t8 \rightarrow t5 \rightarrow t6 \rightarrow t9 \rightarrow t10 \rightarrow t7 \rightarrow t8 \rightarrow t9 \rightarrow t10 \rightarrow t5 \rightarrow t6 \rightarrow t9 \rightarrow t10 \rightarrow t7 \rightarrow t8 \rightarrow t5 \rightarrow t6 \rightarrow t9 \rightarrow t10 \rightarrow t7 \rightarrow t8 \rightarrow t9 \rightarrow t10$ 。

案例二使用 Q-learning 算法的最优调度方案的变迁序列为: $t1 \rightarrow t2 \rightarrow t3 \rightarrow t4 \rightarrow t5 \rightarrow t1 \rightarrow t2 \rightarrow t6 \rightarrow t3 \rightarrow t4 \rightarrow t1 \rightarrow t2 \rightarrow t7 \rightarrow t8 \rightarrow t3 \rightarrow t4 \rightarrow t1 \rightarrow t5 \rightarrow t6 \rightarrow t2 \rightarrow t7 \rightarrow t8 \rightarrow t9 \rightarrow t10 \rightarrow t9 \rightarrow t3 \rightarrow t4 \rightarrow t10 \rightarrow t1 \rightarrow t5 \rightarrow t6 \rightarrow t2 \rightarrow$

[illegible]

$t_{15} \rightarrow t_{16} \rightarrow t_{17} \rightarrow t_{18}$ 。

第六章 总结与展望

6.1 总结

在半导体制造领域，组合设备因其集成多道高精度工序，极大地提高了生产效率和工艺集成度，但同时也带来了复杂的调度难题。各工序之间存在并行、同步和资源竞争等多重关系，稍有不慎便可能导致生产瓶颈、资源闲置甚至系统死锁。因此，深入研究组合设备调度问题不仅有助于揭示各工序之间的内在联系和约束机制，更为实现生产流程的最优化、降低制造成本、提升产品良率提供了理论依据和技术支撑。这对推动半导体制造工艺的智能化、柔性化改造具有重要的现实意义和长远的战略价值。

针对半导体制造业中晶圆加工流程的组合设备调度问题，本文选用离散事件系统中两种常用的形式化模型——Petri 网和自动机，对组合设备进行建模，并分别结合强化学习算法和模型检测方法，对不同加工任务下的调度方案展开研究。本文的主要研究工作包括以下几点：

1. 介绍了一种兼具库所时延 Petri 网和变迁时延 Petri 网特性的库所变迁时延 Petri 网 (PTTPN)，并使用 MATLAB 代码实现了其运行逻辑，以便后续实验使用。结合实际组合设备系统，详细阐述了 PTTPN 的建模思路，并对模型中各变迁和库所的含义进行了充分解释。
2. 针对现有的组合设备 PTTPN 模型，本文基于强化学习中的 Q-learning 算法和 DQN 算法对系统调度问题进行了求解。主要内容包括状态空间设计、动作集设计以及奖励函数的构建，通过训练智能体，最终获得了针对不同加工方案的调度策略。实验工作在 MATLAB 平台上实现，主要通过 MATLAB 代码编写和强化学习工具箱实现。
3. 提出了一种基于时间自动机的组合设备建模思路。该方法采用多个子自动机对组合设备进行建模，包括加工流程自动机、各模块自动机以及系统状态自动机等。子自动机之间通过全局变量和同步通道进行通信，实现并行协作，从而构成了组合设备完整的时间自动机模型。
4. 在构建组合设备时间自动机模型的基础上，本文进一步结合实际系统进行建模，并利用模型检测方法研究其调度方案。通过在 UPPAAL 中搭建系统的时间自动机模型，并采用 TCTL 描述系统性质，利用 UPPAAL 验证器的诊断路径生成功能，对组合设备的调度方案进行求解和验证。

6.2 展望

组合设备作为半导体制造业中的关键设备之一，其调度问题始终是研究的热点与难点。虽然本文在组合设备调度问题上取得了一定进展，但仍存在不足和改进空间。未来研究可从以下几个方面进一步展开：

1. 本文中的两种建模方式主要针对静态或准静态的生产环境，在一定程度上对生产环境进行了理想化处理，未来可进一步引入更多实际生产中的不确定性因素，如设备故障、工艺波动和外部干扰等，以提升模型的鲁棒性和适用性。同时，还需探索如何在保持模型精度的同时降低不同规模和复杂度生产线的计算复杂度，这仍是一项重要挑战。
2. 组合设备调度问题具有多样性和复杂性，本文未涵盖更为复杂的加工场景，如晶圆重入加工调度、多品种晶圆加工调度等。未来可以考虑在时间 Petri 网模型中引入更多特性（例如托肯有色机制），以模拟不同品种晶圆的加工特性，从而应对其他场景下的调度问题。
3. 在强化学习方面，未来可尝试引入更为先进的算法（如 Actor-Critic、深度确定性策略梯度等），并结合经验回放、优先采样等技术进一步提升算法的收敛速度和调度性能。同时，考虑到强化学习在不同设备场景下往往需要从头开始重新训练这一弊端，可通过迁移学习等方法实现模型在不同设备间的知识迁移，从而提高训练效率和模型的泛化能力。此外，还可探索混合求解策略，将强化学习与传统优化方法（如元启发式算法、数学规划）有机结合，以在全局搜索与局部精细调优之间取得更好的平衡。
4. 在模型检测方面，尽管穷尽式模型检测能够确保调度方案达到最短时间，但其代价在于遍历整个状态空间，消耗大量计算资源。对于复杂系统的时间自动机模型，可能会出现“状态爆炸”问题，导致验证时间过长或无法获得验证结果。因此，如何对时间自动机模型进行适当简化以减小状态空间规模，是一项亟待解决的重要挑战。未来的研究可以从多个方向探索对时间自动机模型的适当简化，以应对“状态爆炸”带来的挑战。一方面，可以采用抽象化技术，对模型进行适当的精简，通过构造抽象模型捕捉系统的关键行为，同时忽略不影响整体调度性能的次要细节，从而大幅减少状态空间规模；另一方面，利用分布式或并行计算方法，将模型检测任务分解为若干子任务，借助多核处理器或云平台进行协同验证，也是一种切实可行的解决方案。
5. 本文采用 UPPAAL 作为模型检测工具研究组合设备调度问题。值得注意的是，作为热门模型检测器，UPPAAL 正在不断更新新功能，其扩展模块 UPPAAL

Stratego 将形式化方法与强化学习技术相结合，在时序博弈 (Timed Games) 框架下，通过学习获得既满足时序安全约束又具备较优性能的控制策略，从而实现复杂实时系统的高效调度与控制。未来可尝试利用 UPPAAL Stratego，将模型检测与强化学习结合，以更高效地获得系统的最优调度方案。

6. 本文基于组合设备的形式化模型展开调度方案的研究，调度方案的实施同样依托于两种模型。因此，未来研究应重点借助模拟仿真技术对调度方案的执行效果进行全面评估。构建高保真度的生产环境数字模型，将实际生产线的设备、工艺流程、物料流转以及各类不确定因素纳入仿真体系，是实现真实场景再现的关键。借助基于离散事件或混合动态模型的仿真平台，可直观观察调度方案在不同工况下的执行情况，从而及时识别潜在瓶颈和调度缺陷。通过对调度结果的模拟仿真，不仅验证了所提出算法和模型的有效性，同时也为生产系统的持续优化与智能制造提供了关键的数据支持和理论依据，这一环节在未来调度研究中不可或缺。

参考文献

- [1] 王华, 王令豪. 产业链安全视角下台湾地区半导体产业的全球布局趋势[J/OL]. 亚太经济, 2024(04): 129-139. DOI: 10.16407/j.cnki.1000-6052.2024.04.008.
- [2] BOWN C P. How the United States marched the semiconductor industry into its trade war with China [J]. East Asian Economic Review, 2020, 24(4): 349-388.
- [3] 张晴晴. 晶圆制造设备的行业状况与发展策略[J/OL]. 集成电路应用, 2022, 39(04): 28-29. DOI: 10.19339/j.issn.1674-2583.2022.04.012.
- [4] RAZAVIEH A, ZEITZOFF P, NOWAK E J. Challenges and limitations of CMOS scaling for FinFET and beyond architectures[J]. IEEE Transactions on Nanotechnology, 2019, 18: 999-1004.
- [5] BADER M E, HALL R P, STRASSER G. Integrated processing equipment[J]. Solid State Technology, 1990, 33(5): 149-155.
- [6] PAN C, ZHOU M, QIAO Y, et al. Scheduling cluster tools in semiconductor manufacturing: Recent advances and challenges[J]. IEEE transactions on automation science and engineering, 2017, 15(2): 586-601.
- [7] CASSANDRAS C G, LAFORTUNE S. Introduction to discrete event systems[M]. Springer, 2008.
- [8] SILVA M, VALETTE R. Petri nets and flexible manufacturing[C]//European Workshop on Applications and Theory in Petri Nets. 1988: 374-417.
- [9] BERTHOMIEU B, VERNADAT F. Time Petri Nets Analysis with TINA.[C]//QEST: vol. 6. 2006: 123-124.
- [10] MERLIN SAJINI M, SUJA S, MERLIN GILBERT RAJ S, et al. Finite state machine-based load scheduling algorithm for smart home energy management[J]. IETE Journal of Research, 2023, 69(10): 7460-7475.
- [11] BRENNA L, GEHRKE J, HONG M, et al. Distributed event stream processing with non-deterministic finite automata[C]//Proceedings of the Third ACM International Conference on Distributed Event-Based Systems. 2009: 1-12.
- [12] ALUR R, LA TORRE S, PAPPAS G J. Optimal paths in weighted timed automata[J]. Theoretical Computer Science, 2004, 318(3): 297-322.
- [13] SHYALIKA C, SILVA T, KARUNANANDA A. Reinforcement learning in dynamic task scheduling: A review[J]. SN Computer Science, 2020, 1(6): 306.
- [14] WILLIAMS H P. Model building in mathematical programming[M]. John Wiley & Sons, 2013.
- [15] KIM H J, LEE J H, BAIK S, et al. Scheduling in-line multiple cluster tools[J]. IEEE Transactions on Semiconductor Manufacturing, 2015, 28(2): 171-179.

- [16] BAO T, WANG H. Cyclic scheduling of multi-cluster tools based on mixed integer programming[J]. IEEE Transactions on Semiconductor Manufacturing, 2017, 30(4): 515-525.
- [17] QIAO Y, LU Y, LI J, et al. An efficient binary integer programming model for residency time-constrained cluster tools with chamber cleaning requirements[J]. IEEE Transactions on Automation Science and Engineering, 2021, 19(3): 1757-1771.
- [18] PERKINSON T L, MCLARTY P K, GYURCSIK R S, et al. Single-wafer cluster tool performance: An analysis of throughput[J]. IEEE Transactions on Semiconductor Manufacturing, 1994, 7(3): 369-373.
- [19] PERKINSON T L, GYURCSIK R S, MCLARTY P K. Single-wafer cluster tool performance: An analysis of the effects of redundant chambers and revisitation sequences on throughput[J]. IEEE Transactions on Semiconductor Manufacturing, 1996, 9(3): 384-400.
- [20] ZURAWSKI R, ZHOU M. Petri nets and industrial applications: A tutorial[J]. IEEE Transactions on industrial electronics, 1994, 41(6): 567-583.
- [21] WU N. Necessary and sufficient conditions for deadlock-free operation in flexible manufacturing systems using a colored Petri net model[J]. IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews), 1999, 29(2): 192-204.
- [22] 朱清华, 伍乃骐, 滕少华. 半导体制造中多组合设备的 Petri 网建模及死锁分析[J]. 东南大学学报 (自然科学版), 2010, 40(S2): 267-271.
- [23] 潘春荣, 付锦超. 基于 Petri 网的可重入双臂组合设备故障响应策略[J/OL]. 计算机集成制造系统, 2019, 25(12): 3235-3246. DOI: 10.13196/j.cims.2019.12.025.
- [24] ZHOU M, DICESARE F. Parallel and sequential mutual exclusions for Petri net modeling of manufacturing systems with shared resources[J]. IEEE transactions on robotics and automation, 1991, 7(4): 515-527.
- [25] CHEN Y, LI Z, KHALGUI M, et al. Design of a maximally permissive liveness-enforcing Petri net supervisor for flexible manufacturing systems[J]. IEEE Transactions on automation science and engineering, 2010, 8(2): 374-393.
- [26] SHIN Y H, LEE T E, KIM J H, et al. Modeling and implementing a real-time scheduler for dual-armed cluster tools[J]. Computers in Industry, 2001, 45(1): 13-27.
- [27] WARE S, SU R. Time optimal synthesis based upon sequential abstraction and its application to cluster tools[J]. IEEE Transactions on Automation Science and Engineering, 2016, 14(2): 772-784.
- [28] DHALL S K, LIU C L. On a real-time scheduling problem[J]. Operations research, 1978, 26(1): 127-140.
- [29] BELLMAN R. Mathematical optimization techniques[M]. Univ of California Press, 1963.
- [30] BOYD S, MATTINGLEY J. Branch and bound methods[J]. Notes for EE364b, Stanford University,

- 2007, 2006: 07.
- [31] KIM H J, LEE J H, LEE T E. Noncyclic scheduling of cluster tools with a branch and bound algorithm [J]. IEEE Transactions on Automation Science and Engineering, 2013, 12(2): 690-700.
- [32] KOKASH N. An introduction to heuristic algorithms[J]. Department of Informatics and Telecommunications, 2005: 1-8.
- [33] YING K C. Scheduling identical wafer sorting parallel machines with sequence-dependent setup times using an iterated greedy heuristic[J]. International Journal of Production Research, 2012, 50(10): 2710-2719.
- [34] ZHANG F, TAO Q, YAN Y, et al. Wafer lot assignment for parallel-producing tools based on heuristic clustering algorithm[J]. IEEE Transactions on Semiconductor Manufacturing, 2022, 35(2): 300-308.
- [35] ABDEL-BASSET M, ABDEL-FATAH L, SANGAIAH A K. Metaheuristic algorithms: A comprehensive review[J]. Computational intelligence for multimedia big data on the cloud with engineering applications, 2018: 185-231.
- [36] QU P, MASON S J. Metaheuristic scheduling of 300-mm lots containing multiple orders[J]. IEEE Transactions on Semiconductor Manufacturing, 2005, 18(4): 633-643.
- [37] LIN S W, LEE Z J, YING K C, et al. Meta-heuristic algorithms for wafer sorting scheduling problems [J]. Journal of the Operational Research Society, 2011, 62(1): 165-174.
- [38] AYTUG H, BHATTACHARYYA S, KOEHLER G J, et al. A review of machine learning in scheduling[J]. IEEE Transactions on Engineering Management, 1994, 41(2): 165-171.
- [39] HONG C, LEE T E. Multi-agent reinforcement learning approach for scheduling cluster tools with condition based chamber cleaning operations[C]//2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA). 2018: 885-890.
- [40] SUERICH D, MCILROY T. Artificial intelligence for real time cluster tool scheduling: Eo: Equipment optimization[C]//2022 33rd Annual SEMI Advanced Semiconductor Manufacturing Conference (ASMC). 2022: 1-3.
- [41] KIM H J, LEE J H. Scheduling Cluster Tools for Concurrent Processing: Deep Reinforcement Learning With Adaptive Search[J]. IEEE Transactions on Automation Science and Engineering, 2024.
- [42] PETERSON J L. Petri nets[J]. ACM Computing Surveys (CSUR), 1977, 9(3): 223-252.
- [43] 李志武, 周孟初. 自动制造系统建模, 分析与死锁控制[M]. 科学出版社, 2009.
- [44] 袁崇义. Petri 网应用[M]. 科学出版社, 2013.
- [45] ROZENBERG G, THIAGARAJAN P S. Petri nets: Basic notions, structure, behaviour[J]. Current Trends in Concurrency: Overviews and Tutorials, 1986: 585-668.
- [46] LI Z, ZHOU M. Deadlock resolution in automated manufacturing systems: a novel Petri net approach

- [M]. Springer Science & Business Media, 2009.
- [47] MURATA T. Petri nets: Properties, analysis and applications[J]. Proceedings of the IEEE, 1989, 77(4): 541-580.
- [48] SIFAKIS J. Structural properties of Petri nets[C]//Mathematical Foundations of Computer Science 1978: Proceedings, 7th Symposium Zakopane, Poland, September 4–8, 1978 7. 1978: 474-483.
- [49] 牟行军, 宋文. Petri 网的活性与安全性的理论研究[J]. 西华大学学报 (自然科学版), 2006(03): 86-89+118.
- [50] 郭长友, 郑文艳, 周智刚. 利用可达性图判断 petri 网的可达性以及活性[J]. 中国科技信息, 2006(12): 147-148.
- [51] BOWDEN F D. A brief survey and synthesis of the roles of time in Petri nets[J]. Mathematical and Computer Modelling, 2000, 31(10-12): 55-68.
- [52] ZUBEREK W M. Timed Petri nets definitions, properties, and applications[J]. Microelectronics Reliability, 1991, 31(4): 627-644.
- [53] POPOVA-ZEUGMANN L, POPOVA-ZEUGMANN L. Time petri nets[M]. Springer, 2013.
- [54] KHOUSSAINOV B, NERODE A. Automata theory and its applications: vol. 21[M]. Springer Science & Business Media, 2012.
- [55] RAMADGE P J, WONHAM W M. The control of discrete event systems[J]. Proceedings of the IEEE, 1989, 77(1): 81-98.
- [56] ALUR R. Timed automata[C]//Computer Aided Verification: 11th International Conference, CAV '99 Trento, Italy, July 6–10, 1999 Proceedings 11. 1999: 8-22.
- [57] KAEHLING L P, LITTMAN M L, MOORE A W. Reinforcement learning: A survey[J]. Journal of artificial intelligence research, 1996, 4: 237-285.
- [58] WIERING M A, VAN OTTERLO M. Reinforcement learning[J]. Adaptation, learning, and optimization, 2012, 12(3): 729.
- [59] DYNKIN E B, DYNKIN E B. Markov processes[M]. Springer, 1965.
- [60] 秦智慧, 李宁, 刘晓彤, 等. 无模型强化学习研究综述[J]. 计算机科学, 2021, 48(03): 180-187.
- [61] WATKINS C J, DAYAN P. Q-learning[J]. Machine learning, 1992, 8: 279-292.
- [62] MNH V, KAVUKCUOGLU K, SILVER D, et al. Human-level control through deep reinforcement learning[J]. nature, 2015, 518(7540): 529-533.
- [63] SUTTON R S, MCALLESTER D, SINGH S, et al. Policy gradient methods for reinforcement learning with function approximation[J]. Advances in neural information processing systems, 1999, 12.
- [64] DEGRIS T, PILARSKI P M, SUTTON R S. Model-free reinforcement learning with continuous action in practice[C]//2012 American control conference (ACC). 2012: 2177-2182.
- [65] CLARKE E M. Model checking[C]//Foundations of Software Technology and Theoretical Computer

- Science: 17th Conference Kharagpur, India, December 18–20, 1997 Proceedings 17. 1997: 54-56.
- [66] MÜLLER-OLM M, SCHMIDT D, STEN B. Model-checking: A tutorial introduction[C]//Static Analysis: 6th International Symposium, SAS' 99 Venice, Italy, September 22–24, 1999 Proceedings 6. 1999: 330-354.
- [67] BAIER C, KATOEN J P. Principles of model checking[M]. MIT press, 2008.
- [68] WU N, ZHOU M. Process vs resource-oriented Petri net modeling of automated manufacturing systems[J]. Asian Journal of Control, 2010, 12(3): 267-280.
- [69] JANG B, KIM M, HARERIMANA G, et al. Q-learning algorithms: A comprehensive classification and applications[J]. IEEE access, 2019, 7: 133653-133667.
- [70] THRUN S, LITTMAN M L. Reinforcement learning: An introduction.[J]. AI Magazine, 2000, 21(1): 103-103.
- [71] LARSEN K G, PETTERSSON P, YI W. UPPAAL in a nutshell[J]. International journal on software tools for technology transfer, 1997, 1: 134-152.
- [72] BEHRMANN G, DAVID A, LARSEN K G. A tutorial on Uppaal 4.0[J]. Department of computer science, Aalborg university, 2006, 1(1): 1-48.
- [73] BEHRMANN G, FEHNER A, HUNE T, et al. Efficient guiding towards cost-optimality in UPPAAL[C]//International Conference on Tools and Algorithms for the Construction and Analysis of Systems. 2001: 174-188.

致 谢

时光荏苒，研究生阶段仿佛匆匆而过，也标志着我学生时代的落幕。在这段宝贵的求学旅程中，我经历了挑战与挫折，也收获了成长与喜悦。有人在低谷时给予陪伴，让孤独不再肆虐；有人在成功时分享喜悦，让美好倍增。你们的支持不仅丰富了我的学术与生活，也深刻影响了我的思想与成长。在此，我怀揣感激之心，向所有给予我帮助与关怀的人们致以最诚挚的谢意。

首先，我要由衷地感谢我的恩师刘鼎老师和冯雷老师。本文的完成离不开两位老师的悉心指导，他们不仅是我学术道路上的引路人，更是以自身的勤奋与严谨为我树立了榜样。老师的谆谆教诲和无私奉献让我受益匪浅，使我在科研探索中不断前行。此外，我也要感谢所有曾给予我指导和帮助的师长们，正是你们的辛勤付出和悉心栽培，成就了今天的我。

其次，我要感激我的父母。你们不仅是我的至亲，更是我人生道路上最坚实的后盾。无论顺境还是逆境，你们始终给予我无条件的关爱与支持，在我遭遇挫折时鼓励我前行，在我迷茫困惑时给予我力量。正是这份深沉的爱，让我在面对困难与挑战时更加坚定勇敢。你们用无私的付出为我创造了温暖的成长环境，这份恩情将伴随我一生，成为我最珍贵的财富。

最后，我要感谢我的同学们。求学路上，你们不仅是学术上的伙伴，更是生活中的知己。共同的学习与探索，让我们在求知的旅程中相互砥砺；彼此的陪伴与支持，让这段时光充满温暖与色彩。是你们让我在成长的道路上不再孤单，给予我前行的动力和勇气。

研究生阶段已然落幕，但这段时光中的点滴记忆，将成为我人生旅途中最宝贵的财富。再次向所有给予我帮助、支持和关怀的人们致以最诚挚的谢意！

作者简介

1. 基本情况

高霖，男，陕西宝鸡人，1999 年 9 月出生，西安电子科技大学机电学院电子信息专业 2022 级硕士研究生。

2. 教育背景

2017.09 ~ 2021.07 郑州大学，本科，专业：轨道交通信号与控制

2022.07 ~ 西安电子科技大学，硕士研究生，专业：电子信息

3. 攻读硕士学位期间的研究成果

3.1 申请（授权）专利

[1] 刘鼎, 陈玉峰, 张庚声, 王顺, 高霖. 一种晶圆加工系统中面向对象的调度方法及其系统 [P]. 陕西省:CN202410507526.2,2024-08-16.

3.2 参与科研项目及获奖

[1] XXX 项目, 项目名称, 起止时间, 完成情况, 作者贡献。

[2] XXX, XXX, XXX 等. 科研项目名称. 陕西省科技进步三等奖, 获奖日期.

